

JOIG 春合宿 1-1 リレー競技

解説：熊田順一

問題概要

異なる 3 人を選んで300mリレーのチームを作る。

3人の100m走のタイム(タイム A)を走る順に A_1, A_2, A_3 (ミリ秒)

バトンパスのタイム(タイム B) を B_1, B_2, B_3 (ミリ秒)とすると

記録は $A_1 + \max(B_1, B_2) + A_2 + \max(B_2, B_3) + A_3$ (ミリ秒)

適切に走る人と順番を決めたとき、記録は最短で何ミリ秒？



入出力例

入力例

番号	1	2	3	4
タイムA	1070	1080	1050	1020
タイムB	90	70	60	100

出力例 : 3320

例えば部員4,3,2 の順に走らせると、
記録は $1020 + \max(100, 60) + 1050 + \max(70, 60) + 1080 = 3320$ で最短になる。

小課題 1 ($N \leq 100$, 25点)

走る人の選び方や走る順番を全部調べる。

三重ループを使って、一番目・二番目・三番目に走る人を決める。
同じ人を二回以上選ばないように注意する。

計算量は $O(N^3)$

小課題1 回答例

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    scanf("%d", &N);
    vector<int> A(N), B(N);
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &A[i], &B[i]);
    }
    int ans = 1e9 + 7;
    for (int a = 0; a < N; a++) {
        for (int b = 0; b < N; b++) {
            for (int c = 0; c < N; c++) {
                if (a == b || b == c || c == a) {
                    continue;
                }
                int sum = A[a] + max(B[a], B[b]) + A[b] + max(B[b], B[c]) + A[c];
                ans = min(ans, sum);
            }
        }
    }
    printf("%d\n", ans);
}
```

小課題2 ($N \leq 3000$, 33点)

走る3人を決めた時の、記録の最小値を考える。

タイム B が短い順に人 1, 2, 3 として、

タイム A とタイム B をそれぞれ $A_1, A_2, A_3, B_1, B_2, B_3$ とすると、

- ▶ 二番目に走る人が人 1 の時、
 - ▶ 記録は $A_1 + A_2 + A_3 + \max(B_2, B_1) + \max(B_1, B_3) = A_1 + A_2 + A_3 + B_2 + B_3$
- ▶ 二番目に走る人が人 2 の時、
 - ▶ 記録は $A_1 + A_2 + A_3 + \max(B_1, B_2) + \max(B_2, B_3) = A_1 + A_2 + A_3 + B_2 + B_3$
- ▶ 二番目に走る人が人 3 の時、
 - ▶ 記録は $A_1 + A_2 + A_3 + \max(B_1, B_3) + \max(B_2, B_3) = A_1 + A_2 + A_3 + B_3 + B_3$

よって、記録の最小値は $A_1 + A_2 + A_3 + B_2 + B_3$

小課題2 ($N \leq 3000$, 33点)

タイムBが長い2人を固定した時、
2人よりタイムBが短い中でタイムAが最短の人を選べばよい。

→タイムBでソートした後に前計算しておくことで、定数時間で計算できる。

計算量は $O(N^2)$

小課題2 回答例

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    scanf("%d", &N);
    vector<pair<int, int>> X(N);
    for (auto& [a, b] : X) scanf("%d %d", &b, &a);
    sort(X.begin(), X.end());
    vector<int> mini(N);
    mini[0] = X[0].second;
    for (int i = 1; i < N; i++) mini[i] = min(mini[i - 1], X[i].second);
    int ans = 1e9 + 7;
    for (int i = 1; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
            int sum = mini[i - 1] + X[i].first + X[i].second + X[j].first + X[j].second;
            ans = min(ans, sum);
        }
    }
    printf("%d\n", ans);
}
```


補足

タイム Bの昇順に選手の順番をソート

- ▶ pair を使う

(a, b) のペアをソートすると、

- ▶ まず a の昇順にソートされ、
- ▶ a の値が同じところでは b の昇順にソートされる。

今回は、(タイム B, タイム A)のペアをソートすれば良い。

小課題3 ($A_1 = A_2 = \dots = A_N$, 10点)

小課題2 で求めた式を使うと、この時の記録の最小値は

$$A \times 3 + B_2 + B_3$$

ただし、 $A_1 = A_2 = \dots = A_N = A$ とした。

よって、選手全員のタイム B を短い順に $b_1, b_2, \dots, b_N$ とした時、答えは $A \times 3 + b_2 + b_3$

b_2, b_3 はソートすることで $O(N \log N)$ で計算できる。

最小値を求める要領で、 $O(N)$ でも計算できる。

小課題4 ($N \leq 200000$, 32点)

3人を選んだ時の記録の最小値は

$$\blacktriangleright A_1 + A_2 + A_3 + B_2 + B_3 = A_1 + (A_2 + B_2) + (A_3 + B_3)$$

タイム Bが一番短い人を固定すると、

- ▶ 選んだ人よりタイムBが長い人の中で、
- ▶ タイムA + タイムB が最も短い 2 人を選べば記録が最短になる。

タイムBでソートした後、適切に前計算をすることで、上の2人は定数時間で計算できる。

- ▶ $mi1_{N-1} = \min(A_N + B_N, A_{N-1} + B_{N-1})$, $mi2_{N-1} = \max(A_N + B_N, A_{N-1} + B_{N-1})$
- ▶ $mi1_i = \min(mi1_{i+1}, A_i + B_i)$, $mi2_i = \text{「}(mi1_{i+1}, mi2_{i+1}, A_i + B_i) \text{の中で2番目に小さい数」}$

計算量はソートがネックで $O(N \log N)$



JOIG2021/2022 春合宿 Day1 – JOIG ツアー



(JOIG Tour)

解説：米内山 匠実

問題概要

- 東西に伸びる直線上に J, O, I, G の4種類の文字が N 個置いてある。各文字には座標(西からの距離)が定まっている。
- クエリが Q 個与えられる。 j 番目のクエリは以下の通り。
 - ・ 座標 S_j と T_j が与えられる。 $S_j, 'J', 'O', 'I', 'G', T_j$ の順に巡る経路の最短距離を求めよ。J~G の各文字はどこにあるものでも良い。

制約(満点を狙う場合)

- 置いてある文字の数 $N \leq 100000$
- クエリの個数 $Q \leq 100000$
- 各文字の座標、及び始点と終点 S_j, T_j は 10^{15} 以下(とにかく**大きい**)
- 各文字は1つ以上存在
- 各文字の座標は相異なる&昇順に与えられる
- (S_j, T_j) の組み合わせは各クエリで異なる

例(入出力例1)

クエリ1

	2	S	1						
‘J’	‘O’	‘G’	‘I’	‘O’			‘G’		‘J’
1	T	1	2						

The diagram illustrates a path through a 3x10 grid. The path starts at 'S' (row 1, col 3), moves to 'O' (row 2, col 2), then to 'I' (row 2, col 4), then to 'G' (row 2, col 8), then to 'T' (row 3, col 2), and finally to 'J' (row 2, col 10). The numbers 2, 1, 1, 2, 1 are placed in the top and bottom rows along this path.

合計: 7

例(入出力例1)

クエリ2

						S	3		
‘J’	‘O’	‘G’	‘I’	‘O’			‘G’		‘J’
			1	1			5		
			2	T					

The diagram illustrates a grid-based movement problem. The grid is 3 rows by 10 columns. The second row contains the characters 'J', 'O', 'G', 'I', 'O', followed by three empty cells, 'G', and 'J'. The third row contains two empty cells, '1', '1', an empty cell, '5', and three empty cells. The first row contains six empty cells, 'S', '3', and three empty cells. The fourth row contains three empty cells, '2', 'T', and seven empty cells. Red arrows show movement paths: from 'G' (2,3) to 'I' (2,4) and 'O' (2,5) with cost 1 each; from 'G' (2,3) to 'T' (4,5) with cost 2; from 'J' (2,10) to 'G' (2,8) with cost 5; and from 'S' (1,7) to 'J' (2,10) with cost 3.

合計: 12

考え方①

- 全探索

考えられる全ての経路のうち最短のものを求めたい。

→ 経路を**全探索**！

- J, O, I, G の選び方を全て調べる

それぞれの文字の個数を N_J, N_O, N_I, N_G とすると、それらの積の通り数だけ選び方がある。

- それぞれの選び方について、経路の長さを考える

始点と終点と各文字の位置が決まるから、各経路の長さは簡単！

その経路のうちの最短を求めればOK。

考え方①

■ J, O, I, G の選び方を全て調べる

それぞれの文字の個数を N_J, N_O, N_I, N_G とすると、それらの積の通り数だけ選び方がある。

小課題1を考えると.....

全ての文字の個数は合わせて $N \leq 80$

→ 選び方は高々 20^4 通りくらい。

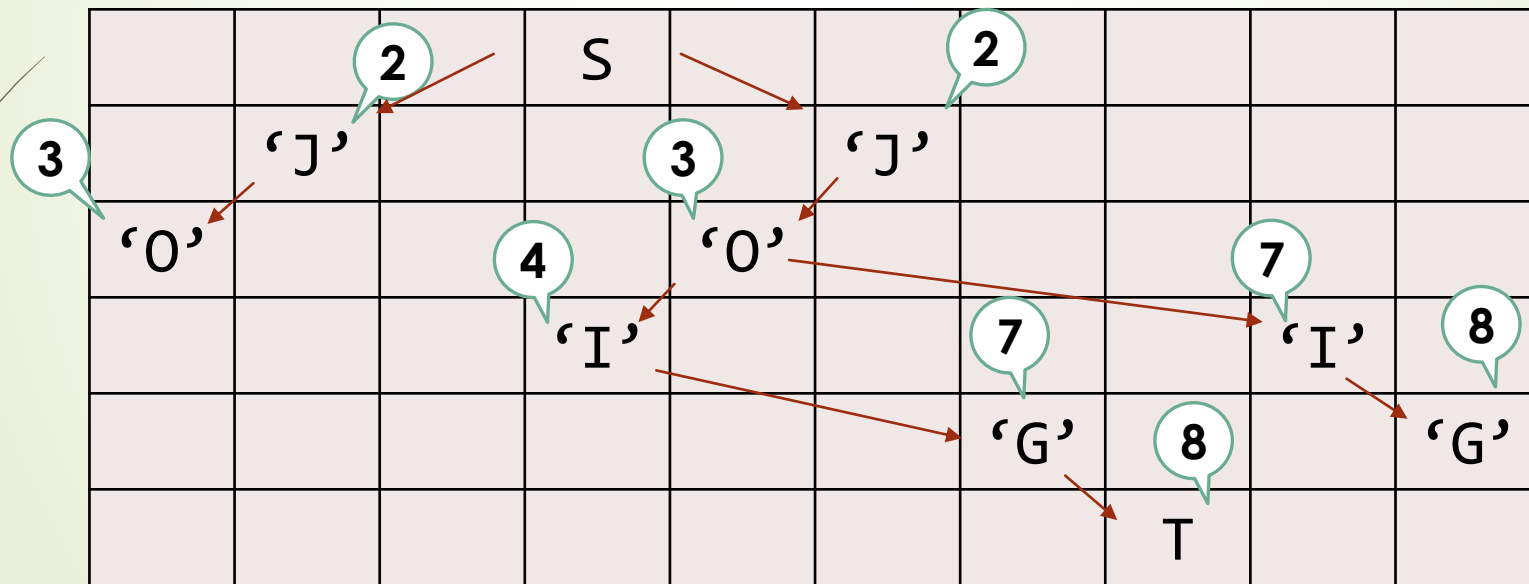
クエリのは数は $Q \leq 10$ なので、**小課題1は解ける。**



考え方②

- 全探索では、小課題2以降は間に合わない.....
→ 全てを探索する必要があるのか？
- 【重要】 求めたいのは**最短経路**のみ
- それぞれの文字にたどり着くための最短経路をメモしながら進めばいい。

考え方②



Jから
O,I,Gの
順に
見ていく

考え方②

- 文字の種類ごとに分けておく。
- J を見る時はそれぞれ s からの距離を記録
- O を見る時は、「ある J を経由した時の距離」を全ての J について探索し、最短を記録
- I を見る時は、「ある O を経由した時の距離」を全ての O について探索し、最短を記録
- G を見る時は、「ある I を経由した時の距離」を全ての I について探索し、最短を記録
- 最後に、全ての G について、そこまでの最短と T からの距離を合わせて最短を決定。

考え方②

■ 計算量？

1クエリごとに、全ての J-O、O-I、I-G の組み合わせを調べることになる。(それぞれ $O(N^2)$)

それと、 S から各 'J'、各 'G' から T までの組は $O(N)$

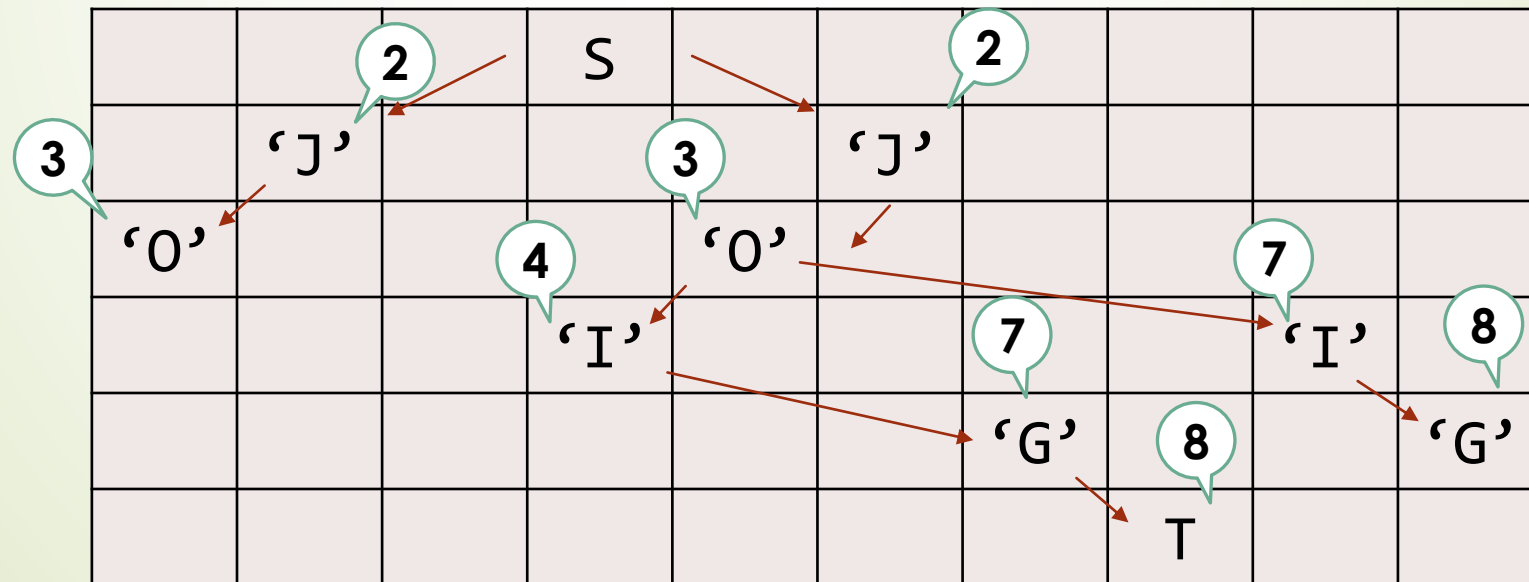
→ 合わせて $O(N^2)$

クエリ Q 個だと $O(N^2Q)$ **小課題3**まで解けた！

(J-O、O-I、I-G の組は実際は N^2 より結構少ない)

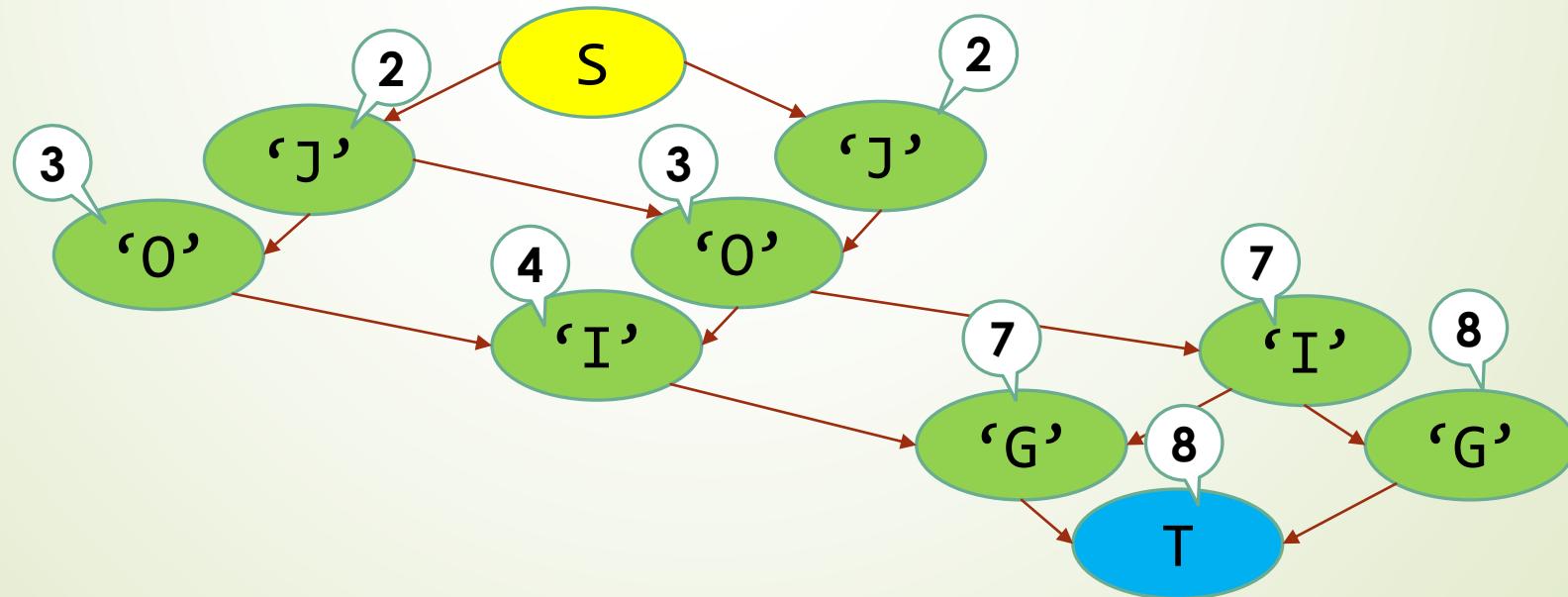
考え方③

- ここまで考えてきたのって、**グラフ**っぽい？
- J-O, O-I, I-G 間をグラフの辺と見なし、各文字+始点終点をグラフの頂点と見なせば.....



考え方③

- ここまで考えてきたのって、**グラフ**っぽい？
- J-O, O-I, I-G 間をグラフの辺と見なし、各文字+始点終点をグラフの頂点と見なせば.....



考え方③

- グラフの最短経路と言えば.....**ダイクストラ**！
- ということで、頂点と辺を設定し、ダイクストラをする。
- クエリごとに始点と終点の頂点は決める必要があるが、それ以外は同じグラフである。
- 頂点は(始点と終点を除き) $O(N)$ 個、辺は $O(N^2)$
- よって、始点と終点を決めた1回の探索では $O(N^2)$
- 結局改善にはならない.....

考え方③の補足

- 最短経路をグラフ化して考えるアプローチは、しばしば新たな解法の筋道になる。
- ダイクストラの計算量は $O(E + V \log E)$ ただし V は頂点数、 E は辺数。
- 今回は辺が多いので E がネックになった。
- グラフ化してダイクストラをするのではなく、単純にDPをすれば $O(\log E)$ の分を省ける場合もある。
- 結論：柔軟に考えよう！



考え方④

- グラフで考えると $O(N^2)$ の辺を用意することになった。

でも.....動き方ってそんなにたくさん考える必要あるの？

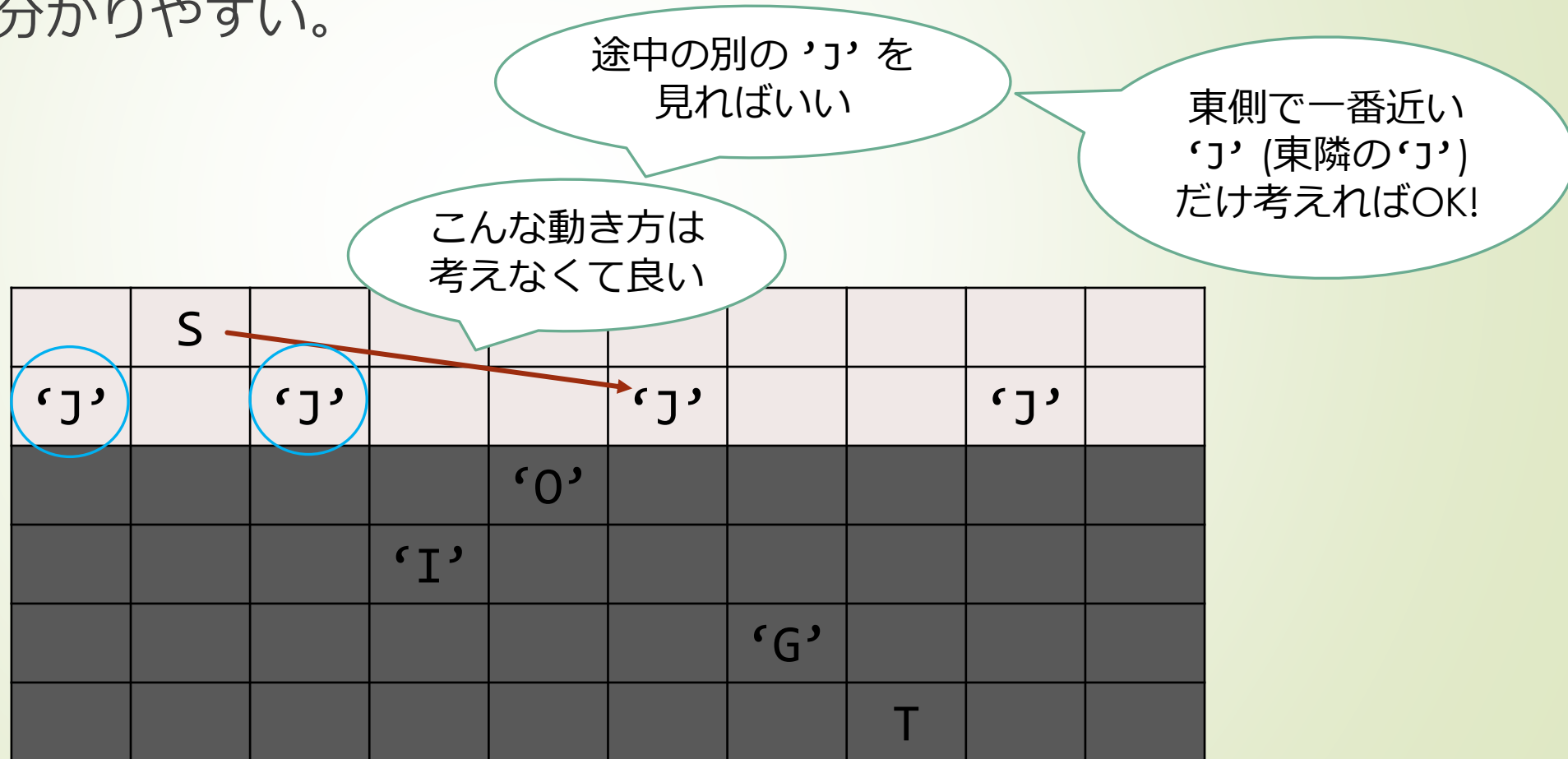
考え方④

- ➡ 例えば**小課題5**のような状況(「しか選ぶ余地がない」)で考えると分かりやすい。

[illegible]

考え方④

- 例えば**小課題5**のような状況(「J」しか選ぶ余地がない)で考えると分かりやすい。



考え方④

- 例えば**小課題5**のような状況(「しか選ぶ余地がない)で考えると分かりやすい。
- 「 s から見て西隣の「J」か、東隣の「J」を選ぶ、2つの場合だけ考えればいい」ことに気付けば**小課題5**は解ける。
- 西隣 or 東隣の「J」は二分探索(lower_bound, upper_bound)で求められる。(後で補足)
- 計算量は1クエリあたり $O(\log N)$ であり、全体で $O(Q \log N)$ なので小課題5の制約に間に合う。

考え方④

- **小課題6**も同様。T から見て西隣の 'G' か東隣の 'G' かだけ考えればいい。

	S								
'J'		'J'			'J'				
				'O'					
			'I'						
	'G'					'G'			
							T		

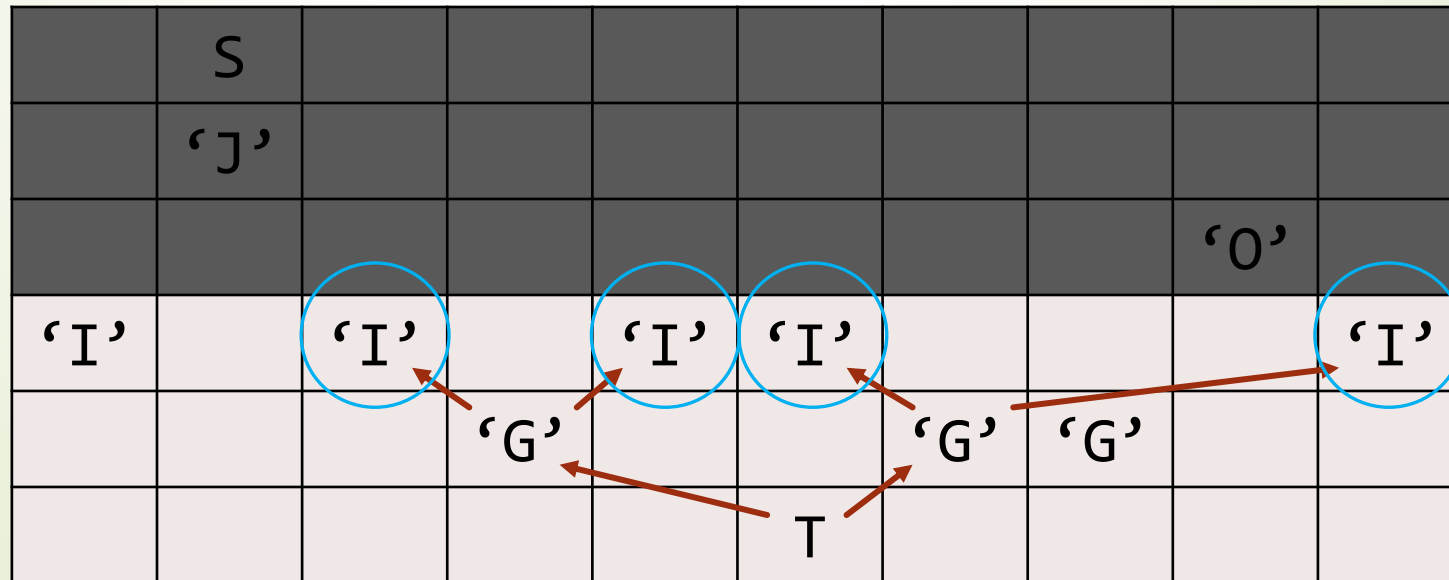
東隣がなければ無視

考え方④

- **小課題6**も同様。 T から見て西隣の ' G ' か東隣の ' G ' かだけ考えればいい。
- S 側の選び方と T 側の選び方は互いに独立なので、それぞれ最小となるものを選べばいい。
- 計算量は1クエリあたり $O(\log N)$ であり、全体で $O(Q \log N)$ なので小課題6の制約に間に合う。

考え方④

- 西隣・東隣だけでいい、という考え方を発展させる。
- **小課題7**のように、'I' も 'G' も選べる場合は、*T* から見て西隣と東隣の2つの 'G' について、さらに西隣の 'I' と東隣の 'I' を考えればいい！



考え方④

- 西隣・東隣だけでいい、という考え方を発展させる。
- **小課題7**のように、'I' も 'G' も選べる場合は、 T から見て西隣と東隣の2つの 'G' について、さらに西隣の 'I' と東隣の 'I' を考えればいい！
- 考えるべきは T からの4通りの「隣の隣」までの行き方と S からの2通りの「隣」までの行き方。
- これも計算量は1クエリあたり $O(\log N)$ であり、全体で $O(Q \log N)$ なので制約に間に合う。

考え方④

➡ ここまで来れば、同様に考えて.....

➡ 満点解法

S から両隣の ' J ' までの経路長を求め、
さらにその2つの ' J ' について、両隣の ' O ' までの経路長を求め、
さらにその4つの ' O ' について、両隣の ' I ' までの経路長を求め、
さらにその8つの ' I ' について、両隣の ' G ' までの経路長を求め、
その16通りの ' G ' について、 T までの経路長を求め、

以上16通りで最も短かったものを出力すれば良い

考え方④

- 西隣・東隣を求めるパート

→ lower_bound, upper_bound で $O(\log N)$

- 1回のクエリでは、16通り(定数倍程度)の経路しか考えず、その経路は西隣・東隣を4回求めるだけでいいから、計算量は $O(\log N)$
- Q 回のクエリでは、 $O(Q \log N)$ → **十分間に合う**



実装

■ lower_bound, upper_bound について

Vector や set などのデータ構造には、ある値以上となる最小の要素を二分探索で求める標準機能が備わっている。それが lower_bound である。また、ある値を超えるものを知りたいなら upper_bound が使える。

説明すると長くなるし、説明はネットにたくさん転がっているの
で、そちらを参照してもらいたい。

キーワード：イテレータ (位置を指し示すもの)

実装

```
//例えば、sの西隣・東隣を探索するコード
long long S;
vector<long long> J;
//配列Jには、文字Jの入っている位置が記録されているとしよう

auto it = lower_bound(J.begin(), J.end(), S);
long long L, R;
if(it == J.begin()) L = -1; //西隣が存在しない
else{
    it--;
    L = *it;
}

it = upper_bound(J.begin(), J.end(), S);
if(it == J.end()) R = LLONG_MAX; //東隣が存在しない
else R = *it;
```

実装

- ちなみに、自力で二分探索を実装するのもあり自分で書けることもそれはそれで重要。

- 二分探索を諦めても点はもぎ取れる
想定解のうち二分探索だけでできなくても、**小課題4**は取れる！

- 16通りを全探索する方法について

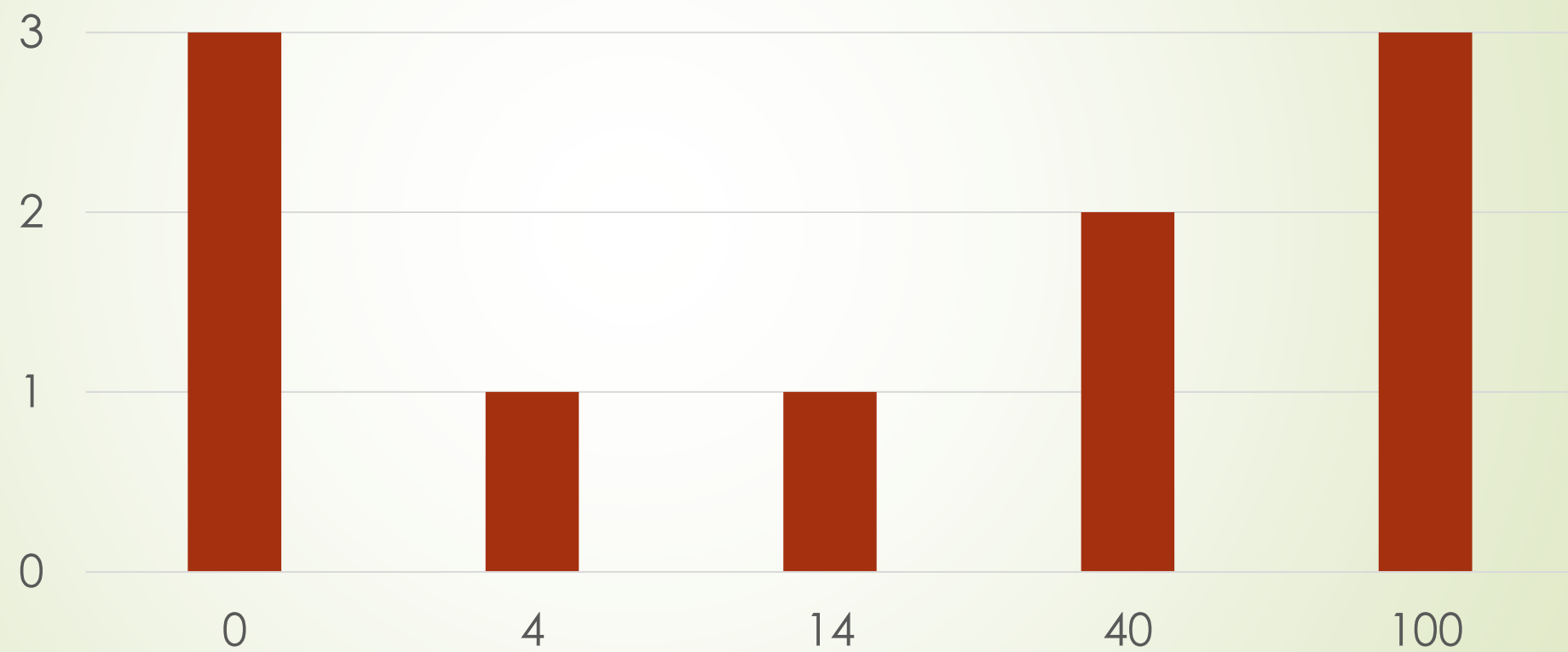
bit演算を用いることもできるが、西隣か東隣かという2択が4つあるので4重ループにしたり、for文をあきらめて愚直に16行書いても良い。



余談

- ・二分探索(lower_bound, upper_bound)を扱えるようになるろう。
- ・テクニックが追い付かないなら、部分点を取ろう。
- ・「結局一番近いところだけ調べればいい」という考え方は頻出。

得点分布





チーム戦 (Team Contest)

JOI / JOIG 2022 春合宿 Day 2 解説

東京大学 1 年 米田寛峻 (square1001)

はじめに

2 / 54

Q. 競プロで重要となる能力は何だと思いますか？

1



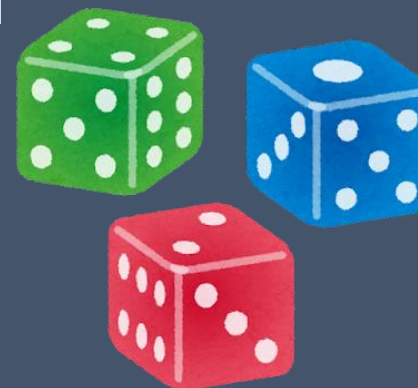
考察力

2



実装力

3



運

問題概要

3 / 54

1 N 人の選手がいます

選手 i ($1 \leq i \leq N$) の考察力は x_i 、実装力は y_i 、運は z_i



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 5



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

4 / 54

2 この中で 3 人の代表選手を選びたいです



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

5 / 54

3 このとき、チームの全員に長所があるようにしたいです
(つまり、全員に「他の 2 人より値が大きい」能力がある)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

6 / 54

4 (考察力の最大値) + (実装力の最大値) + (運の最大値) は最大いくつにできますか？



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

考察力 5 + 実装力 4 + 運 4
= 総合力 13

各小課題の制約

7 / 54

本解説では [1] → [3] → [4] → [5] → [2] → [7] の順に解説します

	配点	N の上限	X_i, Y_i, Z_i の上限
小課題 1	10 点	300	-
小課題 2	25 点	4,000	-
小課題 3	10 点	150,000	5
小課題 4	10 点	150,000	20
小課題 5	10 点	150,000	300
小課題 6	10 点	150,000	4,000
小課題 7	25 点	150,000	-

CONTENTS

8 / 54



はじめに・問題概要



小課題 1 $[N \leq 300]$ の解説



小課題 3 $[X_i, Y_i, Z_i \leq 5]$ の解説



小課題 4 $[X_i, Y_i, Z_i \leq 20]$ の解説



小課題 5 $[X_i, Y_i, Z_i \leq 300]$ の解説



小課題 2 $[N \leq 4000]$ の解説・満点解法に向けて



満点解法 $[N \leq 150000]$ の解説



おまけ

小課題 1 — $N \leq 300$

9 / 54

$N \leq 300$ のように制約が小さい場合…

全探索

をまず考えよう！

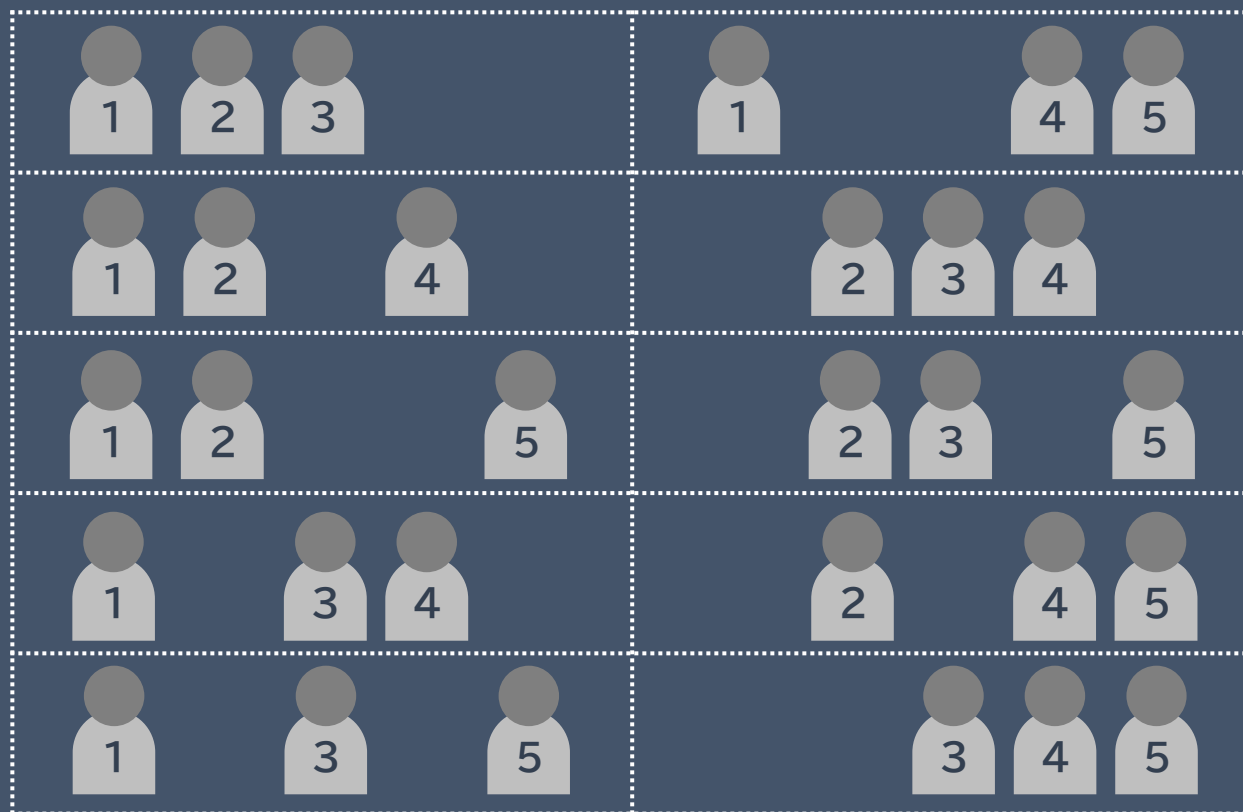
小課題 1 — $N \leq 300$

10 / 54

3 人の選び方は ${}_NC_3 = N(N-1)(N-2)/6$ 通り

これを全パターン試します

→ $N = 300$ でも 450 万通り
十分高速！



全探索の実装

11 / 54

具体的な実装について考えてみよう

選んだ 3 人が選手 i, j, k だったとする

- 選手 i に長所がある: $X_i > \max(X_j, X_k)$ または $Y_i > \max(Y_j, Y_k)$ または $Z_i > \max(Z_j, Z_k)$
- 選手 j に長所がある: $X_j > \max(X_k, X_i)$ または $Y_j > \max(Y_k, Y_i)$ または $Z_j > \max(Z_k, Z_i)$
- 選手 k に長所がある: $X_k > \max(X_i, X_j)$ または $Y_k > \max(Y_i, Y_j)$ または $Z_k > \max(Z_i, Z_j)$
- 総合力 = $\max(X_i, X_j, X_k) + \max(Y_i, Y_j, Y_k) + \max(Z_i, Z_j, Z_k)$

このように状況を整理して、実装してみよう！

全探索の実装 (1 ページ目)

12 / 54

```
#include <iostream>
#include <algorithm>
using namespace std;

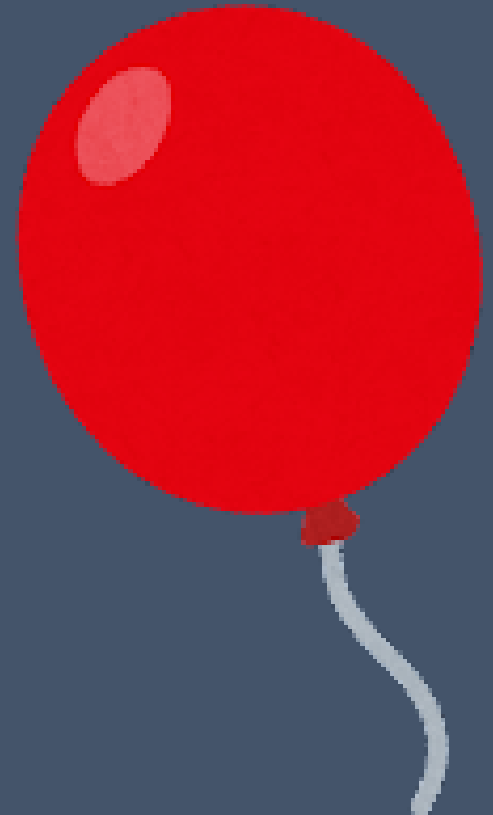
int N, X[309], Y[309], Z[309];

int get_score(int i, int j, int k) {
    if (!(X[i] > max(X[j], X[k]) || Y[i] > max(Y[j], Y[k]) || Z[i] > max(Z[j], Z[k]))) {
        return -1;
    }
    if (!(X[j] > max(X[k], X[i]) || Y[j] > max(Y[k], Y[i]) || Z[j] > max(Z[k], Z[i]))) {
        return -1;
    }
    if (!(X[k] > max(X[i], X[j]) || Y[k] > max(Y[i], Y[j]) || Z[k] > max(Z[i], Z[j]))) {
        return -1;
    }
    return max(X[i], X[j], X[k]) + max(Y[i], Y[j], Y[k]) + max(Z[i], Z[j], Z[k]);
}
```

全探索の実装 (2 ページ目)

13 / 54

```
int main() {  
    cin >> N;  
    for (int i = 0; i < N; i++) {  
        cin >> X[i] >> Y[i] >> Z[i];  
    }  
    int answer = -1;  
    for (int i = 0; i < N; i++) {  
        for (int j = i + 1; j < N; j++) {  
            for (int k = j + 1; k < N; k++) {  
                answer = max(answer, get_score(i, j, k));  
            }  
        }  
    }  
    cout << answer << endl;  
}
```



CONTENTS

14 / 54



はじめに・問題概要



小課題 1 $[N \leq 300]$ の解説



小課題 3 $[X_i, Y_i, Z_i \leq 5]$ の解説



小課題 4 $[X_i, Y_i, Z_i \leq 20]$ の解説



小課題 5 $[X_i, Y_i, Z_i \leq 300]$ の解説



小課題 2 $[N \leq 4000]$ の解説・満点解法に向けて



満点解法 $[N \leq 150000]$ の解説



おまけ

小課題 3 — $X_i, Y_i, Z_i \leq 5$

15 / 54

X_i, Y_i, Z_i の値が非常に小さいです

N が大きければ、「考察力・実装力・運がすべて同じ」人がたくさん出てきます

重要な性質

「考察力・実装力・運」がすべて同じ 2 人は
同じチームには入れない

理由

その 2 人には「自分だけの長所」が 1 つもないから

小課題 3 — $X_i, Y_i, Z_i \leq 5$

16 / 54

X_i, Y_i, Z_i の値が非常に小さいです

N が大きい時に「考察力・実装力・運」がすべて同じ人はいくつか出てきます

「全能力が同じ」人が複数いれば

1人にまとめても良い！

重要な性質

「考察力・実装力・運」がすべて同じ2人は
同じチームには入れない

理由

その 2 人には「自分だけの長所」が 1 つもないから

小課題 3 — $X_i, Y_i, Z_i \leq 5$

17 / 54

要するにということです
「同じものは 1 つ以外（探索に）
必要ない！」



考察力 = 1
実装力 = 2
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 2
運 = 5



考察力 = 3
実装力 = 1
運 = 2



考察力 = 1
実装力 = 2
運 = 5



すると、選手が $5^3 = 125$ 人以下になる

→ 計算量 $O(N^3)$ の全探索ができる！

(計算量は $O(MAX^9)$ になる)



考察力 = 1
実装力 = 2
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 2
運 = 5



考察力 = 3
実装力 = 1
運 = 2



考察力 = 1
実装力 = 2
運 = 5

CONTENTS

18 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 4 — $X_i, Y_i, Z_i \leq 20$

19 / 54

計算量を良くするためには

全探索の工夫

が重要になることもある

小課題 4 — $X_i, Y_i, Z_i \leq 20$

20 / 54

次のような「工夫した全探索」を考えましょう

工夫

考察力の最大値 a 、実装力の最大値 b 、運の最大値 c を全探索

小課題 4 — $X_i, Y_i, Z_i \leq 20$

21 / 54

(a, b, c) が決まると、以下の 3 人でチームを組むことになります

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 4 — $X_i, Y_i, Z_i \leq 20$

22 / 54

この 3 条件を満たす選手が 1 人ずついれば「この (a, b, c) でチームが作れる」

(このときの総合力は $a + b + c$ と決まる)

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

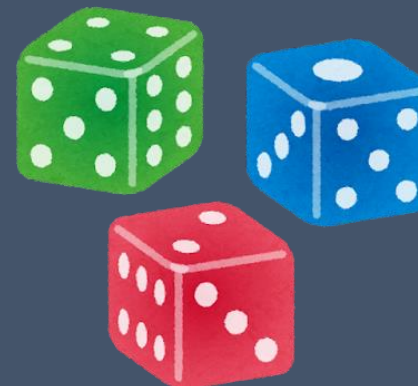
選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 4 — $X_i, Y_i, Z_i \leq 20$

23 / 54

すると、この問題が計算量 $O(MAX^6)$ で解ける

- 1 (a, b, c) の組み合わせ MAX^3 通りを全探索する
- 2 3 人の選手を探すのに $O(N)$ かけると、全体計算量は $O(MAX^3 \times N)$ になる
- 3 しかし、全部同じ能力の選手をまとめると $N \leq MAX^3$ になるので全体計算量が $O(MAX^6)$ になる

これで小課題 4 に AC !



CONTENTS

24 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 5 — $X_i, Y_i, Z_i \leq 300$

25 / 54

小課題 5 を説明する前に、「累積和」について説明します

累積和とは？

長さ N の配列 $[A_1, A_2, \dots, A_N]$ が与えられる
区間の合計 $A_l + A_{l+1} + \dots + A_{r-1}$ が計算量 $O(1)$ で求まる

累積和について知りたい方はこちら

- 「アルゴリズム×数学」p. 140～144 など

(JOIG 2022 本選 3「投票 (Voting)」を解いている方は知っているはず)

小課題 5 — $X_i, Y_i, Z_i \leq 300$

26 / 54

実は、累積和の 2 次元バージョン「2 次元累積和」もあります

1	0	1	2	0
2	1	1	0	1
1	2	1	2	0
0	1	1	2	0
1	0	1	2	1

この範囲の値の合計は $1+1+0+2+1+2+1+1+2=11$

こういうのを計算量 $O(1)$ で求めたい

小課題 5 — $X_i, Y_i, Z_i \leq 300$

27 / 54

ステップ 1

まず、横方向に累積和をとる

1	→	0	→	1	→	2	→	0
2	→	1	→	1	→	0	→	1
1	→	2	→	1	→	2	→	0
0	→	1	→	1	→	2	→	0
1	→	0	→	1	→	2	→	1



1	1	2	4	4
2	3	4	4	5
1	3	4	6	6
0	1	2	4	4
1	1	2	4	5

小課題 5 — $X_i, Y_i, Z_i \leq 300$

28 / 54

ステップ 2

次に、縦方向に累積和をとる

1	1	2	4	4
↓	↓	↓	↓	↓
2	3	4	4	5
↓	↓	↓	↓	↓
1	3	4	6	6
↓	↓	↓	↓	↓
0	1	2	4	4
↓	↓	↓	↓	↓
1	1	2	4	5



1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

小課題 5 — $X_i, Y_i, Z_i \leq 300$

29 / 54

ステップ 3

すると、4 つの値の足し算・引き算で「範囲の値の合計」が求まる

1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

$$18 + 1 - 4 - 4 = 11$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

30 / 54

ステップ 3

すると、4 つの値の足し算・引き算で「範囲の値の合計」が求まる

実は、これと同じように

「3次元累積和」もできる

1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

$$18 + 1 - 4 - 4 = 11$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

31 / 54

ここで「考察力 x 、実装力 y 、運 z の選手の数 $V[x][y][z]$ 」を 3 次元累積和すると…
「考察力重点」「実装力重点」「運重点」にできる選手の数が、それぞれ $O(1)$ で求まる

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

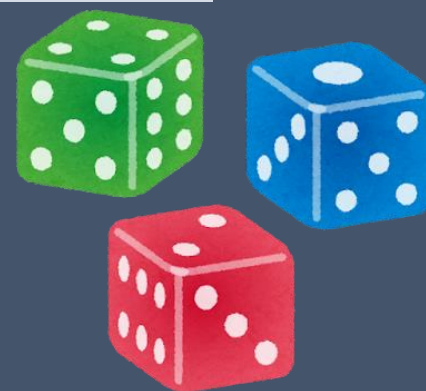
選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

32 / 54

ここで「考察力 x 、実装力 y 、運 z の選手の数 $V[x][y][z]$ 」を 3 次元累積和すると…

「考察力重点」「実装力重点」「運重点」にできる選手の数が、それぞれ $O(1)$ で求まる
すると、各 (a, b, c) に対して

計算量 $O(1)$ で判定できる！

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

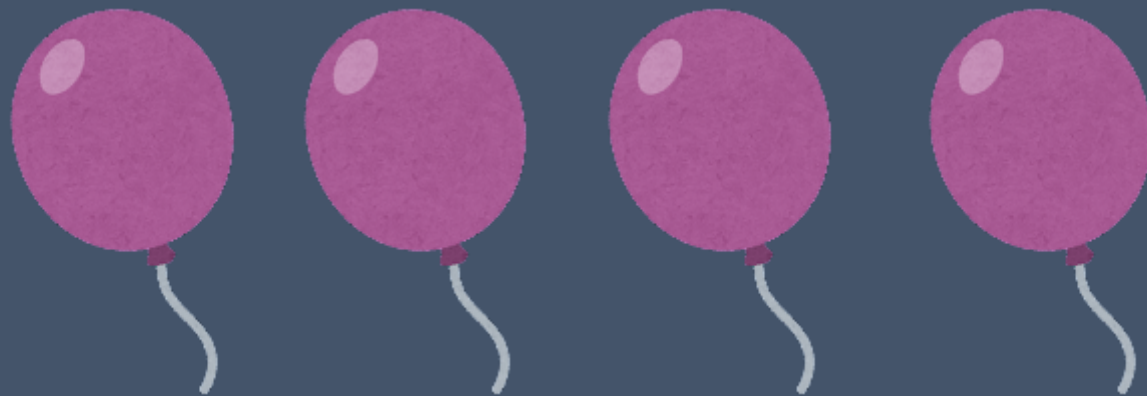
$$X_i < a, Y_i < b, Z_i = c$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

33 / 54

よって、小課題 4 の解法を「3 次元累積和」で高速化すると
全体計算量が $O(MAX^3)$ になる

→ 小課題 5 に通る！



CONTENTS

34 / 54



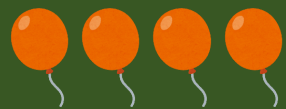
はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



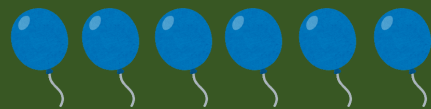
小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 2 — $N \leq 4000$

35 / 54

注意

ここからは解法がガラッと変わります

この点に注意しておいてください

満点解法への考察

36 / 54

もし「考察力が最大の人」「実装力が最大の人」「運が最大の人」が分かれていたら…？



考察力 = 3
実装力 = 1
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 4



考察力 = 5
実装力 = 4
運 = 2



考察力 = 4
実装力 = 2
運 = 3



明らかに、この 3 人を選ぶのが最適



考察力 = 3
実装力 = 1
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 4



考察力 = 5
実装力 = 4
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

37 / 54

では、もし分かれていなかったら？



「最大値のスキル」を 2 つ持ってる人は、チームには入れられない！

(理由は、他の 2 人の選手が合計 1 個しか長所を持てなくなるから)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

38 / 54

1 人消えたので、残った 4 人の中で考えると…？



「最大値のスキル」を 2 つ持ってる人は、チームには入れられない！

(理由は、他の 2 人の選手が合計 1 個しか長所を持てなくなるから)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

39 / 54

また 1 人消えたので、残った 3 人
の中で考えると…？



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



「最大値のスキル」が分かれてる
ので、この組み方が最適！



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

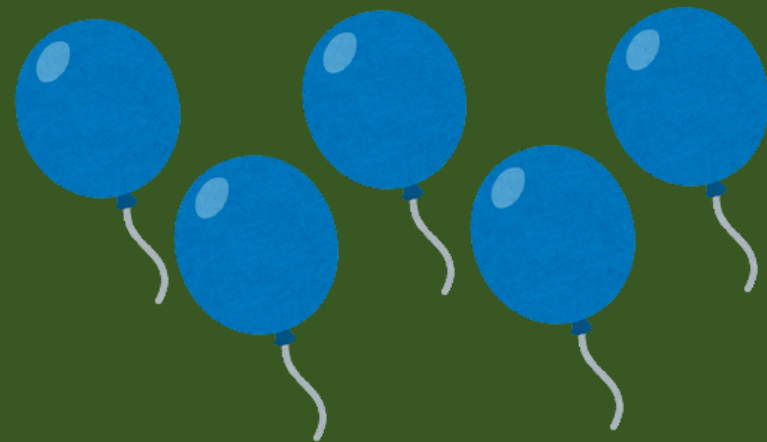
小課題 2 — $N \leq 4000$

40 / 54

このように、以下のようなアルゴリズムで最適解が求められる

- 1 『最大値のスキル』を複数持っている人がいたら消す」ことをできなくなるまで繰り返す
- 2 残った人の中で、最大値のスキルを持つ 3 人でチームを組むのが最適
誰もいなくなったら「チームを組むのが不可能」

各ステップに $O(N)$ かかるので、全体計算量は $O(N^2)$



CONTENTS

41 / 54



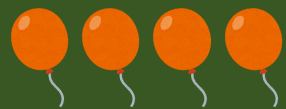
はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



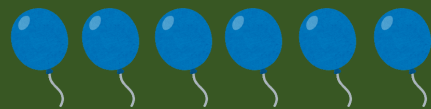
小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

満点解法 — $N \leq 150000$

42 / 54

先ほどの解法の問題点は、各ステップに $O(N)$ かかっていたことで
全体計算量が $O(N^2)$ になっていたこと

→ これを高速化したい！

満点解法 — $N \leq 150000$

43 / 54

ここで、以下の 3 つのリストを用意しておく

- N 人の選手を考察力 x_i の小さい順に並べたリスト
- N 人の選手を実装力 y_i の小さい順に並べたリスト
- N 人の選手を運 z_i の小さい順に並べたリスト

考察力 x_i

 1	 2	 5	 5	 8
---	---	---	---	---

実装力 y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

44 / 54

リストの末尾にいる 3 人をチェック

- 青の選手 (5, 4, 9) は「最大値のスキル」が 1 個だけなので OK
- 紫の選手 (8, 7, 2) は「最大値のスキル」が 2 個なので NG

考察力 X_i

 1	 2	 5	 5	 8
---	---	---	---	---

実装力 Y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 Z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

45 / 54

リストの末尾にいる 3 人をチェック

- 青の選手 (5, 4, 9) は「最大値のスキル」が 1 個だけなので OK
- 紫の選手 (8, 7, 2) は「最大値のスキル」が 2 個なので NG → 全リストから消す

考察力 X_i

 1	 2	 5	 5	 9
---	---	---	---	---

実装力 Y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 Z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

46 / 54

リストの末尾にいる 3 人をチェック

- 赤の選手 (5, 3, 3)、紫の選手 (1, 6, 2) は「最大値のスキル」が 1 個だけなので OK
- 青の選手 (5, 4, 9) は「最大値のスキル」が 2 個なので NG

考察力 X_i

 1	 2	 5	 5	
---	---	---	---	--

実装力 Y_i

 3	 4	 5	 6	
---	---	---	---	--

運 Z_i

 2		 3	 8	 9
---	--	---	---	---

現在の最大値:

(5, 6, 9)

満点解法 — $N \leq 150000$

47 / 54

リストの末尾にいる 3 人をチェック

- 赤の選手 (5, 3, 3)、紫の選手 (1, 6, 2) は「最大値のスキル」が 1 個だけなので OK
- 青の選手 (5, 4, 9) は「最大値のスキル」が 2 個なので NG → 全リストから消す

考察力 X_i

 1	 2	 5	 5	
---	---	---	---	--

実装力 Y_i

 3	 4	 5	 6	
---	---	---	---	--

運 Z_i

 2		 3	 8	 9
---	--	---	---	---

現在の最大値:

(5, 6, 9)

満点解法 — $N \leq 150000$

48 / 54

リストの末尾にいる 3 人をチェック

- 3 人全員が「最大値のスキル」が 1 個だけなので OK
- よって、総合力の最大値は $5 + 6 + 8 = 19$

考察力 X_i

 1	 2		 5	
---	---	--	---	--

実装力 Y_i

 3		 5	 6	
---	--	---	---	--

運 Z_i

 2		 3	 8	
---	--	---	---	--

現在の最大値:

(5, 6, 8)

満点解法 — $N \leq 150000$

49 / 54

このようにシミュレーションすると、各ステップが $O(1)$ でできる
最初のソートに一番時間がかかるので、全体計算量は $O(N \log N)$

→ 100 点満点が取れる！



CONTENTS

50 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

ICPC について

51 / 54

JOI/JOIG に参加できるのは高校生までですが、大学生になると ICPC があります

ICPC = 国際大学対抗プログラミングコンテスト



ICPC について

52 / 54

同じ大学から 3 人 1 組のチームで出場

3～5 時間で 10 問程度を解きます

勝ち抜くと世界大会に進めます

*ACM International Collegiate Programming Contest
Asia Regional Contest, Fukuoka, 2011-11-13*

Problem H ASCII Expression

Input: Standard Input
Time Limit: 30 seconds

Mathematical expressions appearing in old papers and old technical articles are printed with typewriter in several lines, where a fixed-width or monospaced font is required to print characters (digits, symbols and spaces). Let us consider the following mathematical expression.

$$\left(1 - \frac{4}{3^2}\right)^2 \times -5 + 6$$

It is printed in the following four lines:

```
      4   2
( 1 - ---- ) * - 5 + 6
      2
      3
```

where “- 5” indicates unary minus followed by 5. We call such an expression of lines “ASCII expression”.

For helping those who want to evaluate ASCII expressions obtained through optical character recognition (OCR) from old papers, your job is to write a program that recognizes the structure of ASCII expressions and computes their values.

For the sake of simplicity, you may assume that ASCII expressions are constructed by the following rules. Its syntax is shown in Table H.1.

ICPC について

53 / 54

ICPC では 1 問解くごとに風船がもらえます

なので、会場には風船がたくさん
これも ICPC の数ある特徴の 1 つです

大学生になったらぜひ ICPC に参加し
てみましょう！



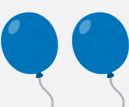
得点分布

54 / 54

JOI

100 点 

73 点 

64 点 

55 点 

17 点 

8 点 

JOIG

100 点 

26 点 

17 点 

8 点 

0 点 

コピー & ペースト 3 解説

松尾 凜太郎 / QCFium

問題概要

x, y は最初空文字列。 x を S に等しくしたい。

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

それぞれ A, B, C のコストがかかるとき、 S を入力するためにかかるコストの最小値は？

考察 1

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

どう考えても

- 操作 A で入力する文字は S に含まれる文字のみ
- x, y を S の長さ以上にするのは無駄

小課題 1

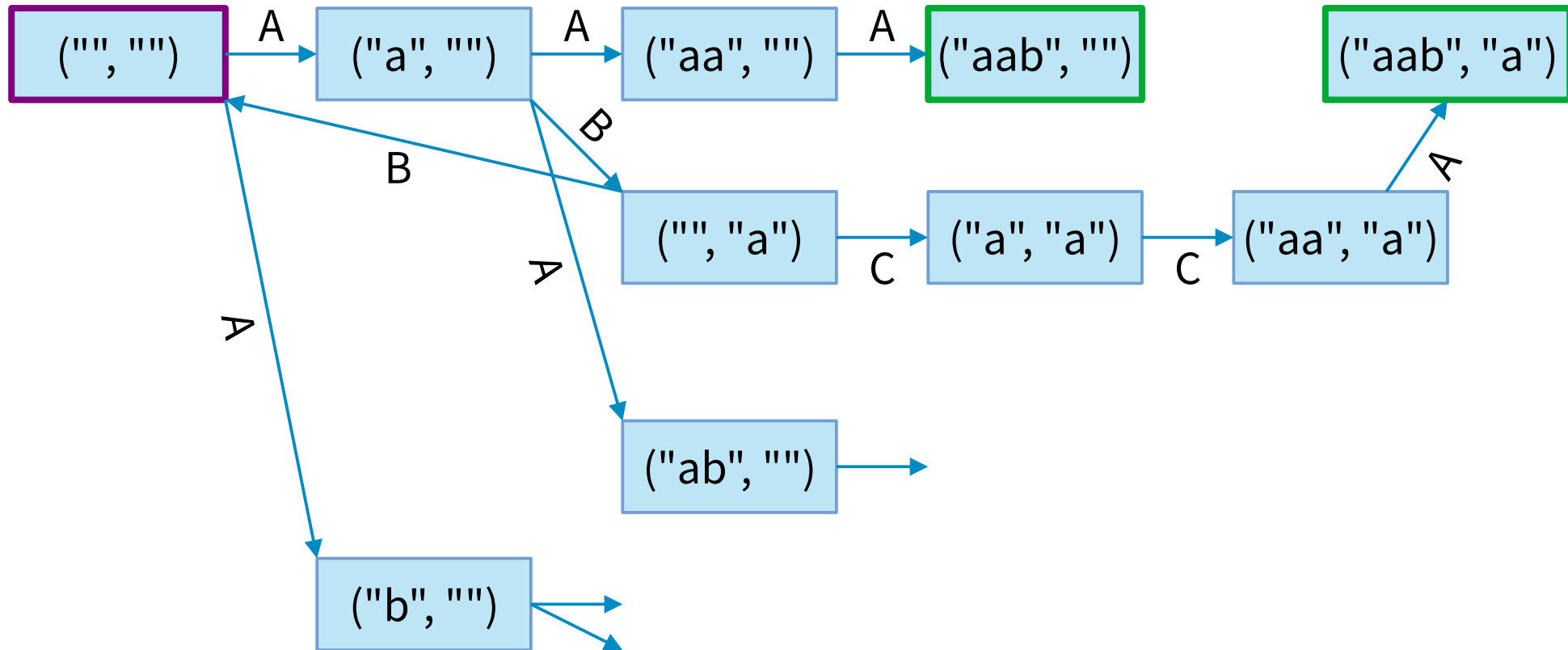
制約 : $N = 3$

N が小さい $\rightarrow$ x, y の状態を全て管理してダイクストラ法
あり得る x, y の種類はそれぞれ $1+3+9+27=40$ (S が全て異なる
文字の場合)

40×40 頂点のグラフでダイクストラ法

小課題 1

例 : $S = \text{"aab"}$



小課題 1

頂点数は 40×40 以下

各頂点からは

- 操作 A の辺が高々 3 本
- 操作 B の辺が 1 本
- 操作 C の辺が高々 1 本

→ 間に合う

小課題 2

制約：S は 'a' のみからなる

x, y として考えられるのはそれぞれ $O(N)$ 個

→ 最短経路を求めるグラフの頂点数は $O(N^2)$

各頂点からでる辺の本数は多くとも

- 操作 A の辺が 1 本
- 操作 B, C も 1 本ずつ

→ $O(N^2 \log(N))$

考察 2

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

x は空文字列 $\rightarrow y$ を捨てるのは無駄
 $\rightarrow$ 切り取り直後の y は x の部分文字列になる
 $\rightarrow y$ は S の (連続する) 部分文字列になるとしてよい

任意の時点での x も S の (連続する) 部分文字列になるとしてよい

小課題 3

制約 : $N \leq 30$

x, y は S の連続する部分列なので $O(N^2)$ 個

少し工夫して辺を張れば最短経路を求めるグラフ (頂点数 $O(N^4)$ 辺数 $O(N^5)$) を $O(N^6)$ で構築できる

- 操作 A

各 x について $x+c$ で表すことのできる S の部分文字列の位置を前計算 ($O(N^4)$) しておけばよい

- 操作 B

頂点ごとに S の部分文字列で $x+y$ に等しいものを検索 (愚直にやると $O(N^6)$)

考察 3

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

グラフを作って最短経路を求めるのは限界そう

((A,C を何回か) $\rightarrow$ B) の繰り返しと考える

$S[l:r]$: S の l 番目から $r-1$ 番目までの部分文字列 とする

$dp[l][r]$: 操作 B の直後で $x=""$, $y=S[l:r]$ である状態にするまでの最小コスト

$r-1$ の昇順に遷移していけばよい

$dp[s][t]$ から $dp[u][v]$ への遷移は？

考察 3

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

$S[s:t] : \text{bba}$

$S[u:v] : \text{aa}\text{bba}\text{bbab}$

$A(t-s) < C$ だと $S[s:t]$ をクリップボードにコピーした意味がない

$A(t-s) \geq C$ ならできるだけ限りクリップボードを使う回数を多くしたい

→ 前から貪欲に貼り付けを使っていく

考察 3

クリップボード : bab

abababaababbab



太青線部は貼り付け、それ以外は
1 文字入力するのが最適

考察 3

クリップボード : bab

abababaababbab



赤線（遷移先）が多すぎるので左のような赤線のみへの遷移を考える
自分と被らない次の青線に移ることを繰り返して更新可能

考察 3

クリップボード : bab

ababaababbab



はみ出した分は 1 文字入力するしかない
→ 赤線に遷移した後左右に 1 伸ばすの
をそれぞれコスト A で自由にすることに
すればよい

考察 3

最初に全ての s, t について $S[s:t] = S[i:i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

$dp[s][t]$ からの遷移は

- $dp[s][t+1]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最後に操作 A で $S[t]$ を入力したことにする
- $dp[s-1][t]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最初に操作 A で $S[s-1]$ を入力したことにする
- $a[s, t]$ に含まれる全ての i について :
 $a[s, t]$ のうち $i+(t-s)$ 以上の最小のものを辿っていき、更新

考察 3

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

考察 3

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の要素 j で $j \geq t$ を満たす最小のものも前計算しておく

$dp[s][t]$ からの遷移は

- $dp[s][t+1]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最後に操作 A で $S[t]$ を入力したことにする
- $dp[s-1][t]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最初に操作 A で $S[s-1]$ を入力したことにする
- $a[s, t]$ に含まれる全ての i について :
 $a[s, t]$ のうち $i+(t-s)$ 以上の最小のものを辿っていき、更新

小課題 4

クリップボード : bab

ababaababbab

=====

=====

=====

=====

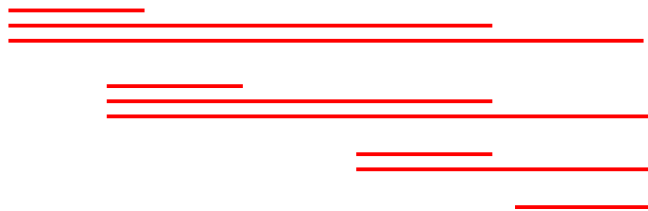
$dp[1][4], dp[3][6], dp[8][11], dp[11][14]$ をまとめて遷移
→ 1 遷移あたり $O(N)$

全体で $O(N^3)$

小課題 5

$N \leq 1000$

ababaababbab



よく見ると 1 遷移あたり
 $O(N / (\text{長さ}))$ になっているので
合計 $O(N^2 \log(N))$

小課題 6 (満点)

$$N \leq 2500$$

$O(N^2 \log(N))$ で通る場合と通らない場合がある

小課題 6 (満点)

$N \leq 2500$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る
 $a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

小課題 6 (満点)

$N \leq 2500$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

小課題 6 (満点)

$N \leq 2500$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

全部分文字列のハッシュを求めたあと長さごとに分け map で処理すると TLE が消える

Z algorithm 等で上の処理を $O(N^2)$ にすることもできる

おわり

ありがとうございました

J0IG 春合宿 2-1

メロンの出荷

解説：熊田順一

問題概要

メロンが N 個あり、 i 番目のメロンの重さは A_i

x 番目のメロンから N 番目のメロンまで、以下のように箱詰めする。

- ▶ 箱のメロンの重さの合計が L 以下の間は箱に詰める
- ▶ 箱に詰めたら L を超える場合は、そのメロンは箱に詰めないで、箱を出荷する。
 - ▶ 詰めなかったメロンは次の箱に詰める。
- ▶ N 番目のメロンを箱に詰めたら、その箱を出荷する。

$x = 1, 2, \dots, N$ それぞれについて、出荷した箱の個数と、最後に出荷した箱に入っているメロンの総重量を求めよ。

入出力例

入力例 出力例

7 100 4 10

20 4 10

80 3 10

50 2 90

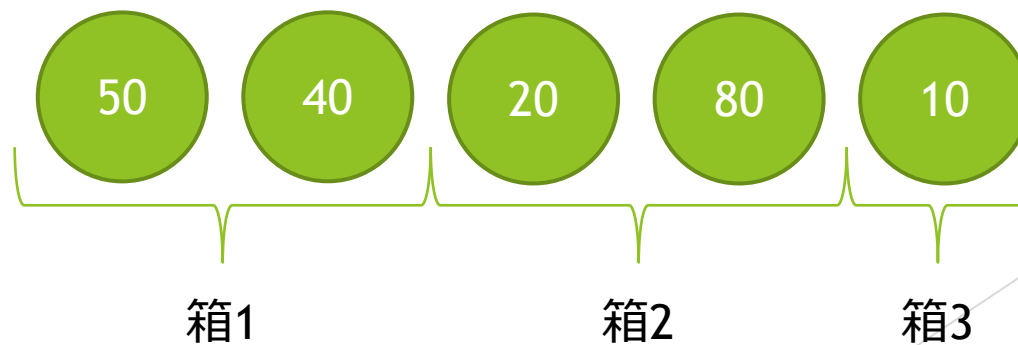
40 2 10

20 1 90

80 1 10

10

ex) $x = 3$ の時



小課題 1 ($A_1 = A_2 = \dots = A_N$, 6点)

$A_1 = A_2 = \dots = A_N = A$ とする。

また、 $[a]$ で a 以下の最大の整数を表す。(例 : $[0.1] = 0$, $[2.4] = 2$)

箱に詰められるメロンの個数は $[L/A]$ で一定。

この値を M とすると、出荷する箱の個数は $(N-x+1) / M$ 以上の最小の整数 $= [(N - x + M) / M]$ 。

また、最後に詰めるメロンの個数は、

- ▶ $(N - x + 1)$ が M の倍数の時 M 個
- ▶ そうでない時、個数は $(N - x + 1)$ を M で割ったあまり

各 $x = 1, 2, \dots, N$ について、定数時間で答えを計算できるため、全体の計算量は $O(N)$

小課題1 回答例

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, L;
    scanf("%d %d", &N, &L);
    int A;
    scanf("%d", &A); // 2つ目以降の値はいらない
    int M = L / A;
    for (int x = 1; x <= N; x++) {
        printf("%d ", (N - x) / M + 1);
        if ((N - x + 1) % M == 0) {
            printf("%d¥n", M * A);
        } else {
            printf("%d¥n", ((N - x + 1) % M) * A);
        }
    }
}
```

小課題2 ($N \leq 1000$, 21点)

各 x について、題の通りにシミュレートする。

- ▶ 出荷した箱の個数を num とし、作業中の箱にあるメロンの重さの和を now とする。
- ▶ 始め、 $\text{num} = 0$, $\text{now} = 0$ としておく。
- ▶ 重さ a のメロンを詰める時、
 - ▶ $\text{now} + a \leq L$ であれば now に a を加える。(そのまま詰める。)
 - ▶ そうでない時、 num を 1 増やし、 now の値を a に更新する。(今の箱を出荷し、新しい箱に詰める。)
- ▶ N 番目までのメロンまで繰り返した後の、 $\text{num} + 1$, now が答え。

計算量は $O(N^2)$

小課題2 回答例

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, L;
    scanf("%d %d", &N, &L);
    vector<int> A(N);
    for (auto& a : A) {
        scanf("%d", &a);
    }
    for (int i = 0; i < N; i++) {
        int num = 0, now = 0;
        for (int j = i; j < N; j++) {
            if (now + A[j] <= L) now += A[j];
            else {
                num++;
                now = A[j];
            }
        }
        printf("%d %d\n", num + 1, now);
    }
}
```

小課題3 (出荷する箱の数は 10 個以下, 29点)

i 番目のメロンから始めたら、j 番目のメロンを入れる直前で箱を出荷する

$\Leftrightarrow j$ は $A_i + A_{i+1} + \dots + A_j > L$ を満たす最小の i 以上の整数

$\Leftrightarrow j$ は $(A_1 + A_2 + \dots + A_j) - (A_1 + A_2 + \dots + A_{i-1}) > L$ を満たす最小の整数

よって、 $sum_i = A_1 + A_2 + \dots + A_{i-1}$ ($i = 0$ の時 $sum_i = 0$) とすると、

1. $sum_{j+1} > sum_i + L$ を満たす最小の j を二分探索で探す。

1. なければ、 N まで箱に詰めて終了。

2. あれば、 $j - 1$ 番目のメロンまで箱に詰めて、 j 番目のメロンから同様に調べていく。

とすることで、 $O(NM \log N)$ で計算できる。

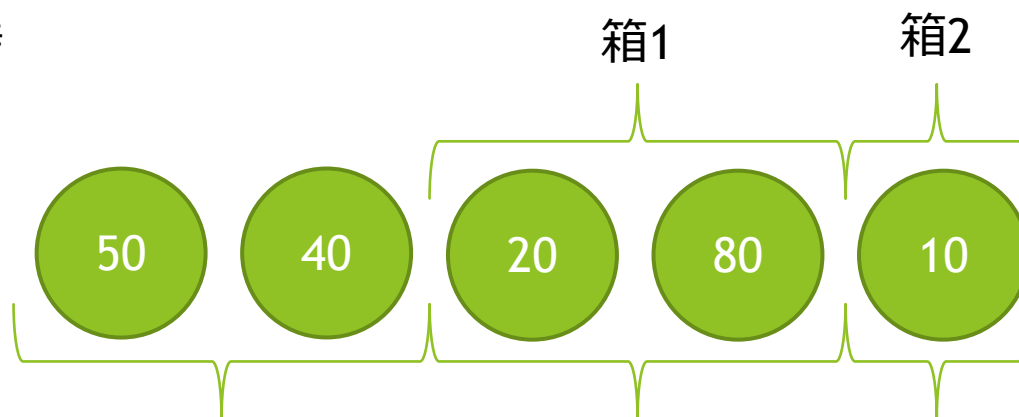
ただし M は出荷する箱の個数。

実装する際は、オーバーフローに注意。(sum の値は32bit 整数に収まらない場合がある。)

小課題4 導入

何か気づくことはありませんか？

$x = 5$ の時



$x = 3$ の時

箱1

箱2

箱3

小課題4 (箱に詰めるメロンは最大10個、33点)

メロンを詰め切った次のメロンから始めた場合と状況がよく似ている→ DP で解ける。

- ▶ $dp1[i] := x=i$ とした時の、出荷した箱の個数
 - ▶ $dp2[i] := x=i$ とした時の、最後に出荷した箱に入っていたメロンの総重量
- とすると、

i 番目のメロンから N 番目のメロンまで一つの箱に詰められる時、

- ▶ $dp1[i] = 1, dp2[i] = A_i + A_{i+1} + \dots + A_N$

i 番目のメロンから始めて、次の箱に最初に詰めるのが j 番目のメロンの時、

- ▶ $dp1[i] = dp1[j] + 1, dp2[i] = dp2[j]$

よって、 i の値を $N \rightarrow 1$ と移動させ、最初の箱だけシミュレーションすることで、 $O(NC)$ で解ける。

ただし C は一つの箱に詰めたメロンの個数の最大値。

小課題5 ($N \leq 200000$, 11点)

小課題3 の解法と 小課題4 の解法を組み合わせる。

小課題4 では次の箱に最初に入れるメロンのインデックスを愚直に求めている。
これを小課題3 の二分探索に置き換えることで、高速に計算する。

後は小課題4 でやった通りにすることで、時間内に計算することができる。

計算量は $O(N \log N)$

小課題3 と同様、オーバーフローに注意。

回答例

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int N;
    ll L;
    scanf("%d %lld", &N, &L);
    vector<ll> A(N), sum(N + 2), dp1(N + 1), dp2(N + 1);
    for (int i = 0; i < N; i++) scanf("%lld", &A[i]);
    sum[0] = 0;
    for (int i = 0; i < N; i++) sum[i + 1] = sum[i] + A[i];
    sum[N + 1] = (ll)1e18;
    dp1[N] = 0;
    dp2[N] = -1;
    for (int i = N - 1; i >= 0; i--) {
        // sum[fl] > sum[i] + L を満たす最小の fl を計算
        int fl = upper_bound(sum.begin(), sum.end(), sum[i] + L) - sum.begin();
        dp1[i] = dp1[fl - 1] + 1;
        // 出荷する箱が一つかどうかで場合分け
        dp2[i] = (dp1[i] > 1 ? dp2[fl - 1] : sum[fl - 1] - sum[i]);
    }
    for (int i = 0; i < N; i++) printf("%lld %lld\n", dp1[i], dp2[i]);
}
```

小課題5 の別解

小課題4 で愚直に求めていた j は、尺取法で全体で $O(N)$ で計算できる

- ▶ ひとつ前で求めた j から、合計が L 以下になるまでシミュレーションをする。

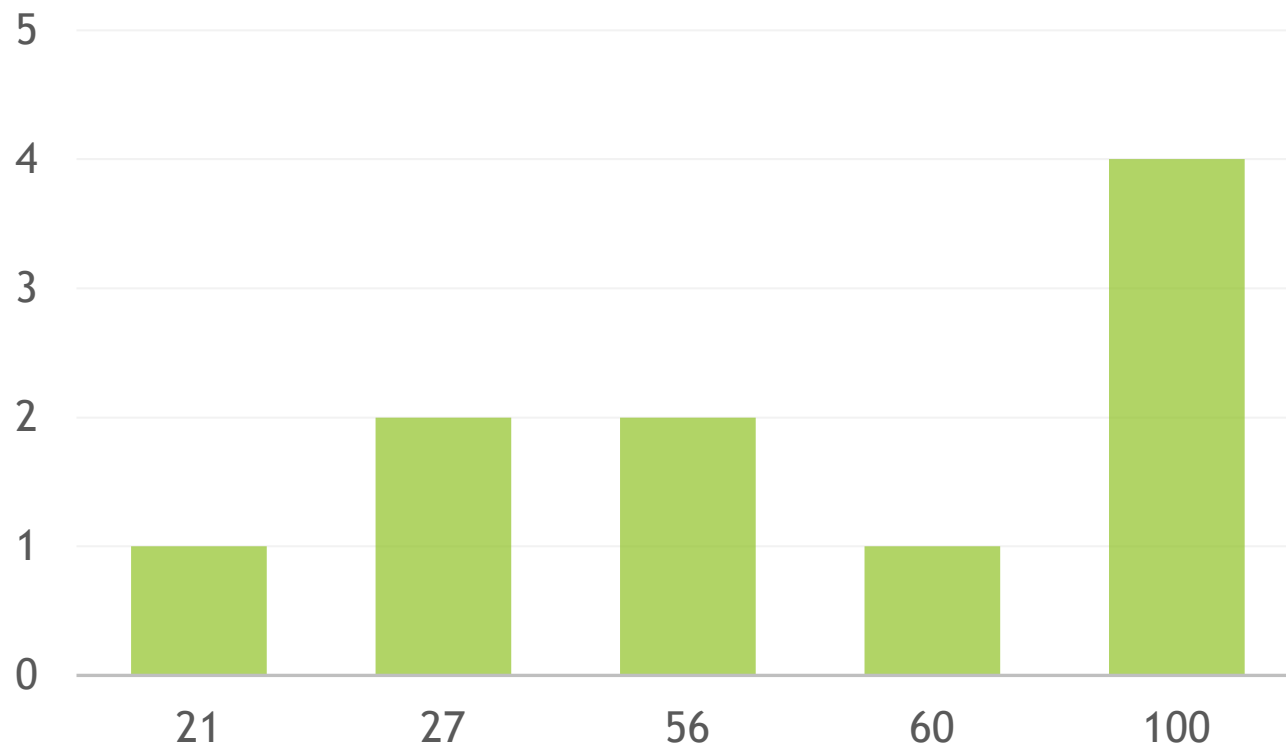
全体として計算量 $O(N)$ で計算することも可能

回答例

尺取法

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, L;
    scanf("%d %d", &N, &L);
    vector<int> A(N), dp1(N + 1), dp2(N + 1);
    for (int i = 0; i < N; i++) scanf("%d", &A[i]);
    int j = N;
    int sum = 0;
    dp1[N] = dp2[N] = 0;
    for (int i = N - 1; i >= 0; i--) {
        sum += A[i];
        while (sum > L) {
            j--;
            sum -= A[j];
        }
        dp1[i] = dp1[j] + 1;
        dp2[i] = (j == N ? sum : dp2[j]);
    }
    for (int i = 0; i < N; i++) printf("%d %d\n", dp1[i], dp2[i]);
}
```

得点分布 (スライドだけ)



JOIG 2021/2022 春合宿 競技2 2問目

Airport 解説

大佐 健人

問題概要

- 空港に、 N 本の滑走路がある
- M 個の着陸計画がある（使う滑走路は自由に決められる）
- 合間を縫って、 T 分間でできるだけ多くの飛行機を離陸させる
- ただし、1 回の離陸で K 分間、着陸で L 分間 1 つの滑走路を占有する

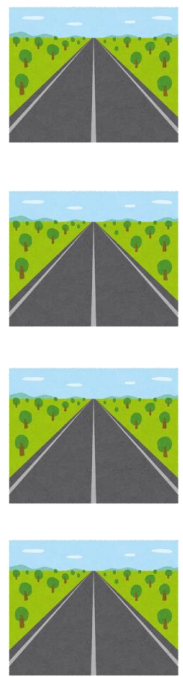
問題概要

- 空港に、 N 本の滑走路がある



問題概要

- 空港に、 N 本の滑走路がある
- M 個の着陸計画がある（使う滑走路は自由に決められる）



時刻 0



時刻 T

問題概要

- 空港に、 N 本の滑走路がある
- M 個の着陸計画がある（使う滑走路は自由に決められる）



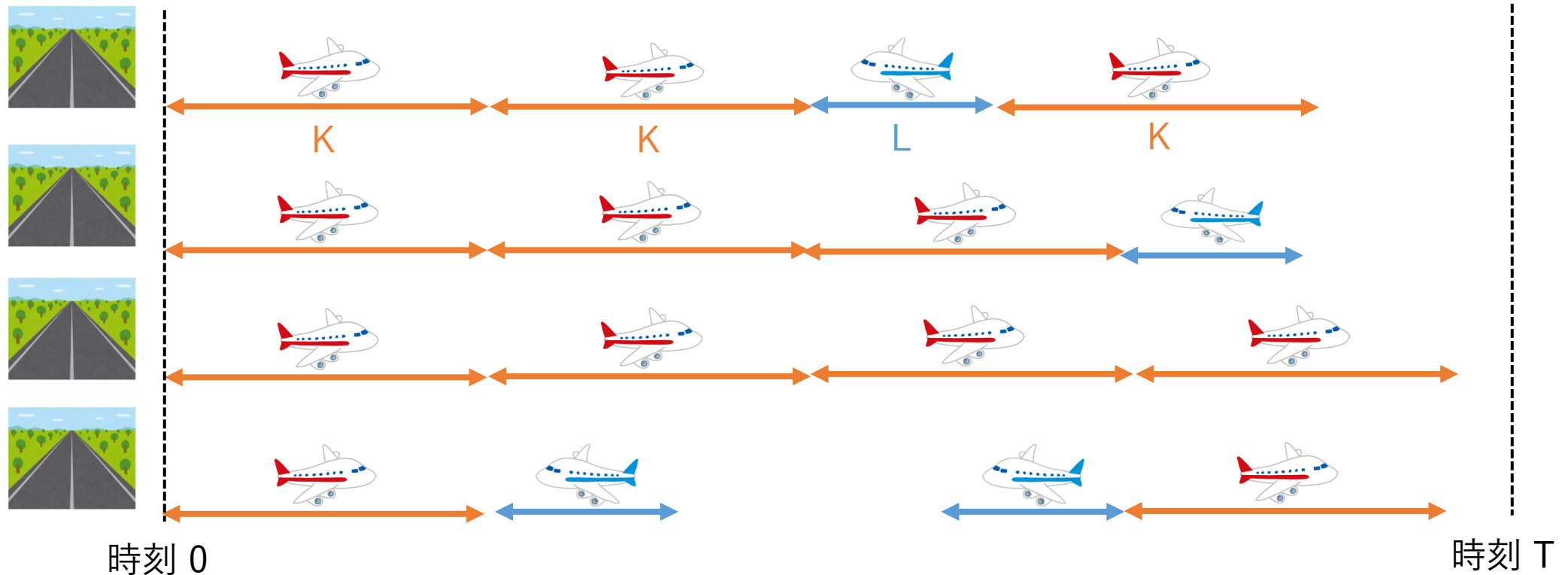
時刻 0



時刻 T

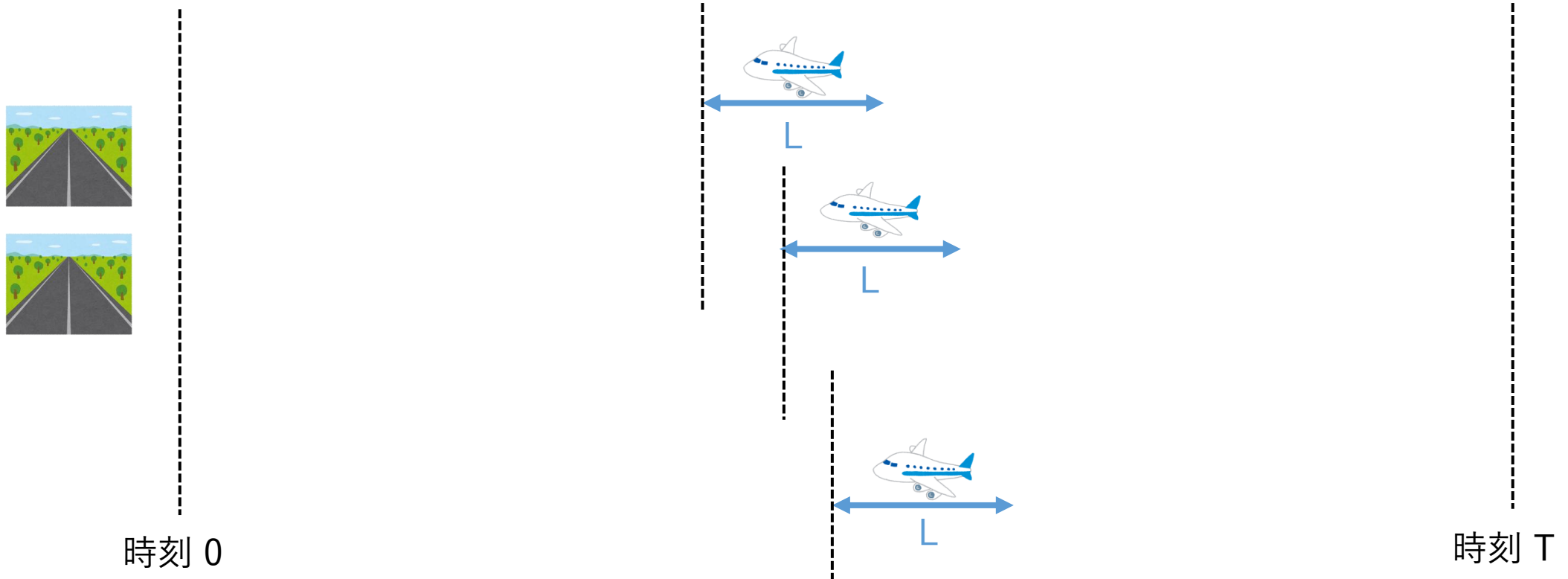
問題概要

- 空港に、 N 本の滑走路がある
- M 個の着陸計画がある
- 合間を縫って、 T 分間でできるだけ多くの飛行機を離陸させる



問題概要

- 下図のような場合は、着陸する時間がどうしても被るので、スケジュールを立てられない（-1 を出力）

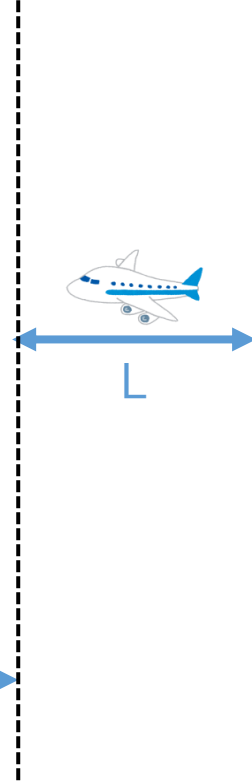
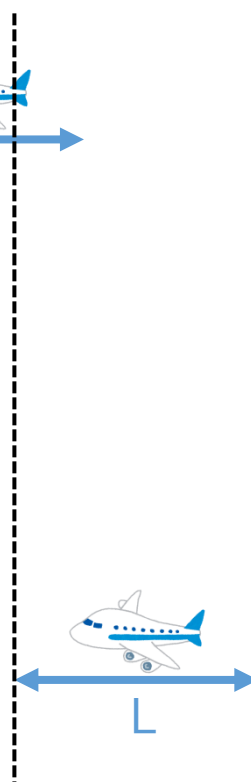
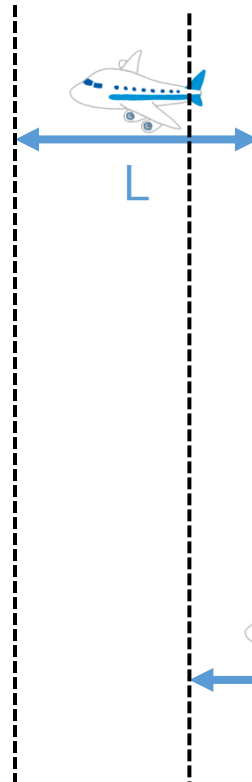
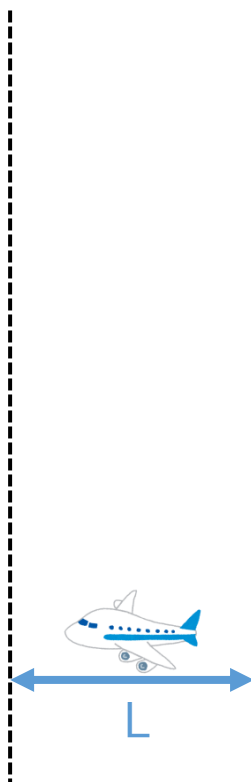


基本的な考え方

①まず、各着陸でどの滑走路を使うかを定める



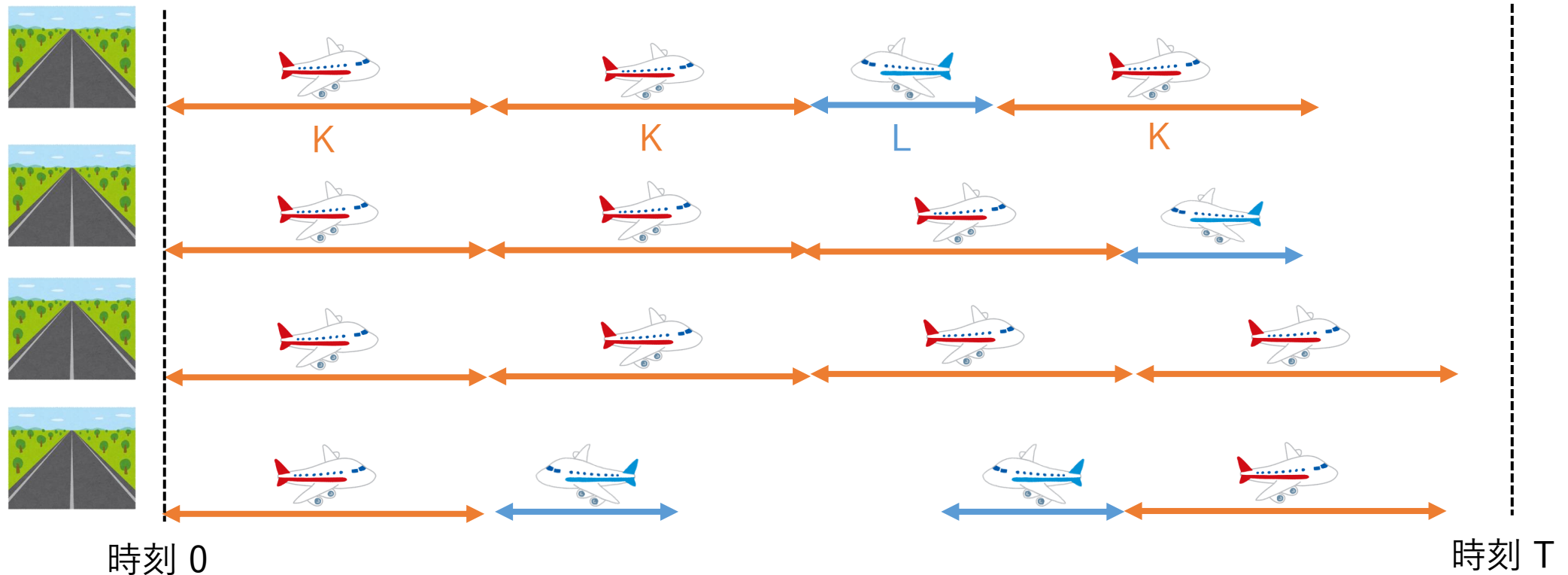
時刻 0



時刻 T

基本的な考え方

- ①まず、各着陸でどの滑走路を使うかを定める
- ②そのもとで、最大どれだけ離陸させられるかを計算

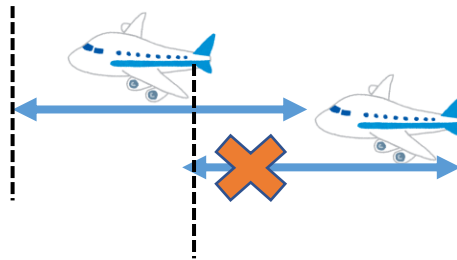


小課題 1

- $N = 1$
 - 滑走路は 1 つなので、どの滑走路を使うか気にする必要はない
 - とにかく離陸数を多くすることを考える
-
- A_i の番号の付けかたに意味はないので、まずソートして良い
 - 以降、 $A_1 \leq A_2 \leq \dots \leq A_M$ とする

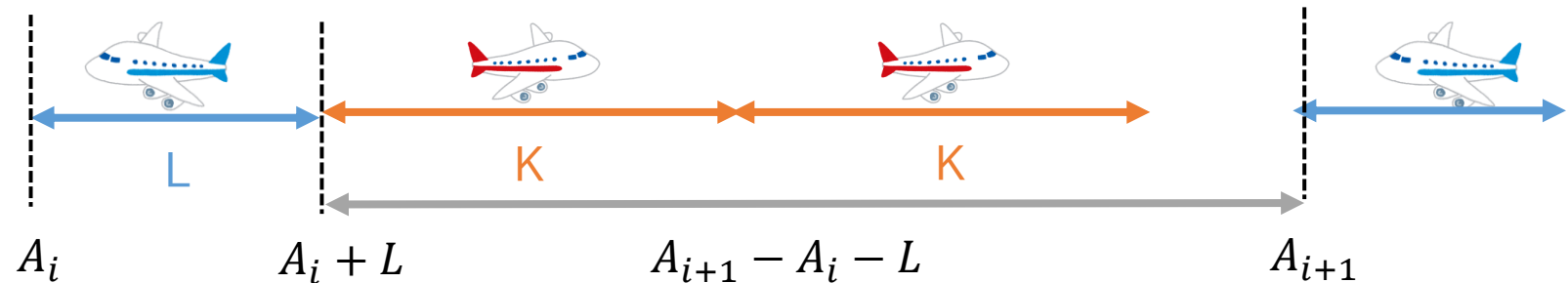
小課題 1：-1 になる条件

- $A_{i+1} - A_i < L$ なる i が存在すると NG
- 着陸している最中に次の飛行機が着陸を始めてしまうため



小課題 1

- -1 でないとき、着陸の合間に $A_{i+1} - A_i - L$ 分間の空き時間がある
- Q. この間にどれだけ離陸させられる？
- A. $\frac{A_{i+1} - A_i - L}{K}$ の小数点以下を切り捨てた数



小課題 1

- 答えは、すべての i についての $\frac{A_{i+1}-A_i-L}{K}$ (の切り捨て) の和
- A のソートがボトルネックで、計算量は $O(M \log M)$

- 7 点

小課題 2

- $N = 2, M \leq 20$
- 各飛行機について、どちらの滑走路を使うかを決める必要がある
- 着陸する飛行機だけでも決められないか？

小課題 2：考察

- M 機の飛行機について、滑走路の選び方は 2 通りあるので、全体で 2^M 通り
- 全ての選び方について、そのときの離陸数の最大値を求める
 - -1 になる場合 (同じ滑走路で着陸が重なる) はちゃんと除く
- 小課題 1 と同じように、(空き時間)/ K を切り捨てた値の和を 2 つの滑走路について計算
- その合計が離陸数

小課題 2：計算量

- 滑走路の選び方を決めるのに $O(2^M)$
 - 滑走路ごとに離陸数の最大値を求めるのに $O(NM)$
 - 全体で $O(2^M NM)$ で、 $M \leq 20$ なので間に合う ($2^{20} \approx 10^6$)
-
- 12 点

小課題 2：実装

- 2^M 通りの全探索は、2 進数を使った方法が有名
- 飛行機 i が滑走路 1 を使うなら i 桁目を 0、滑走路 2 を使うなら i 桁目を 1 とした 2 進数で状態を表現する
- 10 進数で 0 から $2^M - 1$ まで探索すれば、全状態を探索できる
- いわゆる「bit 全探索」

小課題 3

- $N = 2, K = 2, L = 2$
- 偶奇が重要な小課題？

小課題 3： -1 の場合

- 明らかに NG な場合： $A_i = A_{i+1} = A_{i+2}$



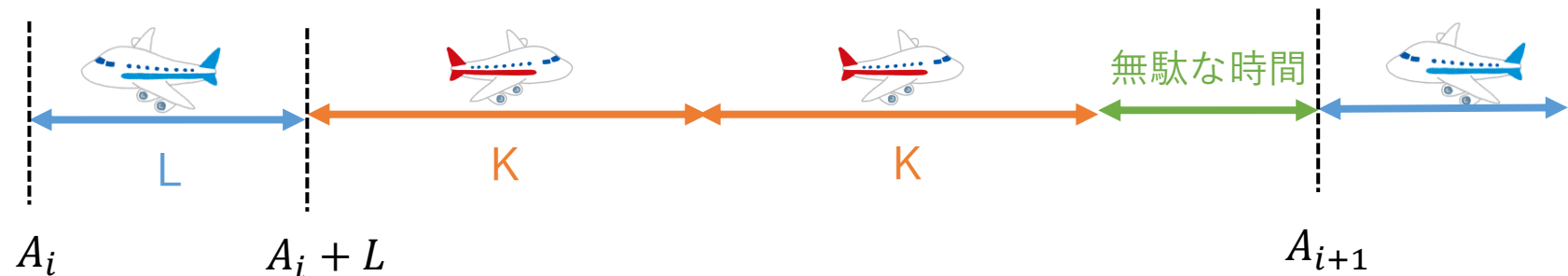
- これも NG： $A_i + 1 = A_{i+1} = A_{i+2}$ または $A_i + 1 = A_{i+1} + 1 = A_{i+2}$



- つまり、長さ 2 の区間の中に A_i が 3 つ以上あると NG
 - 逆に、そうじゃなかったら OK であることがいえる

考え方の転換

- (空き時間)/ K を切り捨てた数だけ離陸できるということは、切り捨てられた分の時間が無駄になっている
- 無駄な時間の和を最小化できれば、答えが最大化できる
- (無駄な時間) = 空き時間を K で割った余り をできるだけ小さくすることを考えよう



小課題 3： A_i が相異なる場合

- 無駄な時間：空き時間が奇数のとき 1、偶数のとき 0 になる
→できるだけ空き時間を偶数にしたい
- A_i が偶数のものを滑走路 1 に割り当てる
- A_i が奇数のものを滑走路 2 に割り当てる
- すると、同じ滑走路内での空き時間（端を除く）は必ず偶数！
 - $A_i + L$ は $L = 2$ より A_i と偶奇が同じで、かつ (偶数) - (偶数) = (偶数)、
(奇数) - (奇数) = (偶数) だから

小課題 3： A_i が相異なる場合

- 空き時間が偶数、かつ $K = 2$ より、空き時間に離陸を詰め込める
 - 無駄な時間が端以外に生まれず、これが最適になる
 - 無駄な時間をこれ以下にすることができないことが示せる
- A_i に被りがある場合も、この戦略を応用すれば OK

小課題 3

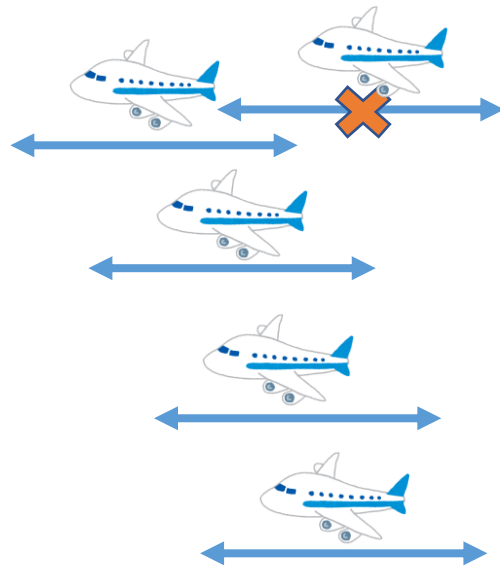
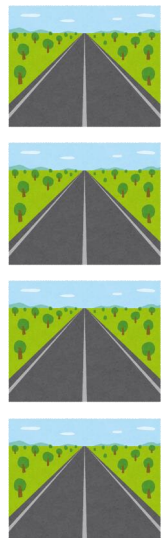
- A_i が被っている場所は他にやりようがないので、滑走路を両方使う
- その他の場所は、 A_i の偶奇で滑走路を分ける
- 実は、これが離陸数を最大にする戦略
 - 滑走路を同時刻に両方使うと滑走路 1 と滑走路 2 が等価になり、時刻 0 と同じ状況になる
- この通りに滑走路を割り振って計算すれば、 $O(M)$ で答えが求まる
- 17 点

小課題 4

- $N = 2$
- 今までの小課題で分かったこと
 - 着陸する滑走路が決まると、空き時間を K で割って切り捨てたものの和が答えになる→空き時間を K で割った余りが無駄
 - $K = 2, L = 2$ なら 長さ 2 の区間に 3 つ以上 A_i があると NG
 - $K = 2, L = 2$ なら A_i の偶奇を滑走路内でできるだけそろえると良い
- これを総動員しよう

小課題 4：-1 になる条件

- $N = 2, K = 2, L = 2$ のとき、長さ 2 の区間の中に A_i が 3 つ以上あると NG だった
- 一般の場合では、長さ L の区間の中に A_i が $N + 1$ 個以上あると NG という条件になり、 $O(M)$ で判定できる



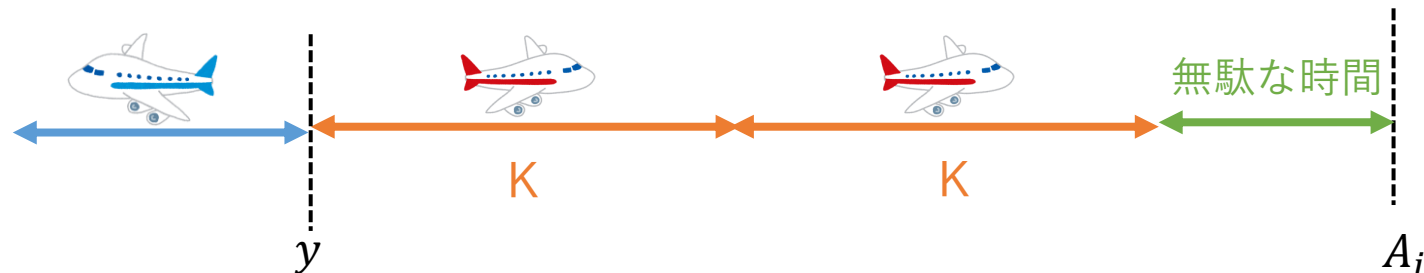
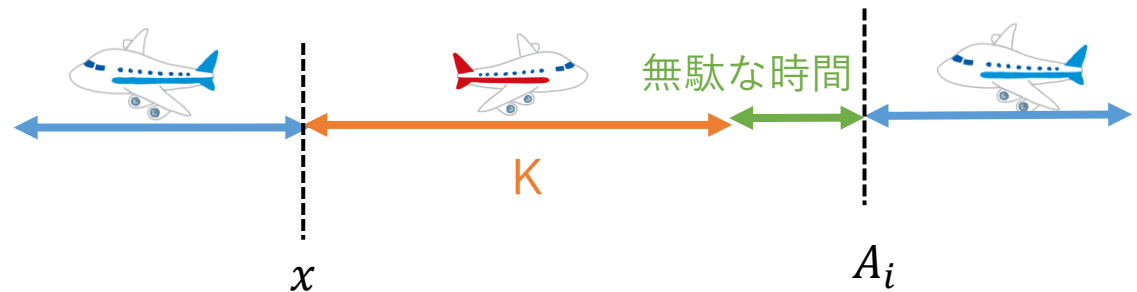
←どこにも着陸できない

小課題 4

- 空き時間を K で割った余りの和をできるだけ小さくしたい
 - 小課題 3 では、同じ滑走路ではなるべく A_i の偶奇を同じにすればよかった
- 滑走路 1, 2 で最後に着陸を終えた時間を x, y とする
- 次の飛行機が時刻 A_i に着陸するとき、生じる無駄な時間は？
 - 滑走路 1 に着陸するとき $(A_i - x + K) \% K$
 - 滑走路 2 に着陸するとき $(A_i - y + K) \% K$

小課題 4

- $(A_i - x + K) \% K$, $(A_i - y + K) \% K$ を計算することで、どちらに着陸すれば次に生じる無駄な時間が小さくなるかわかる
- つまり、毎回「生じる無駄な時間が最も小さい滑走路を選ぶ」という貪欲法ができる



小課題 4

- 普通、貪欲法は最終的に最適になるとは限らない（注意！）
 - そういう時は動的計画法が使われる
- ただし、**この問題では**貪欲法で最適解を出せる
- 各滑走路で最後に着陸を終えた時間を持てば、 $O(NM)$ で実装できる
- $12 + 17 + 28 = 57$ 点
- 貪欲法は正当性を確かめるのが大事（今回はちょっと難しい）

貪欲法の正当性

- 証明の概要

- 滑走路 1,2 で最後に着陸を終えた時間を x, y とする
- 滑走路 1 を選ぶと、最後に着陸を終えた時間は (滑走路 1, 滑走路 2) = $(A_i + L, y)$ となる
- 滑走路 2 を選ぶと、最後に着陸を終えた時間は (滑走路 1, 滑走路 2) = $(x, A_i + L)$ となるが、これは $(A_i + L, x)$ と等価
- 貪欲法だと滑走路 1 を選ぶことになるとする $((A_i - x + K) \% K \leq (A_i - y + K) \% K)$
- このもとで、任意の z について、
 $(A_i - x + K) \% K + (z - y + K) \% K \leq (A_i - y + K) \% K + (z - x + K) \% K$ がいえる
- よって、**滑走路 2 を選んでも得になることはない**ので、滑走路 1 を選ぶのが最適

- 一般の N の場合も同様に示せるので、以降の小課題でも貪欲法が使える

小課題 4 別解

- $N = 2$ ならば、 $dp[i][j]$: 滑走路 1,2 で最後に着陸したのが飛行機 i, j のときの離陸数の最大値 という DP ができる
- これは $O(M^2)$ かかるが、Segment Tree で $O(M \log N)$ に高速化でき、小課題 4 が通る
- ただ、この解法は $N = 2$ 限定なので、満点には繋がらない…
 - この解法自体けっこう難しい

小課題 5

- $N \leq 100$
- 小課題 4 を、一般の N に応用するだけ
- 各滑走路について、「最後に着陸で使ったときの終了時刻を K で割った余り」を保存した配列をつくる
- 最も無駄な時間が少なくなる滑走路を $O(N)$ で選ぶ
 - ただし、着陸時間が被るものを除くようにする
- $O(NM)$ で解けた
- 75 点

満点解法

- $N \leq 10^5$
- 小課題 5 の解法を高速化したい
- 「最も無駄な時間が少なくなる滑走路」を高速に求めたい
→ 二分探索！
- ただし、二分探索はソートされていないとできないので、工夫する必要がある
 - 終了時刻 % K の配列は中身の大きさ順に並んでないことに注意

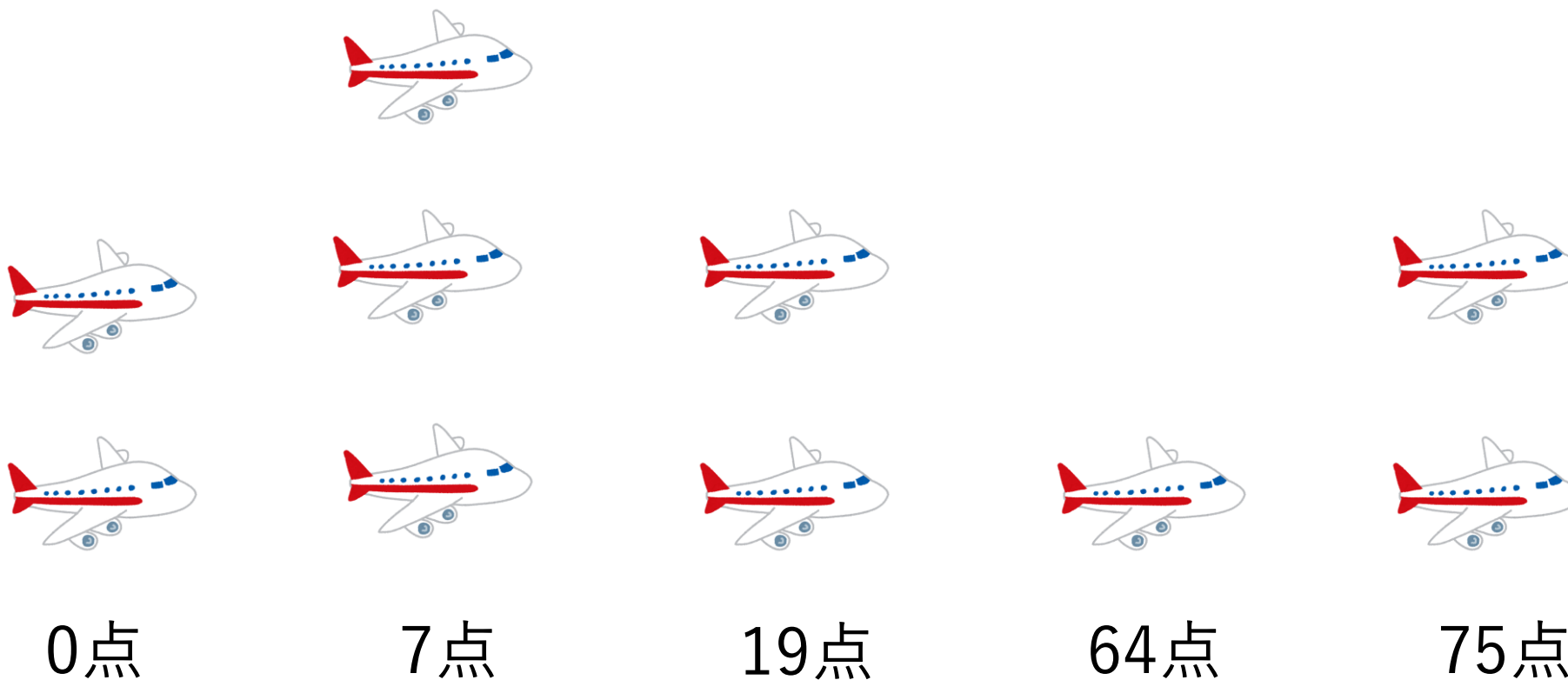
満点解法

- multiset という、集合を管理するデータ構造がある
 - set の重複を許したバージョン
 - 挿入、削除、検索、二分探索などが $O(\log N)$ でできる
 - C++ の標準ライブラリにある
- multiset で終了時刻 $\% K$ を管理する
- 各 A_i について、終了時刻 $\% K$ が $A_i \% K$ 以下のうち最大の滑走路を二分探索で求めれば、使う滑走路が分かる
- $O(M \log N)$ で解けた
- 100 点

満点解法

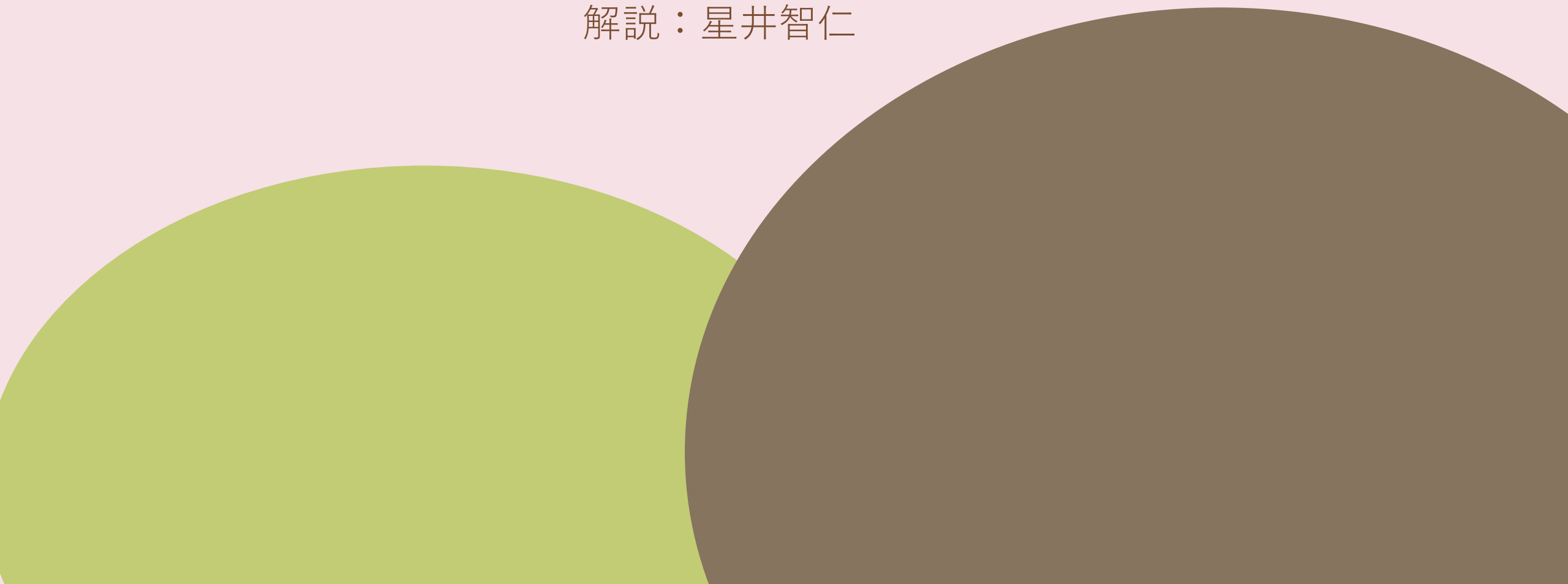
- 着陸時間が被るものを multiset に入れないように注意
- 被らなくなるまで queue に入れておいたりして取り除けば OK

得点分布



一流の団子職人

解説：星井智仁



問題概要

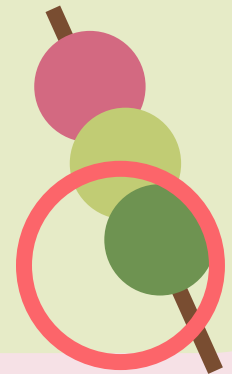
1

N色の団子がそれぞれM個ずつ 合計NM個ある

NM個ある団子を色が異なるようにN個ずつM組に分けたい

使う団子を団子チェッカーにいくつか指定すると作れる串の数を教えてくれる

ただし団子チェッカーを使える回数は50000回以下である



小課題

2

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題

3

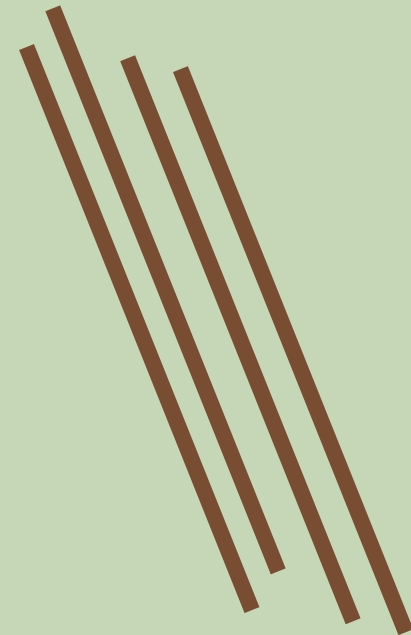
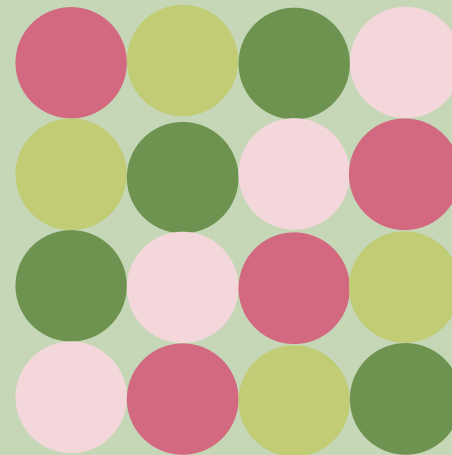
1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題 1 $N=4$ $M=4$

4

$N=4$, $M=4$

団子は $NM=16$ 個ある
全通り試せるかな



小課題 1 $N=4$ $M=4$

5

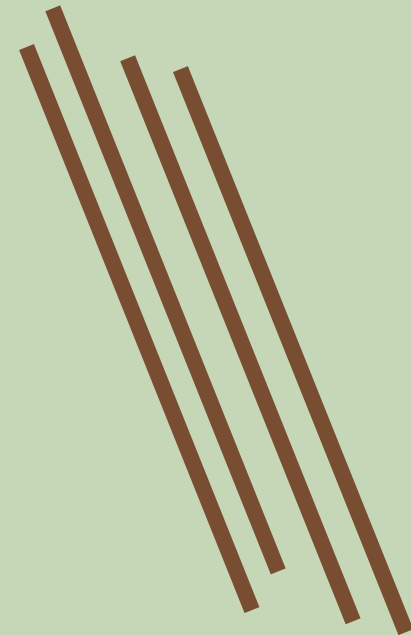
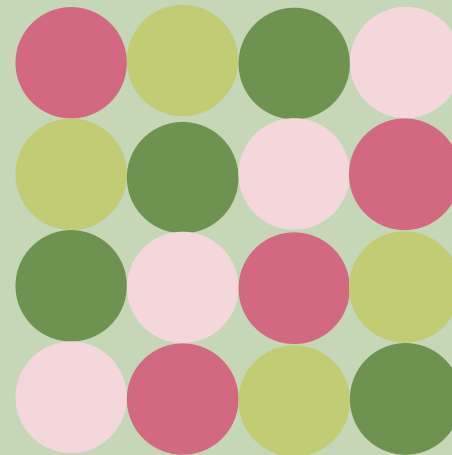
$N=4$, $M=4$

団子は $NM=16$ 個ある

全通り試せるかな

$$2^{16} = 65536 > 50000$$

試せない



小課題 1 $N=4$ $M=4$

6

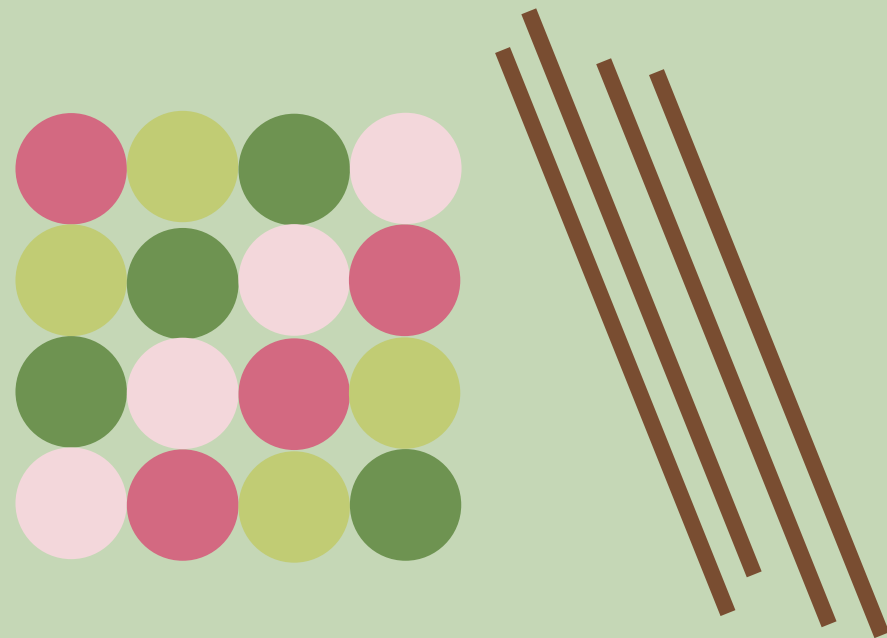
$N=4$, $M=4$

団子は $NM=16$ 個ある

全通り試せるかな

$${}_{16}C_4 = 1820 \leq 50000$$

試せる



小課題

7

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

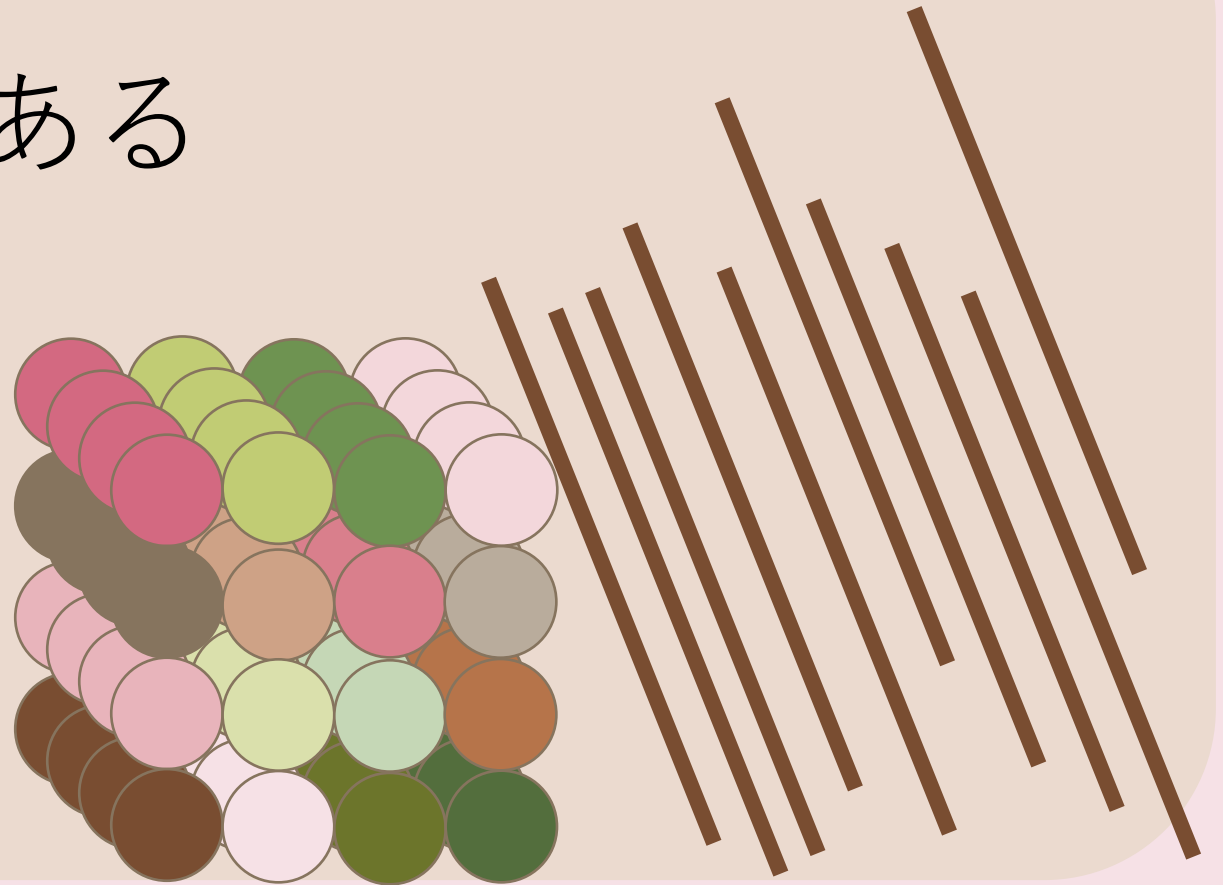
小課題 2 $N=100$ $M=10$

8

$N=100$, $M=10$

団子は $NM=1000$ 個ある

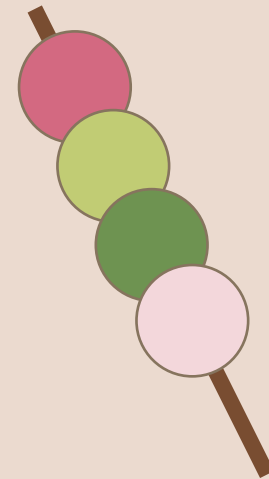
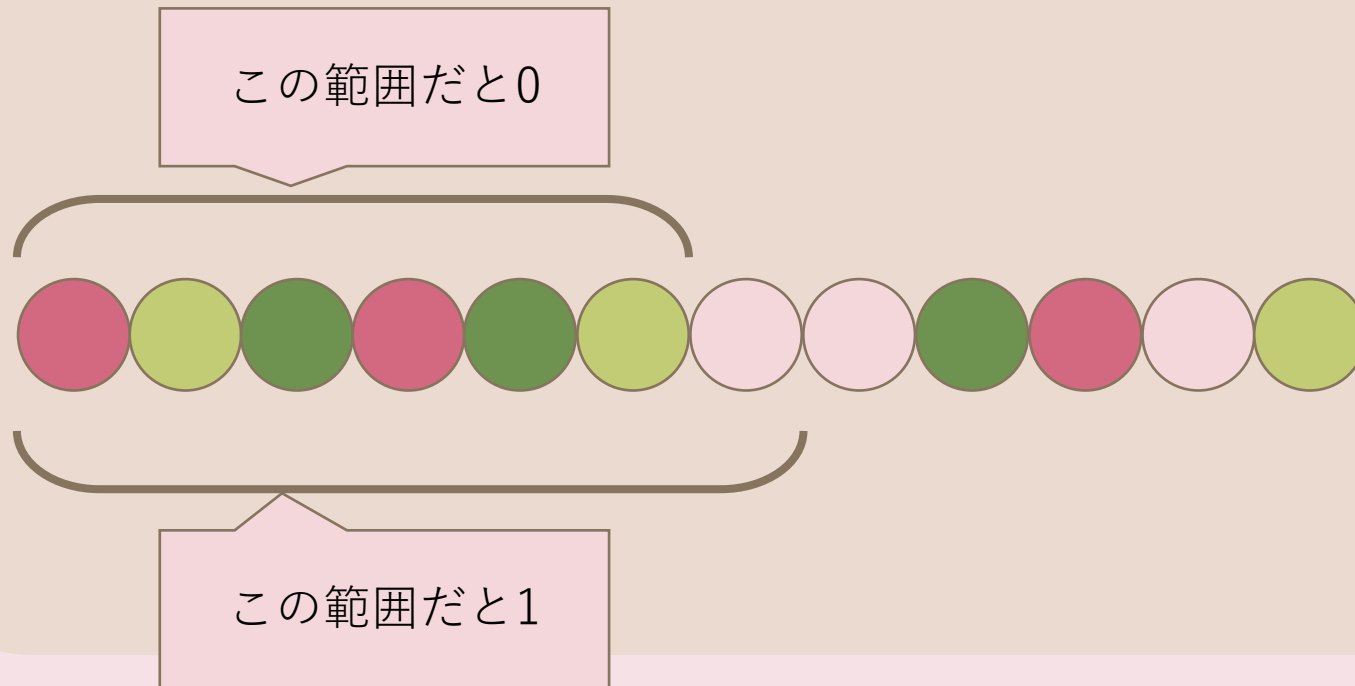
$${}_{1000}C_{10} \approx 2 \times 10^{23}$$



小課題 2 $N=100$ $M=10$

9

団子チェッカーで「団子 1 串作れる」と判定される条件
→すべての色が揃う必要あり=1つでも欠けていると判定されない

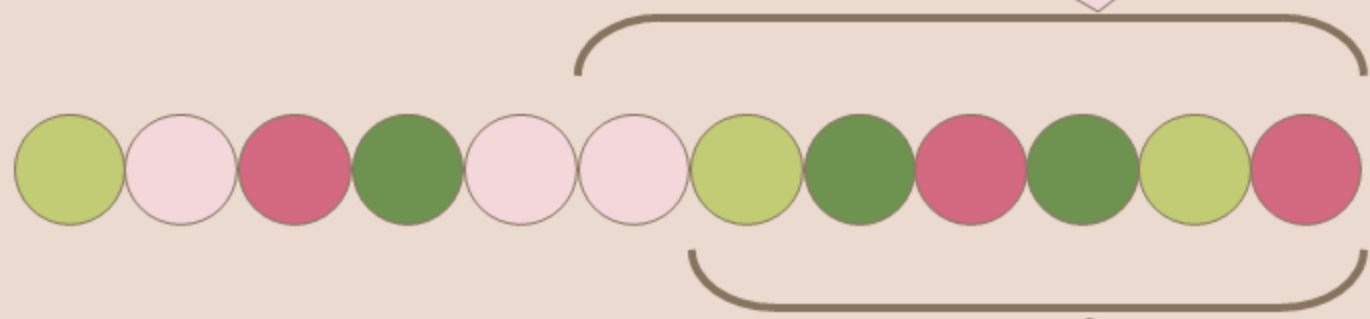


小課題2 N=100 M=10

発想を逆転させよう

因子マシーンで「因子1串作れる」と判定される条件

→オバマの角が描るが西ちいー1つズレたはマインズと判定されたい



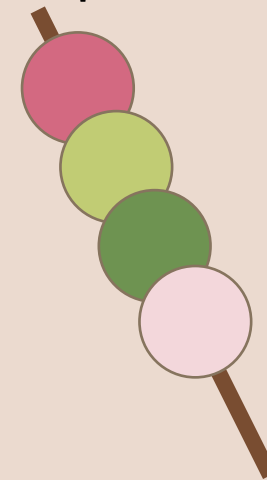
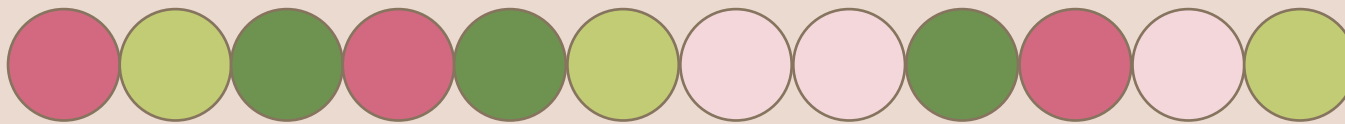
小課題 2 $N=100$ $M=10$

11

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり=1つでも欠けていると判定されない

→全選択し、いくつか取り除いて判定



小課題 2 $N=100$ $M=10$

12

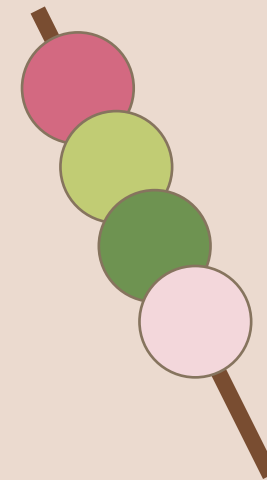
団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり=1つでも欠けていると判定されない

→全選択し、いくつか取り除いて判定

最悪はざっくり $N \times M^2/2 = 50000$ 回

これはざっくり見積もりすぎで、もうちょっと回数は少ない



小課題

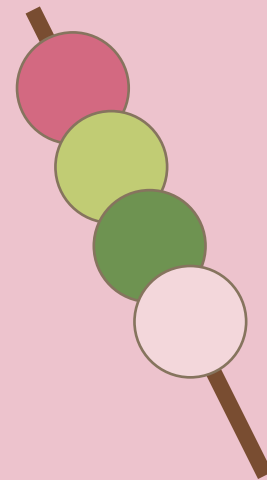
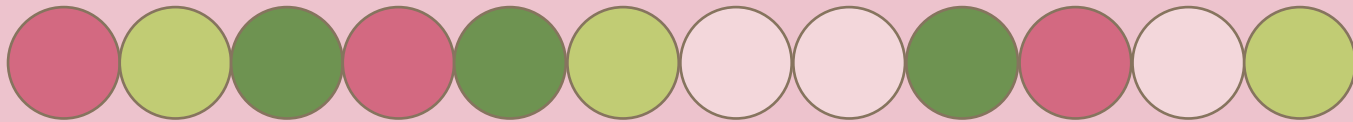
13

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題 3 $N=200$ $M=25$

14

小課題 2: 全選択し、いくつか取り除いて判定
最悪はざっくり $N \times M^2/2$ 回

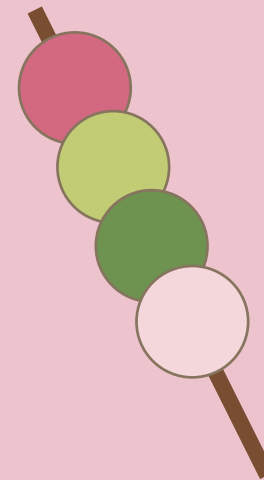
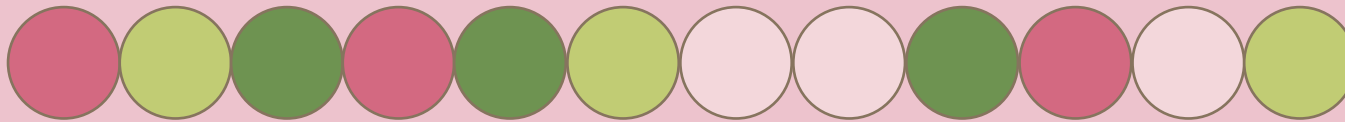


小課題 3 $N=200$ $M=25$

15

小課題 2: 全選択し、いくつか取り除いて判定

最悪はざっくり $N \times M^2/2$

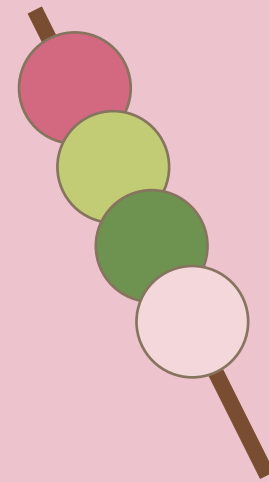
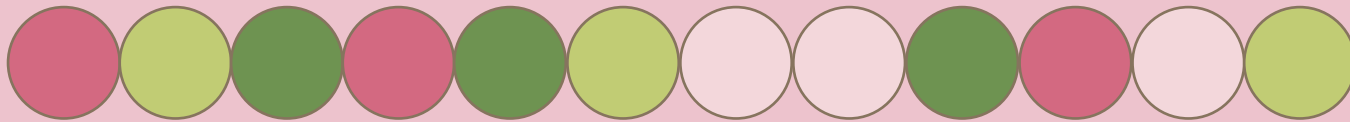


小課題 3 $N=200$ $M=25$

16

小課題 2: 全選択し、いくつか取り除いて判定

最悪 はざっくり $N \times M^2/2$



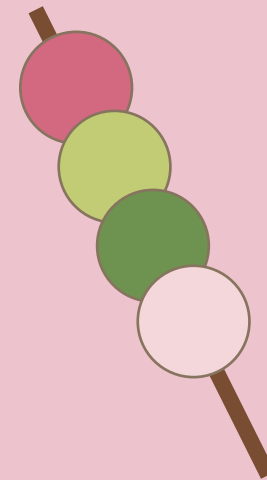
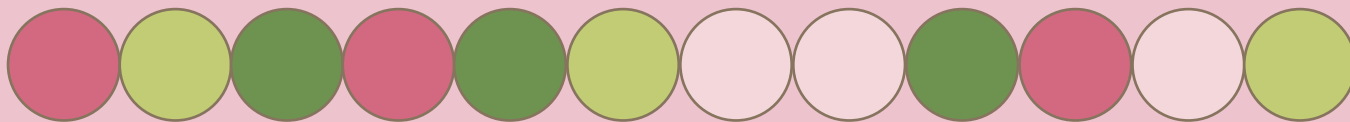
小課題 3 $N=200$ $M=25$

17

小課題 2: 全選択し、いくつか取り除いて判定

平均 はざっくり $N M \log_e N_{\text{回}}$

ランダムに選ぶと調和級数に落ちる 27000回くらい



これを撃墜するジャッジがこの世に存在する気がないので、書いてみると通ります

小課題 3 $N=200$ $M=25$

18

あまりに酷い解法を紹介してしまったので
マシな解法も紹介します

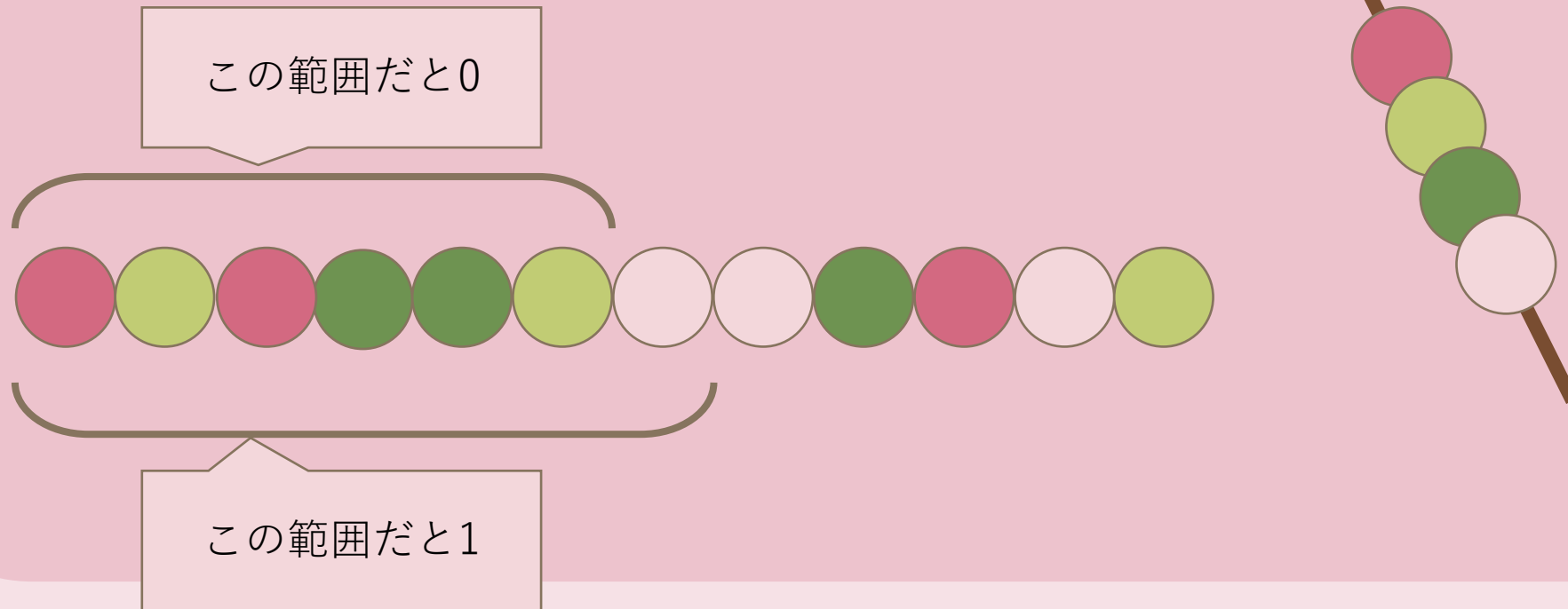
小課題 3 $N=200$ $M=25$

19

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり



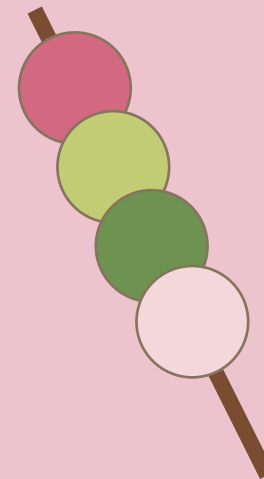
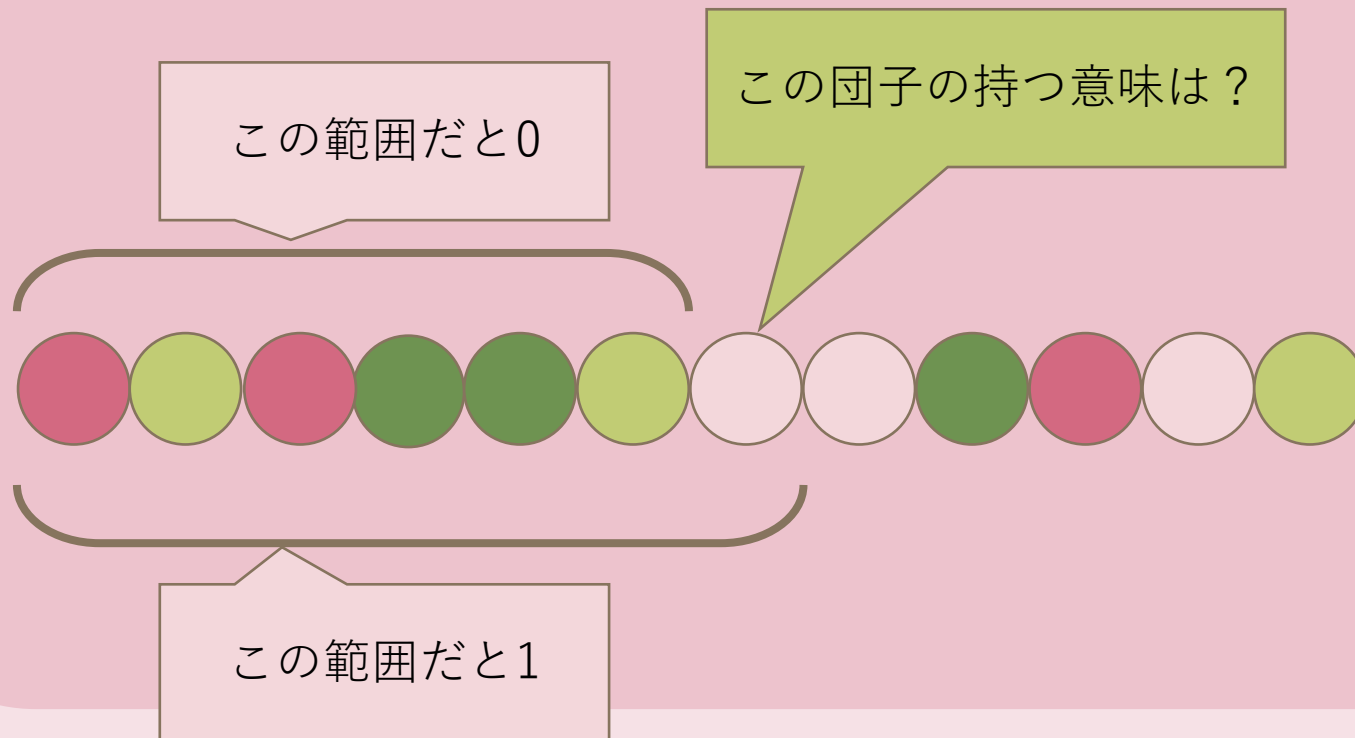
小課題 3 $N=200$ $M=25$

20

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり



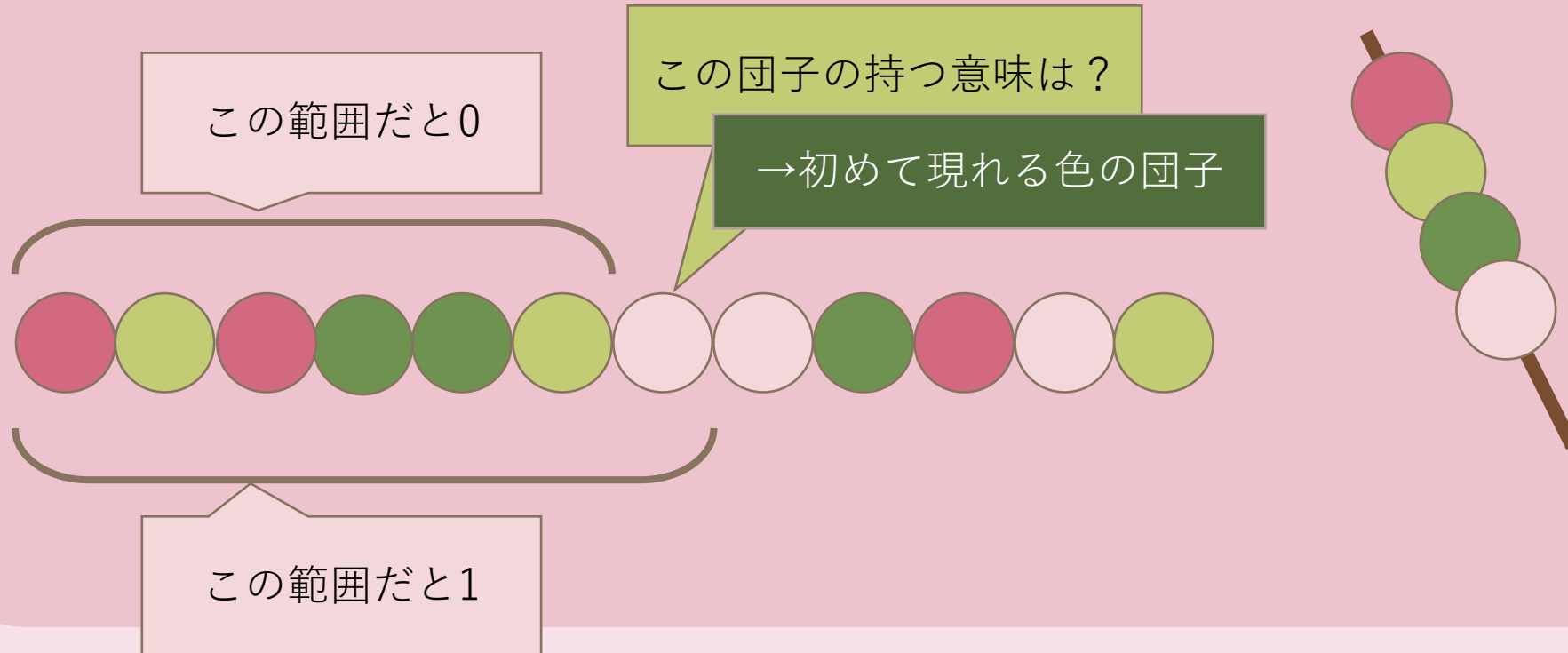
小課題 3 $N=200$ $M=25$

21

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

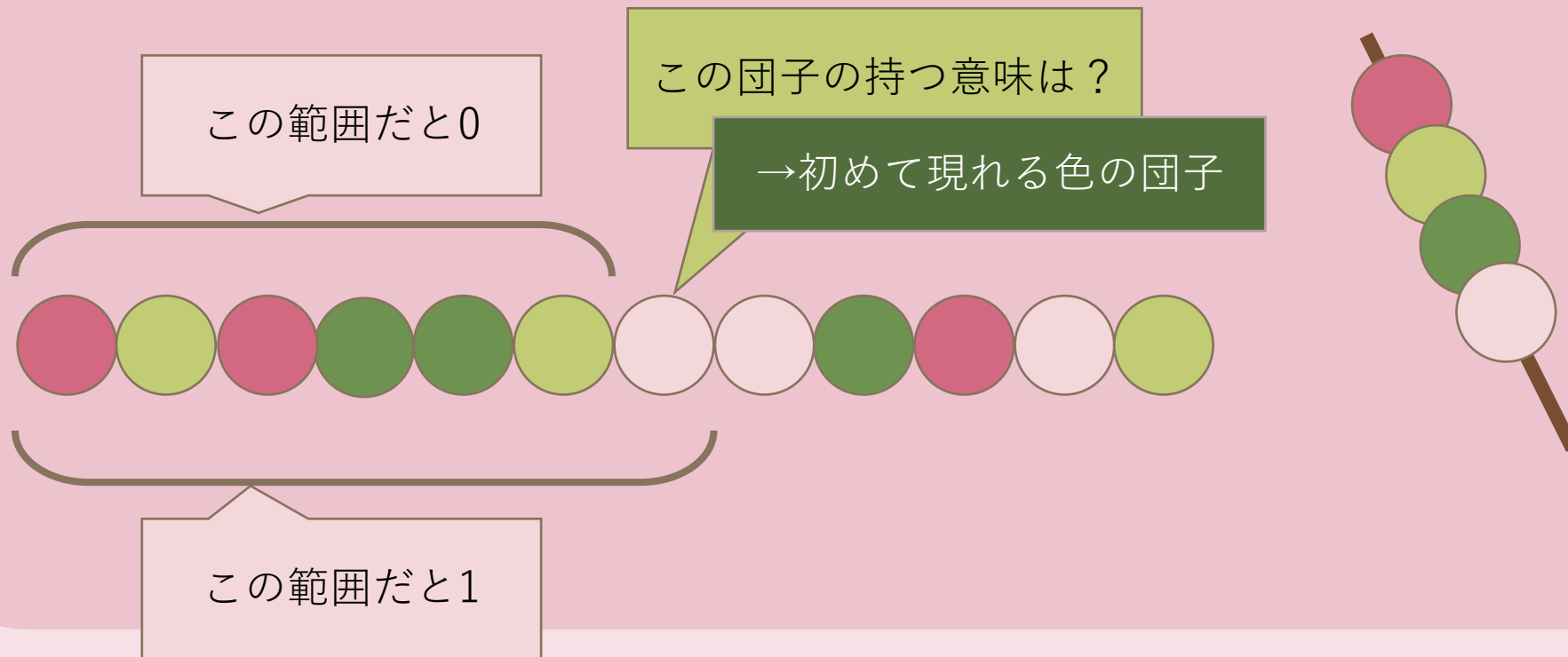
→すべての色が揃う必要あり



小課題 3 $N=200$ $M=25$

22

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる

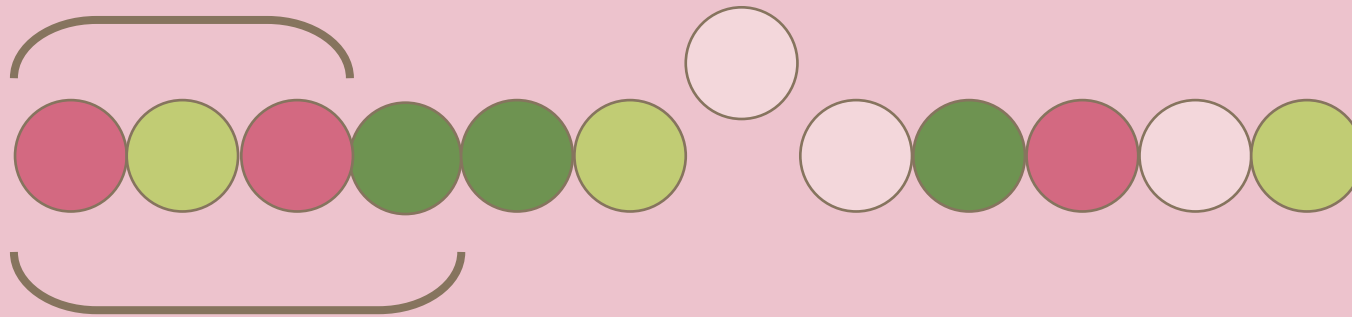


小課題 3 $N=200$ $M=25$

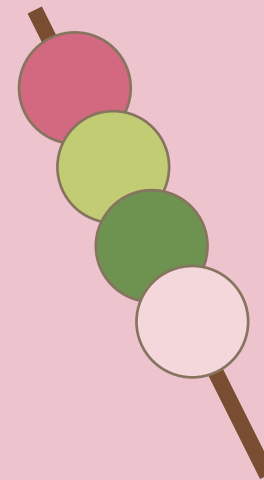
23

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる

この範囲+  だと0



この範囲+  だと1

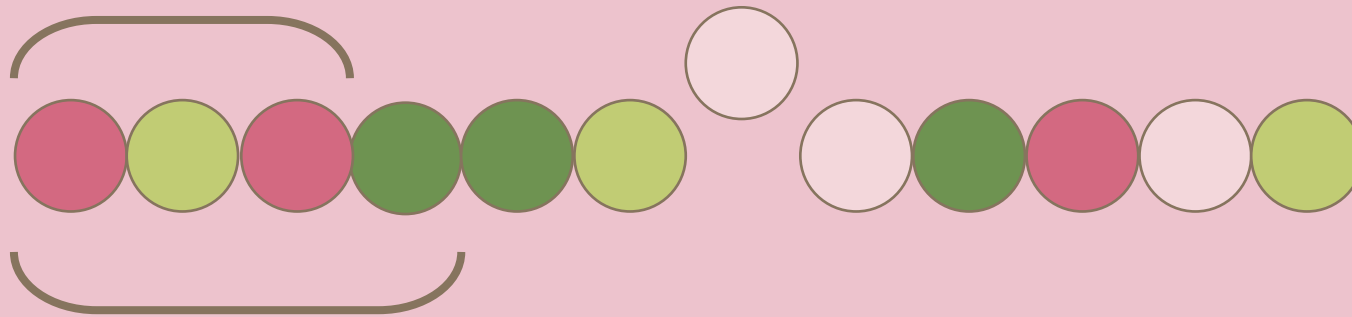


小課題 3 $N=200$ $M=25$

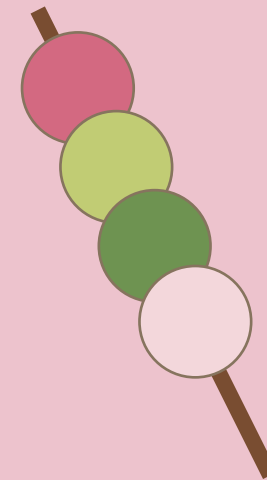
24

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる
繰り返すと $N\log(NM)$ 回で1本の串団子が作れる

この範囲+  だと0



この範囲+  だと1



小課題 3 $N=200$ $M=25$

25

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)

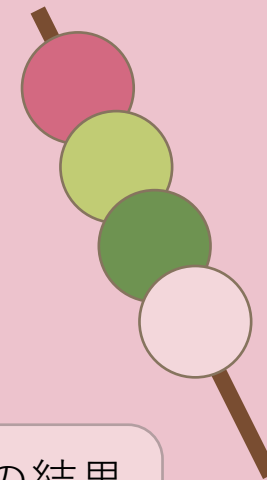


使う

余り



クエリの結果
1

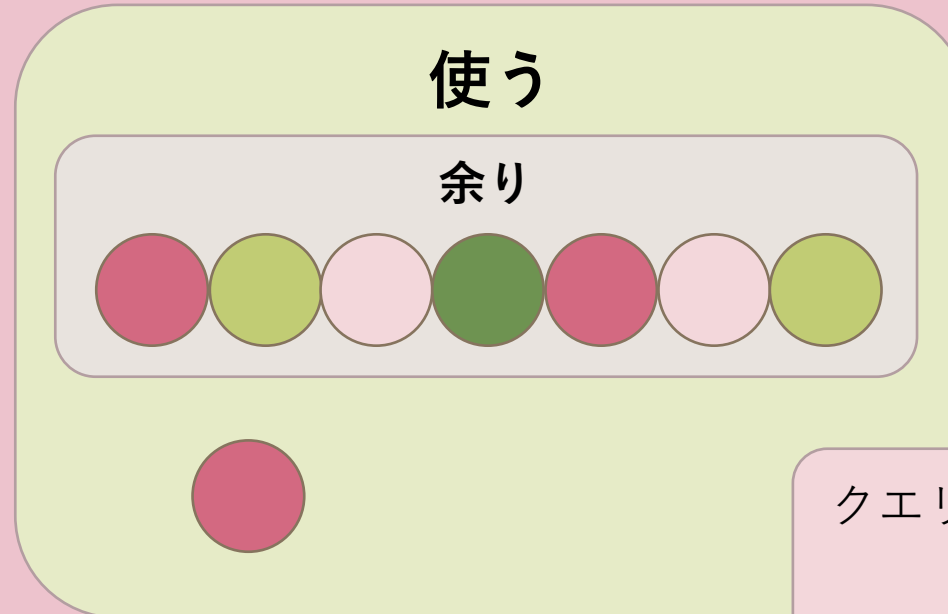


小課題 3 $N=200$ $M=25$

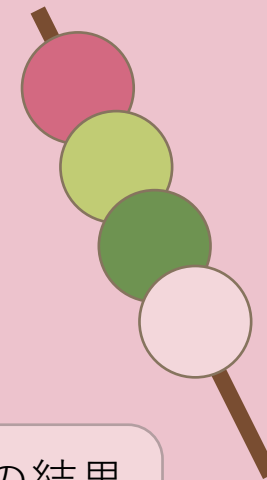
26

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



クエリの結果
1

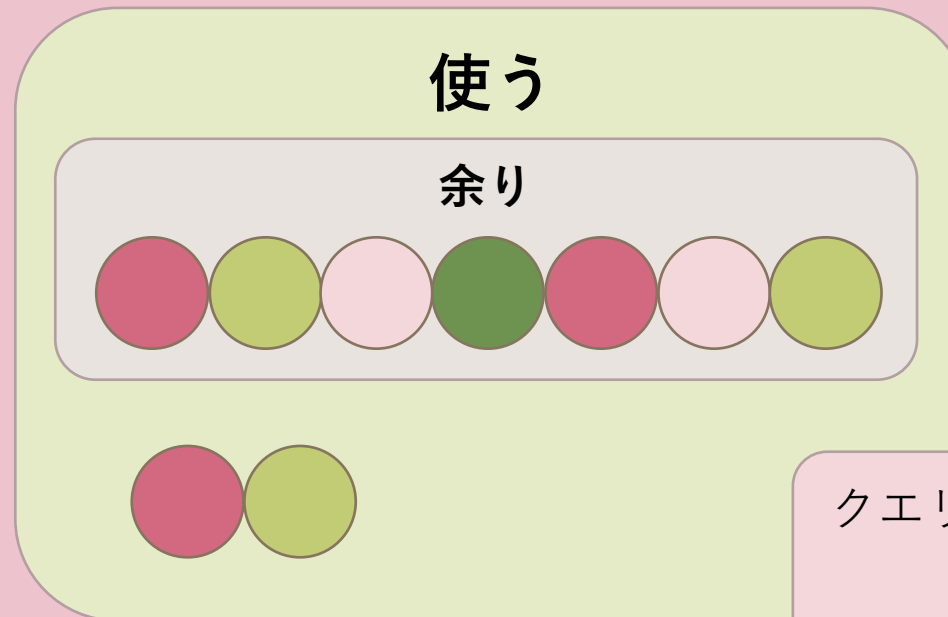
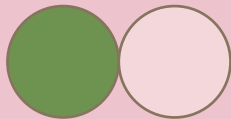


小課題 3 $N=200$ $M=25$

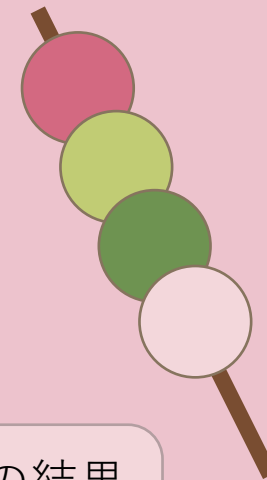
27

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



クエリの結果
1

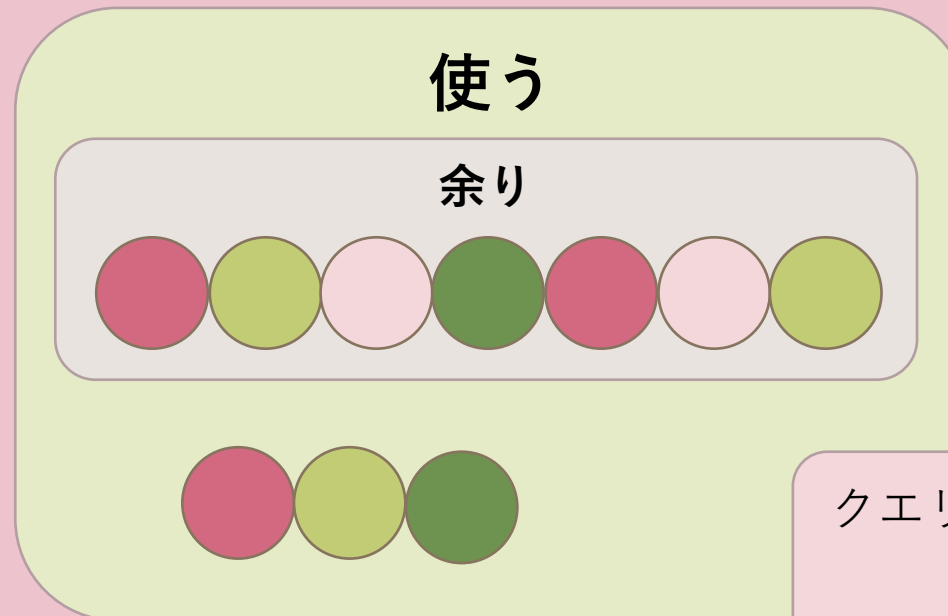


小課題 3 $N=200$ $M=25$

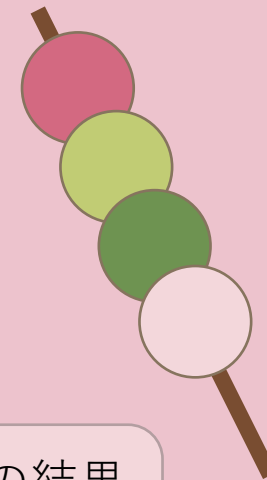
28

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



クエリの結果
2

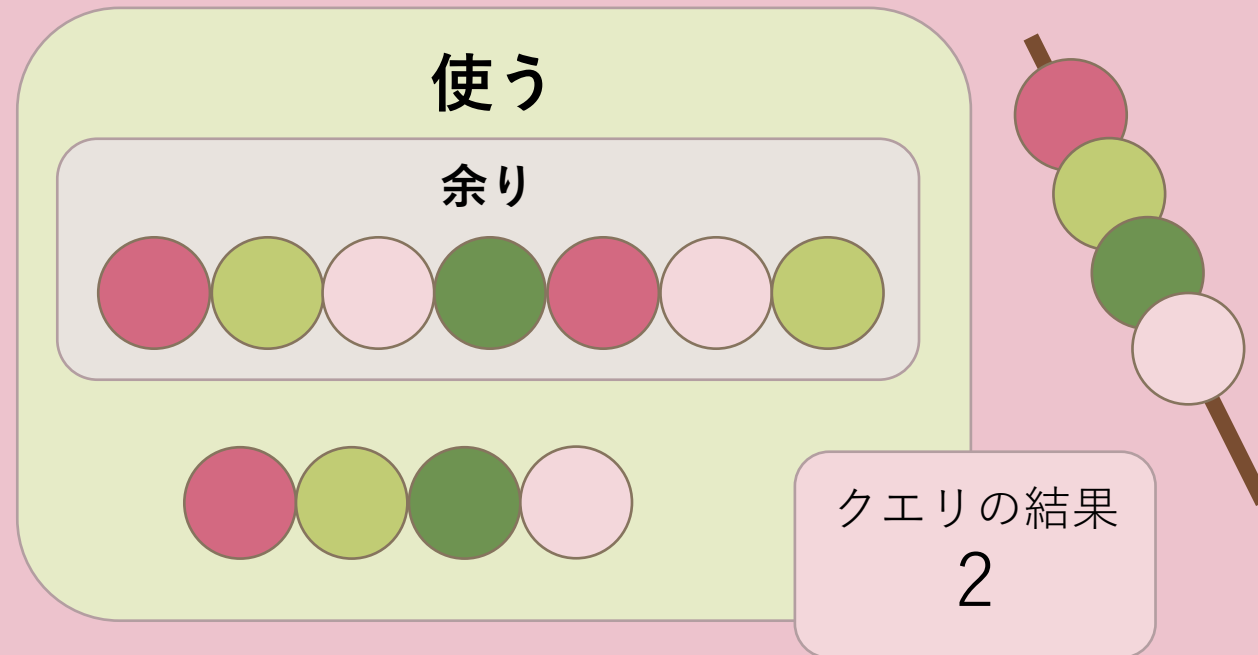


小課題 3 $N=200$ $M=25$

29

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



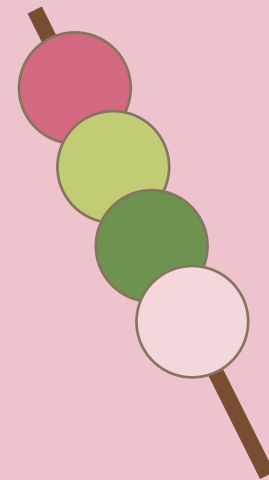
小課題 3 $N=200$ $M=25$

30

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる

$$N \log(NM) + NM \log N = 41000 \text{ 回くらい}$$



小課題

31

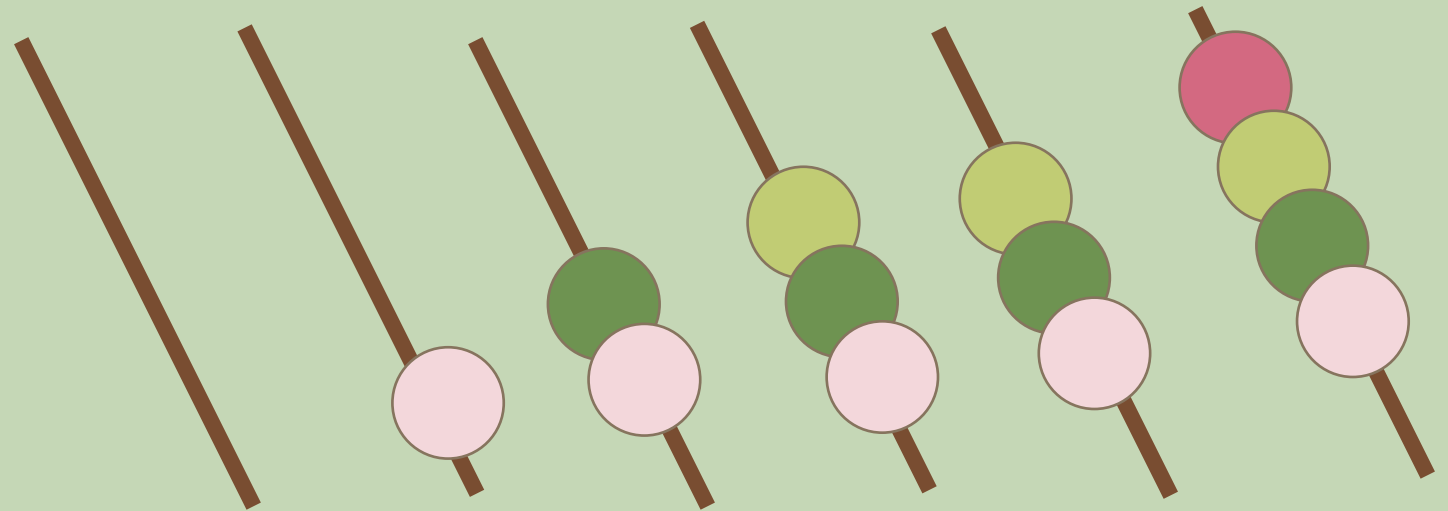
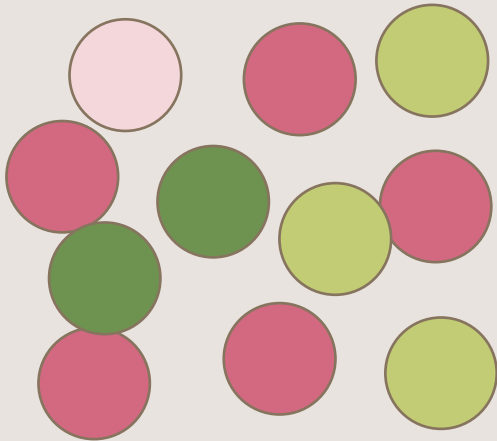
1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題 4 $N=400$ $M=25$

32

串をM本用意して 串に団子を刺していくことを考える

未確定



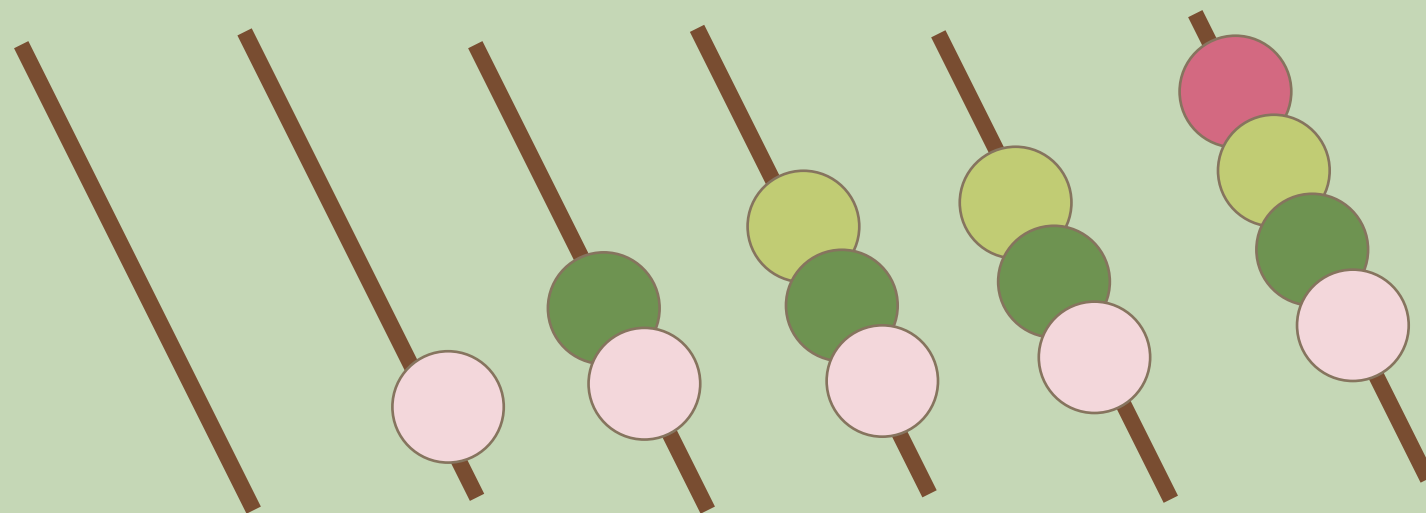
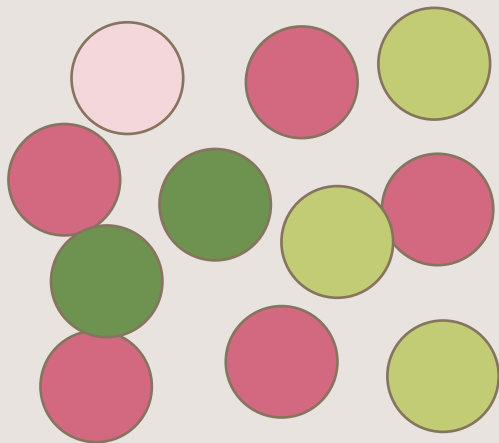
この後に説明する操作をすると 上図のように単調性をもつように串に団子を刺せる

小課題 4 $N=400$ $M=25$

33

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する

未確定



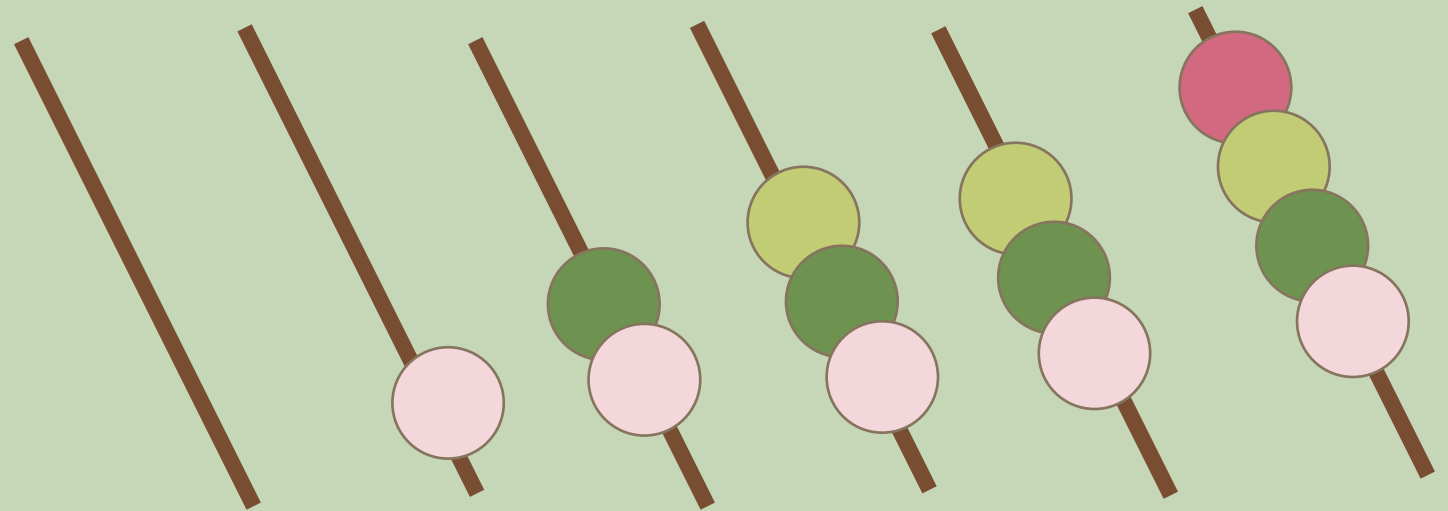
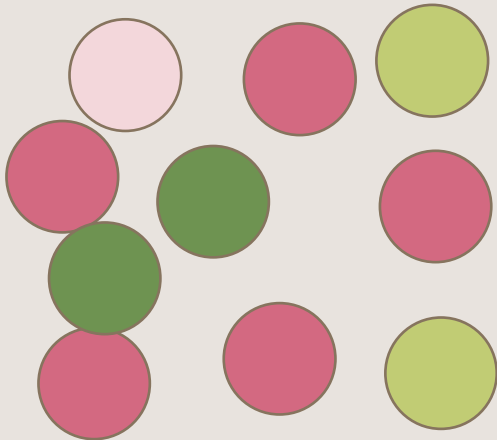
この後に説明する操作をすると 上図のように単調性をもつように串に団子を刺せる

小課題 4 $N=400$ $M=25$

34

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

未確定



←どこに刺せるか知りたい

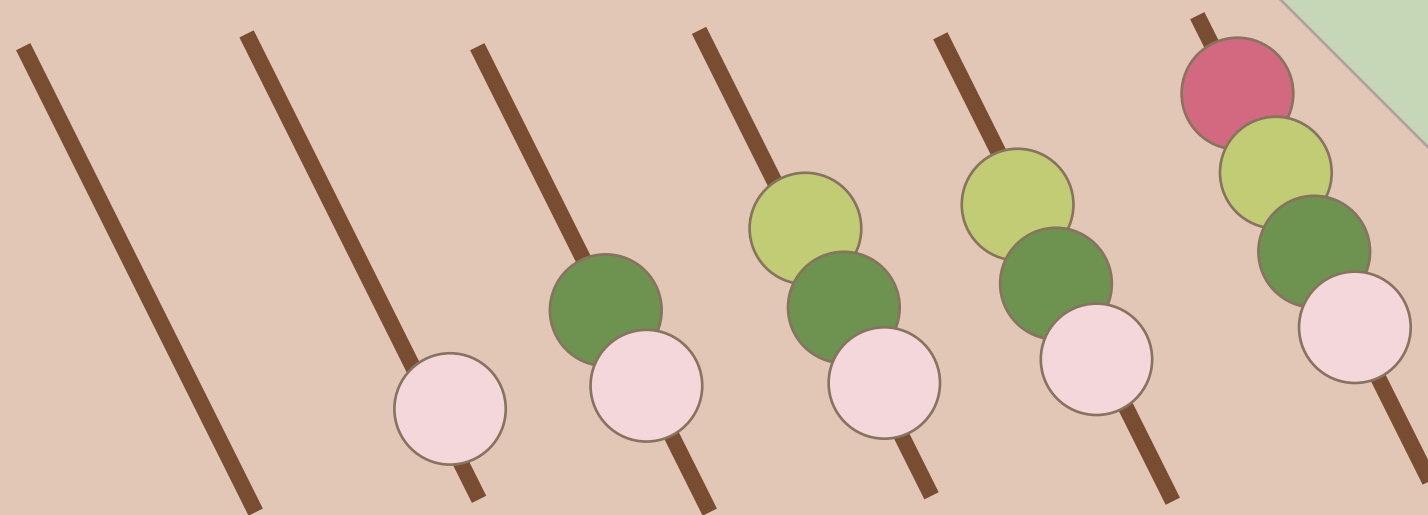
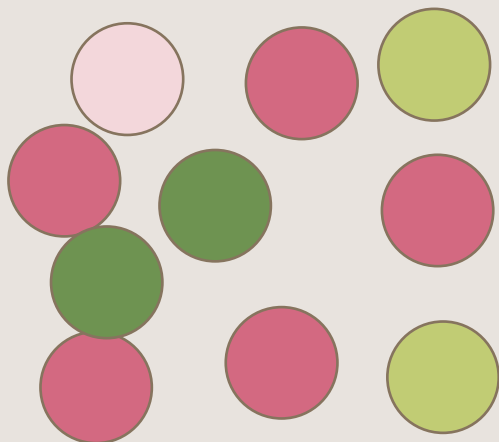
小課題 4 $N=400$ $M=25$

35

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

使う

未確定



クエリの結果
5

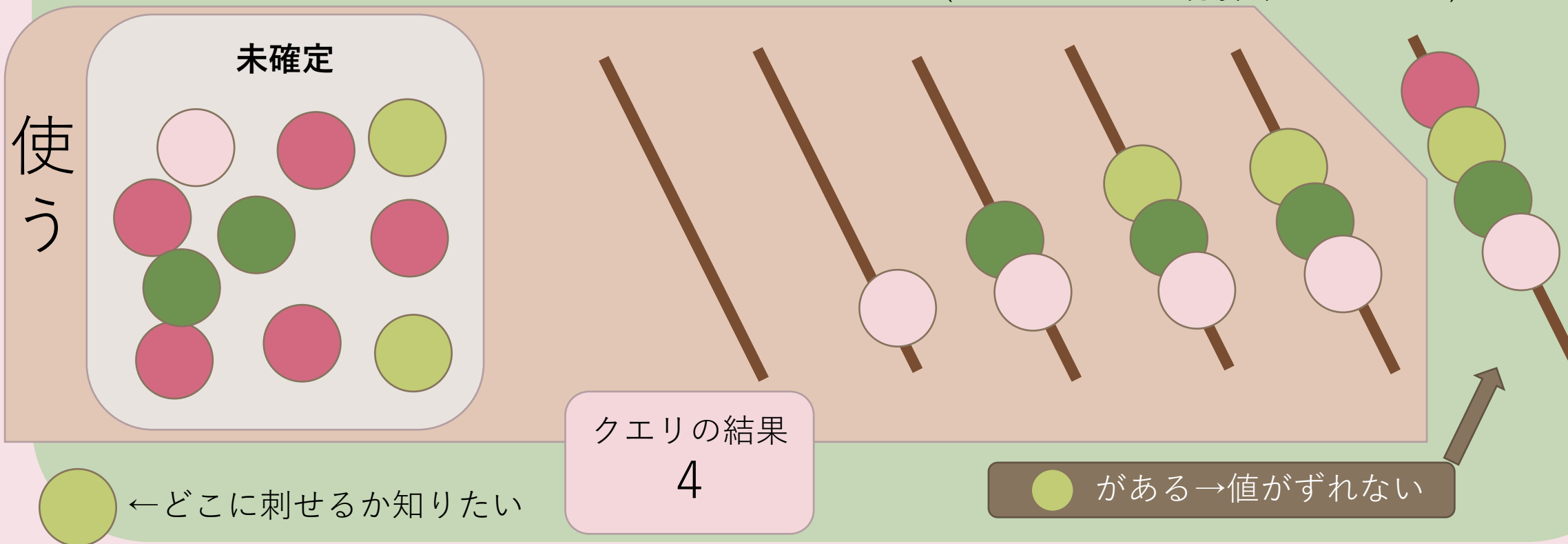
←どこに刺せるか知りたい

● がある → 値がずれない

小課題 4 $N=400$ $M=25$

36

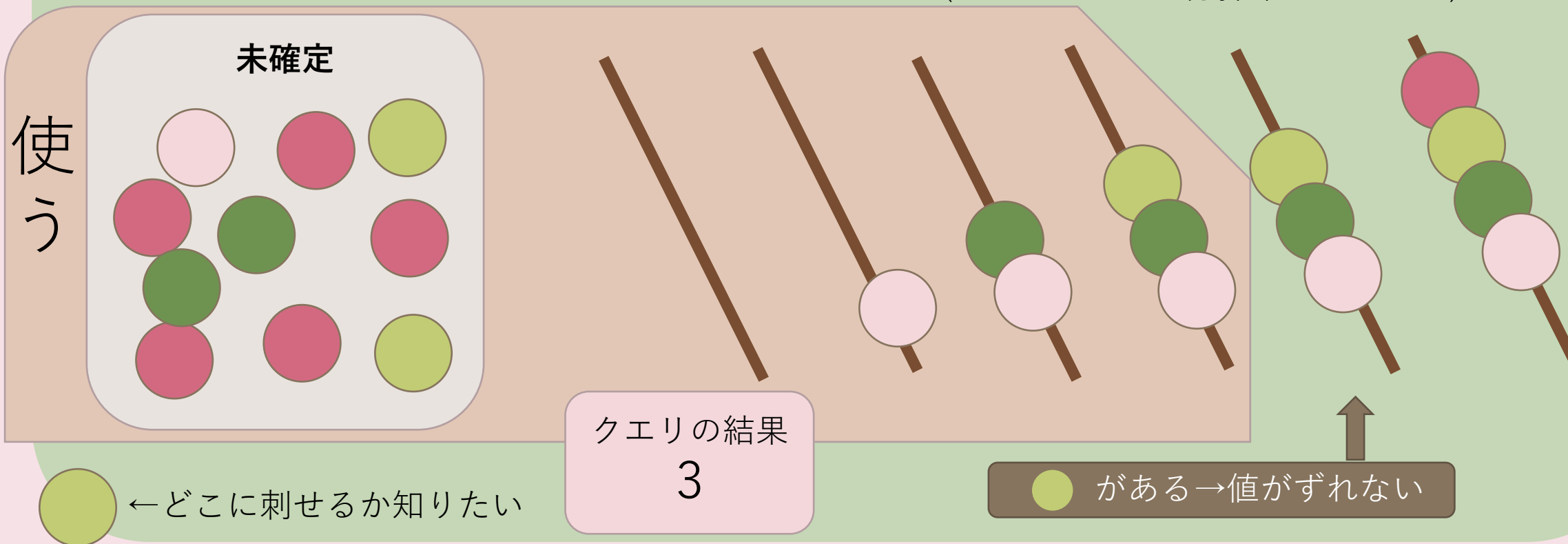
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

37

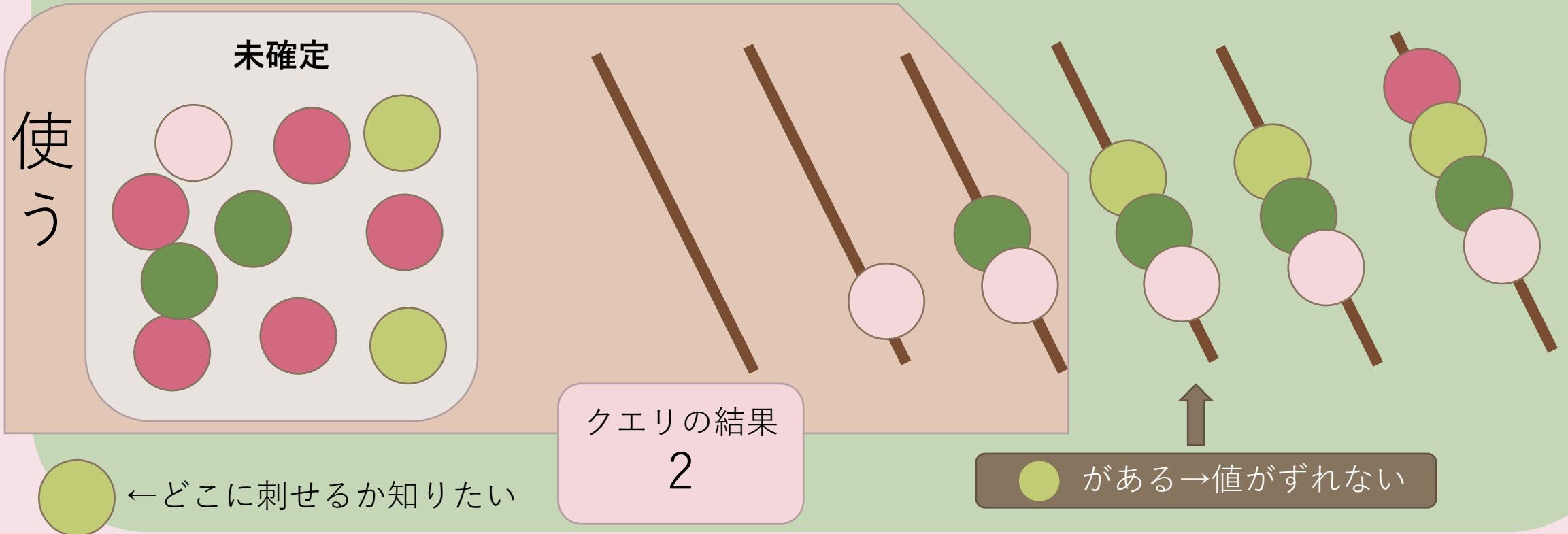
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

38

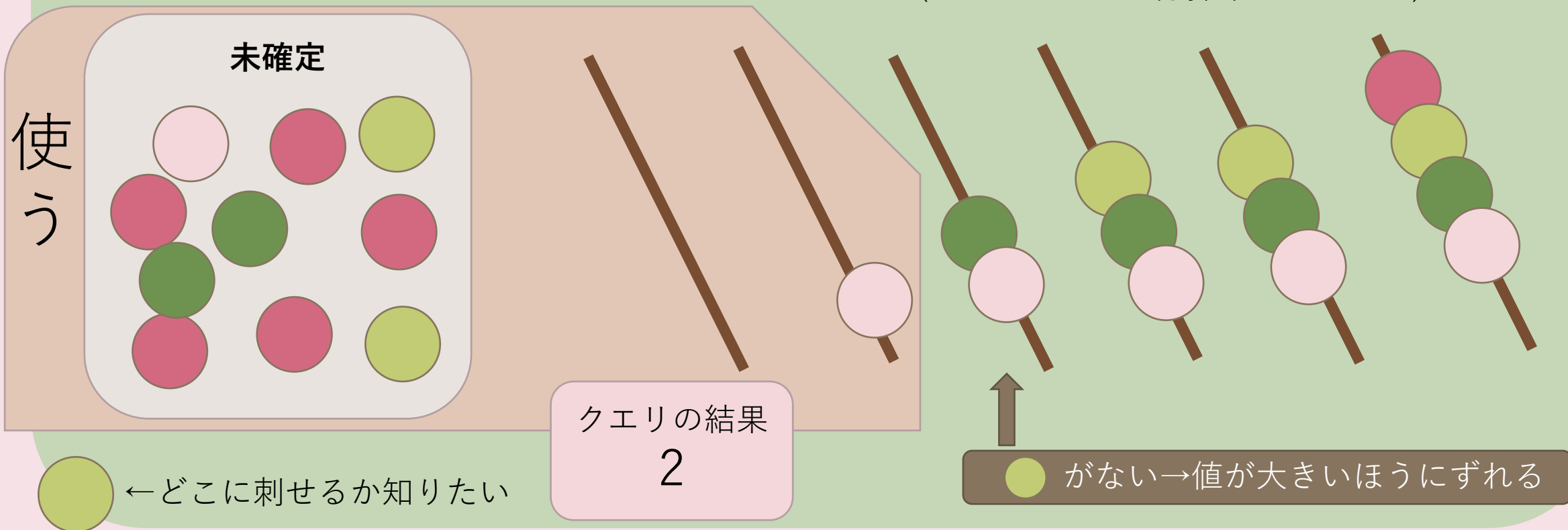
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

39

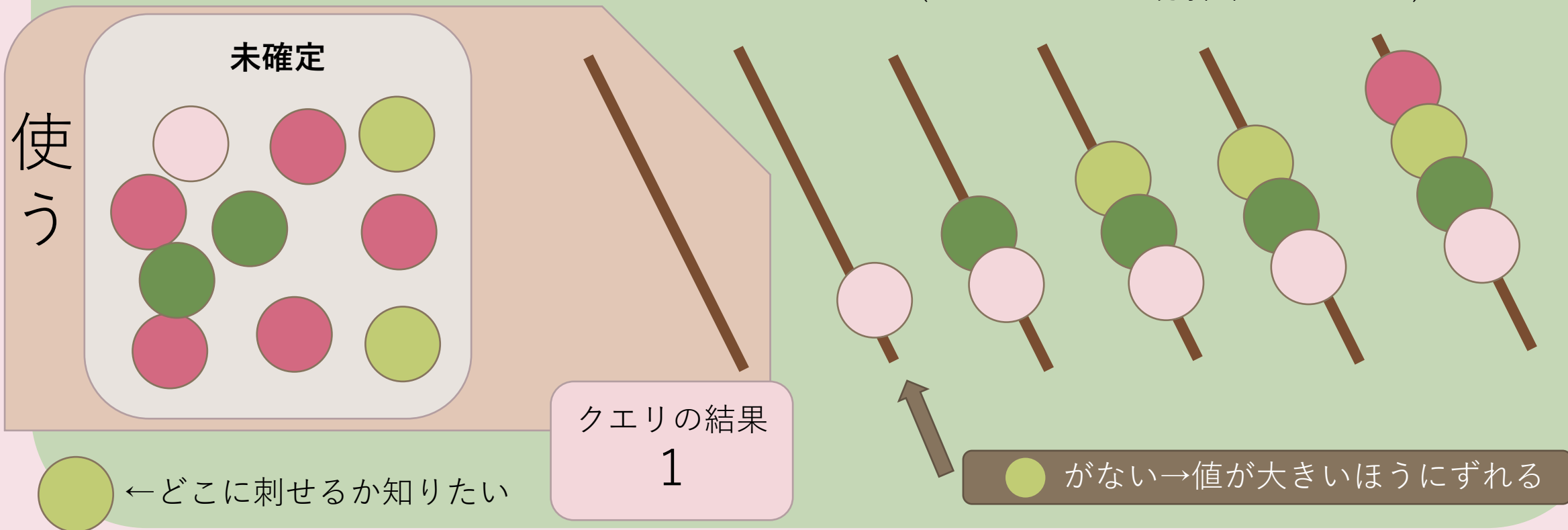
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

40

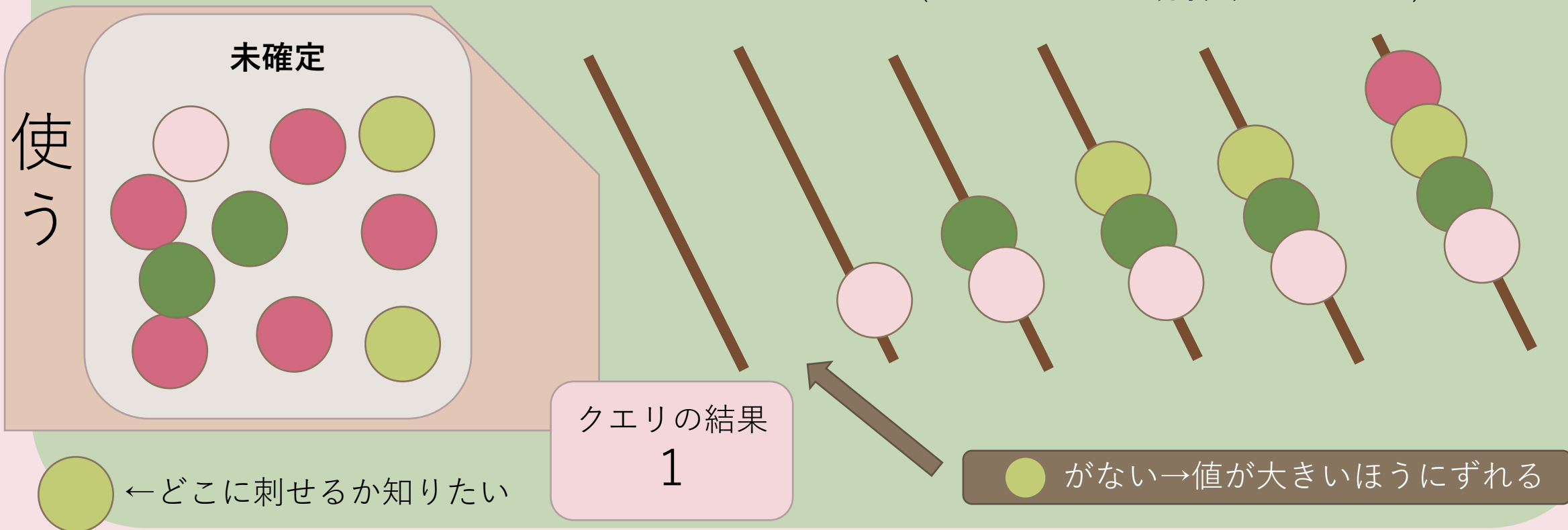
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

41

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

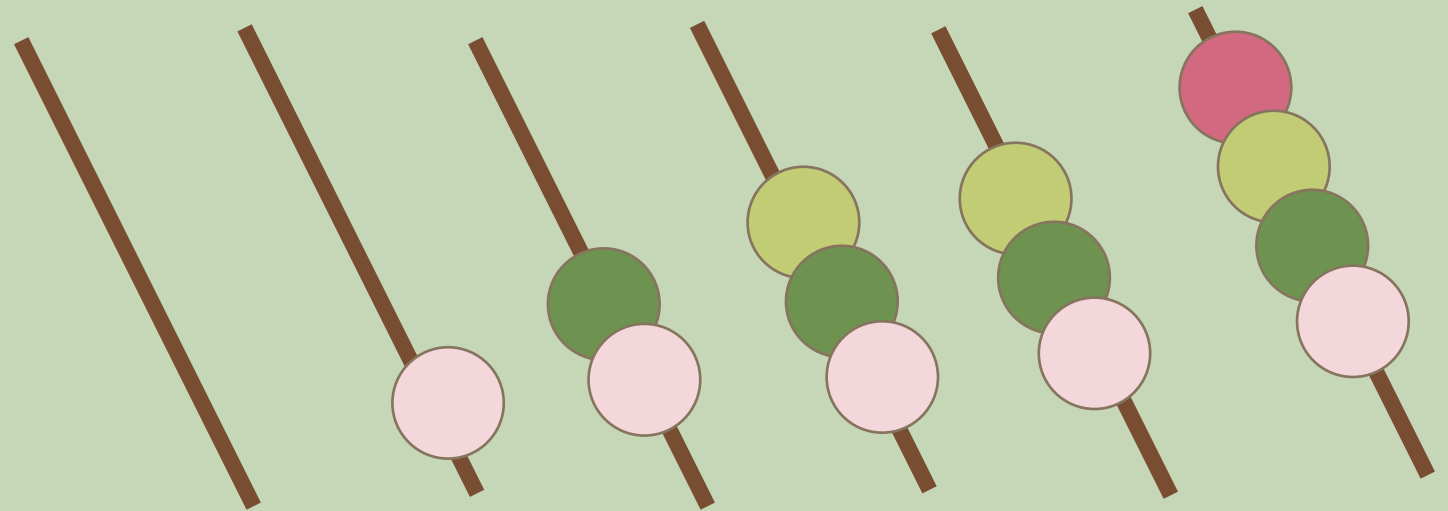
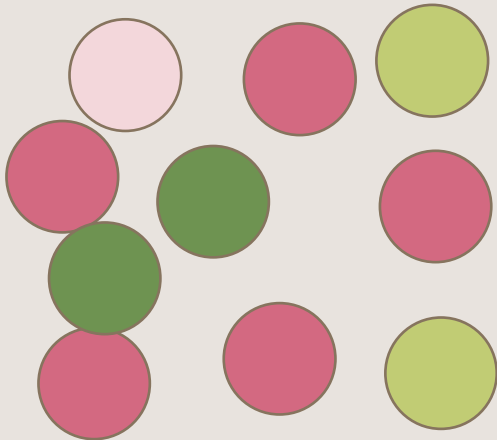


小課題 4 $N=400$ $M=25$

42

$NM \log M = 45000$ くらい

未確定



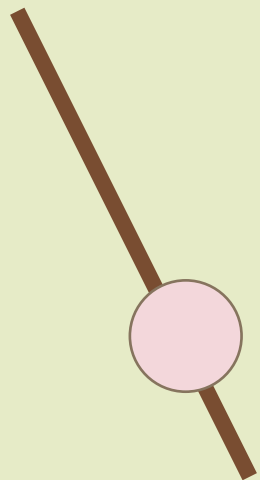
←どこに刺せるか知りたい

得点分布 JOI

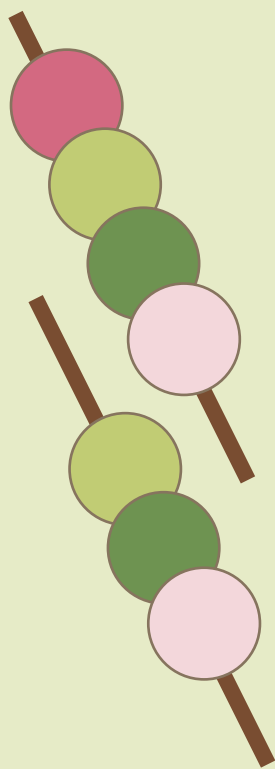
43



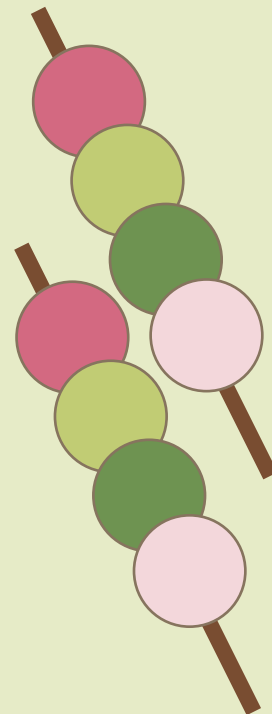
0



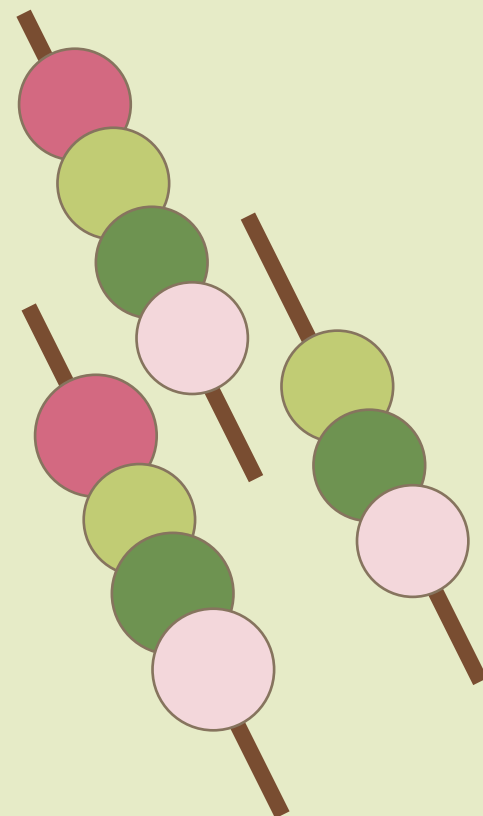
2



7



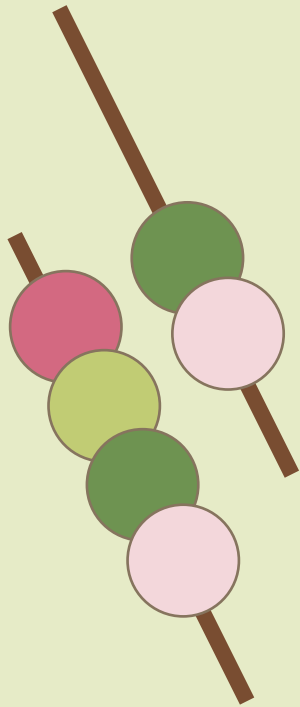
22



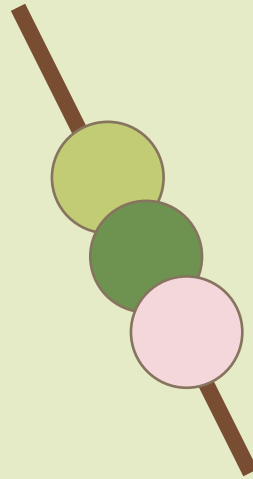
100

得点分布 JOIG

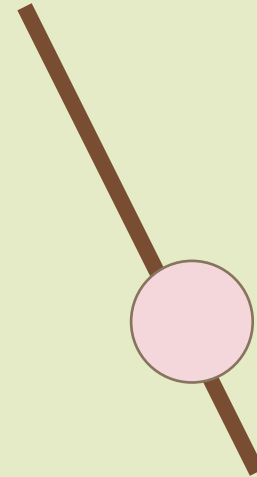
44



0



5



100

復興事業 解説

松尾 凜太郎 / QCFium

問題概要

辺に整数が書かれている無向グラフが与えられる。
以下の形のクエリを Q 個処理せよ：

整数 x が与えられる。コスト 1 で辺に書かれている整数を 1 増減させることで、 x が書かれている辺のみで全頂点間を行き来できるようにするときの最小コストを求める

問題概要

辺に整数が書かれている無向グラフが与えられる。
以下の形のクエリを Q 個処理せよ：

整数 x が与えられる。整数 c が書かれている辺のコストを $|c - x|$ としたとき、グラフの最小全域木を求める

$$N \leq 500$$

$$M \leq 100000$$

$$Q \leq 1000000$$

小課題 1

$$M \leq 16, Q \leq 10$$

全域木に使う辺の集合を全探索 (2^M 通り)

→ 本当に連結になっているならば使った辺についてコストを計算する

$$O(Q \times 2^M \times M)$$

クラスカル法

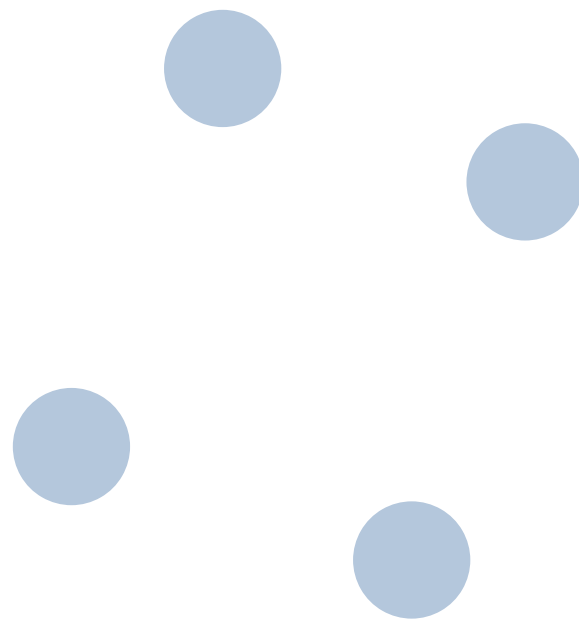
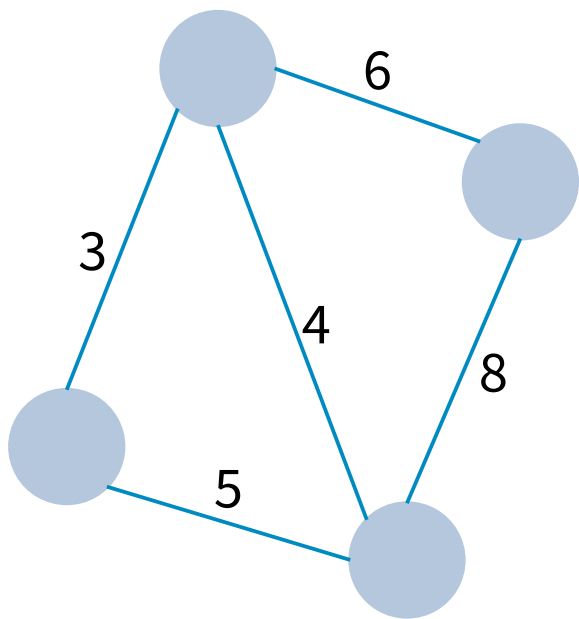
全ての辺を、コストの小さい順に見ていく

まだその辺が繋ぐ2点が連結でないならばその辺を追加する
既に連結ならばスキップする
最終的に追加された辺が最小全域木となる

辺の追加 & 連結判定 → UnionFind

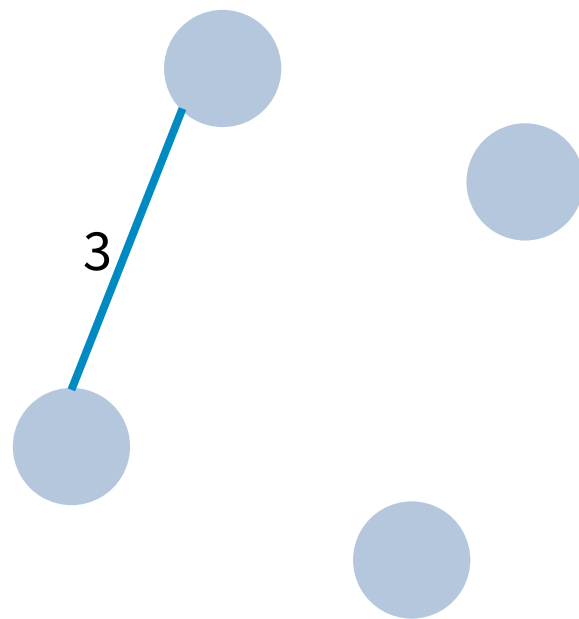
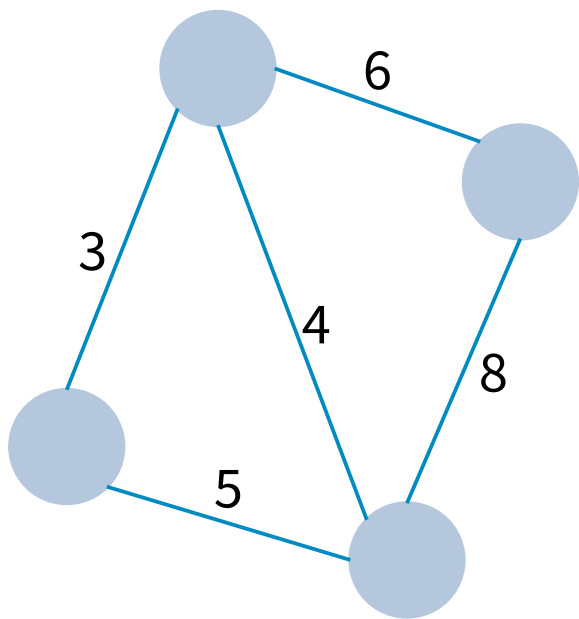
クラスカル法

全ての辺を、コストの小さい順に見ていく



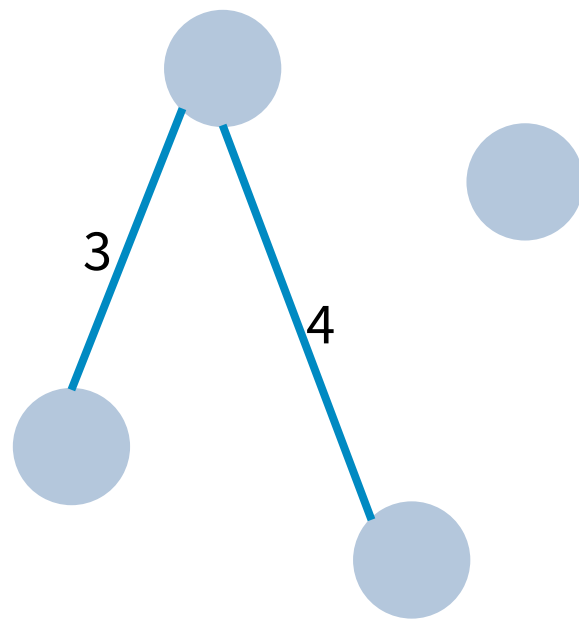
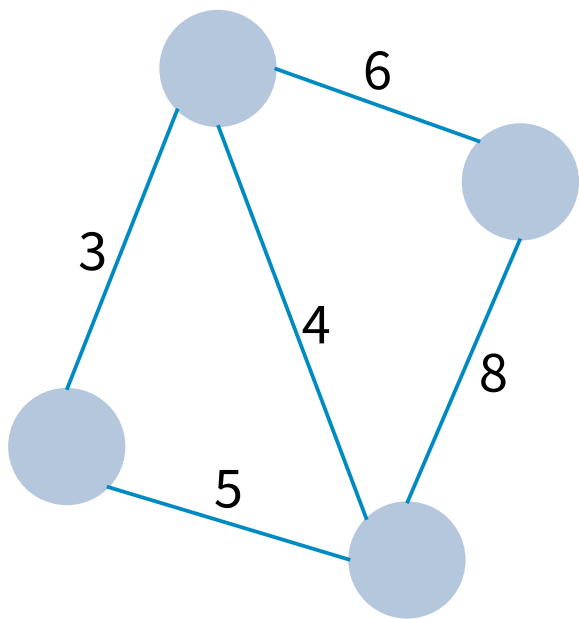
クラスカル法

全ての辺を、コストの小さい順に見ていく



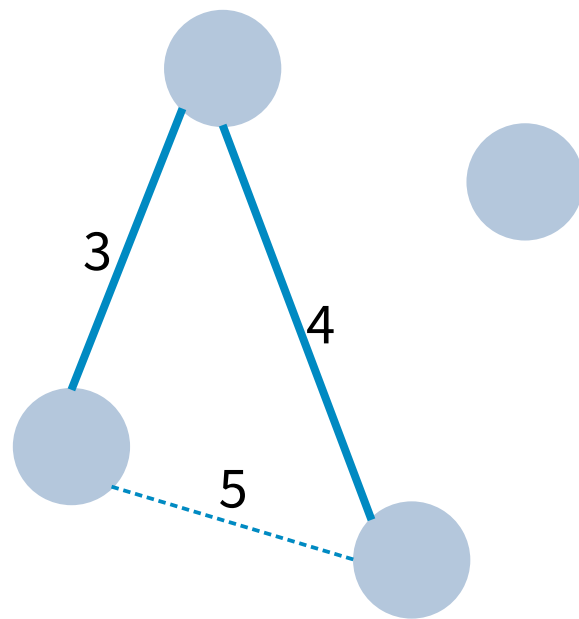
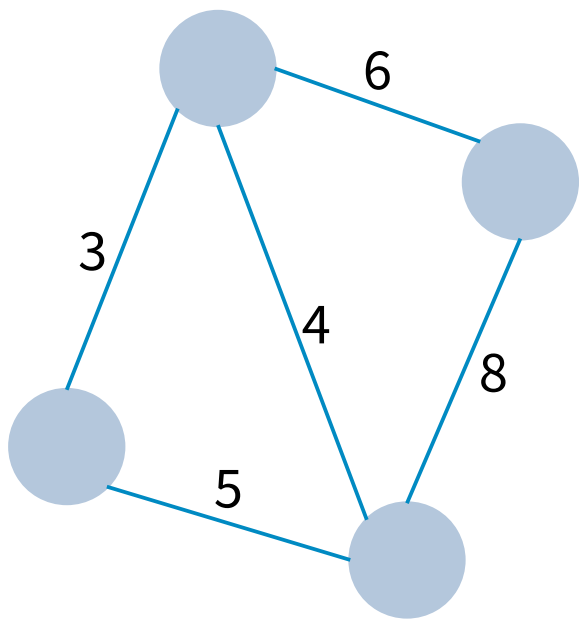
クラスカル法

全ての辺を、コストの小さい順に見ていく



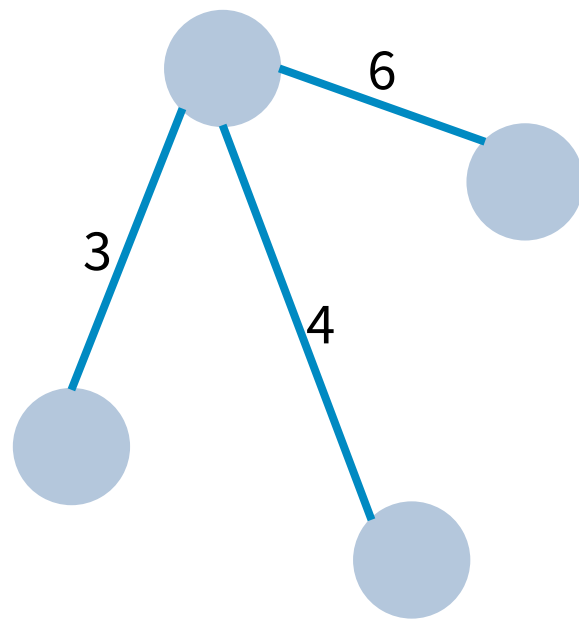
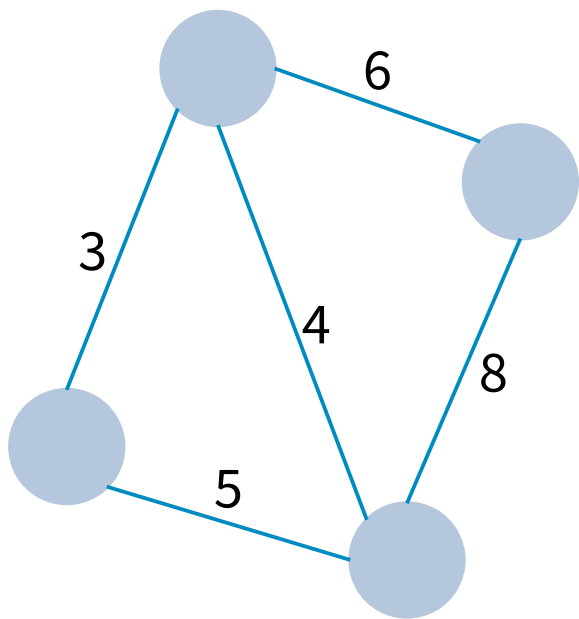
クラスカル法

全ての辺を、コストの小さい順に見ていく



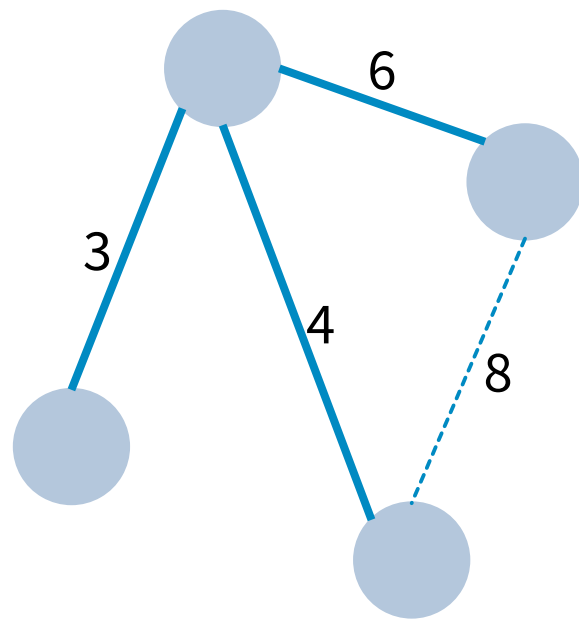
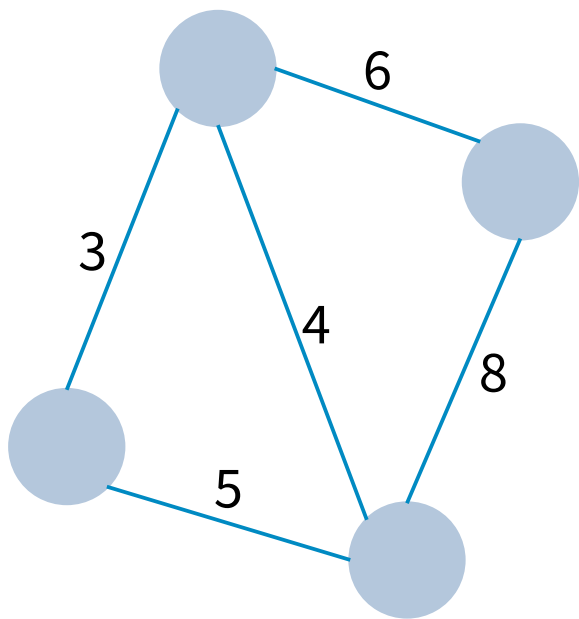
クラスカル法

全ての辺を、コストの小さい順に見ていく



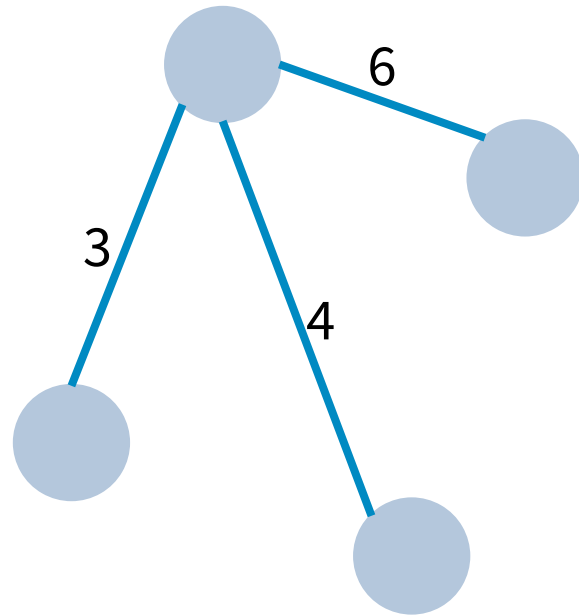
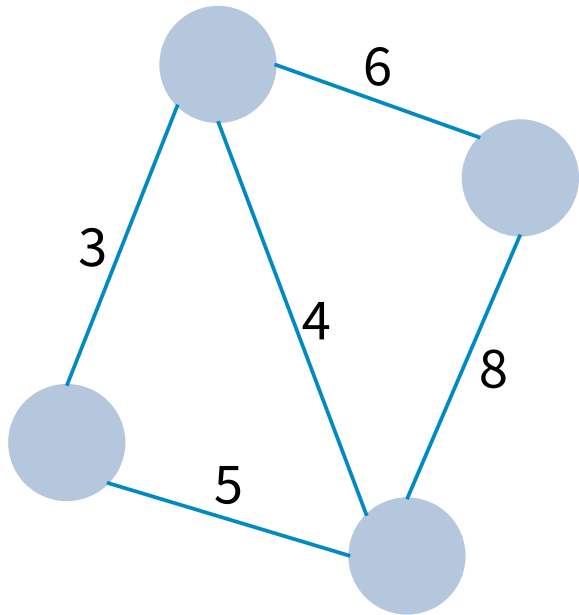
クラスカル法

全ての辺を、コストの小さい順に見ていく



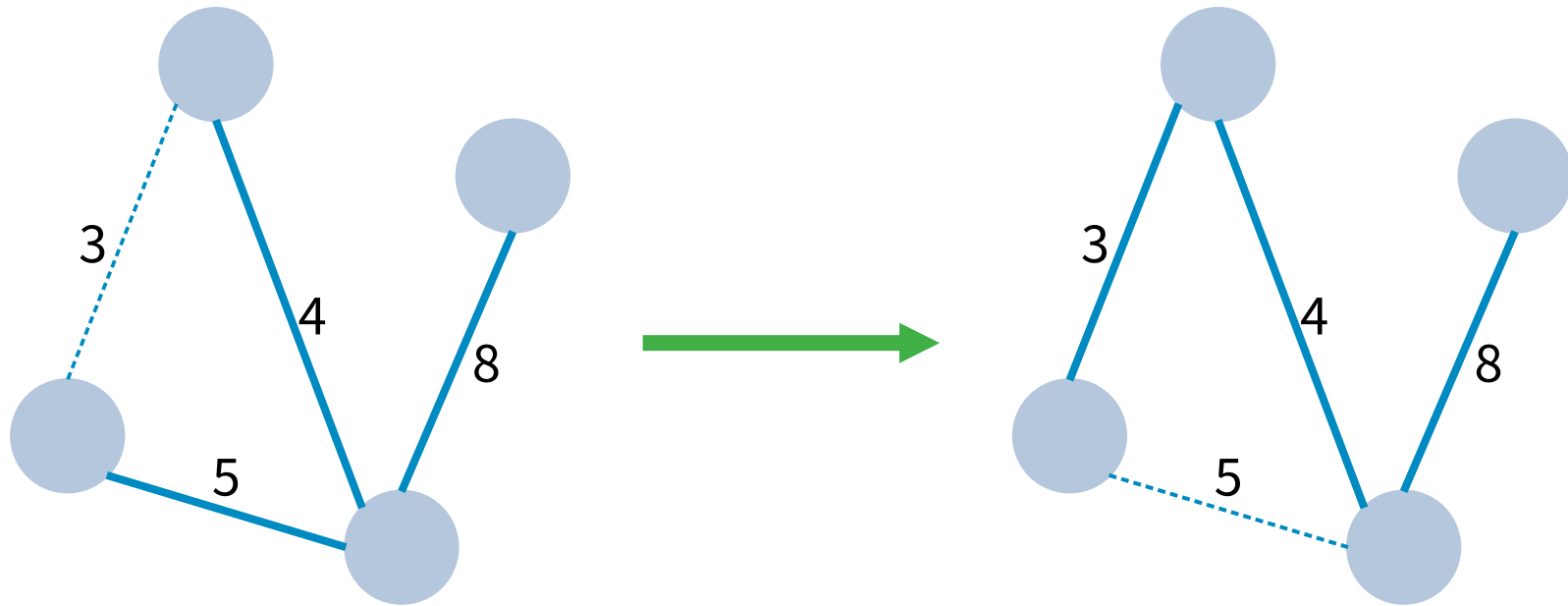
クラスカル法

全ての辺を、コストの小さい順に見ていく



クラスカル法 証明

なぜこれでよいのか
最小コストの辺を使わないのが最小だったとすると ...

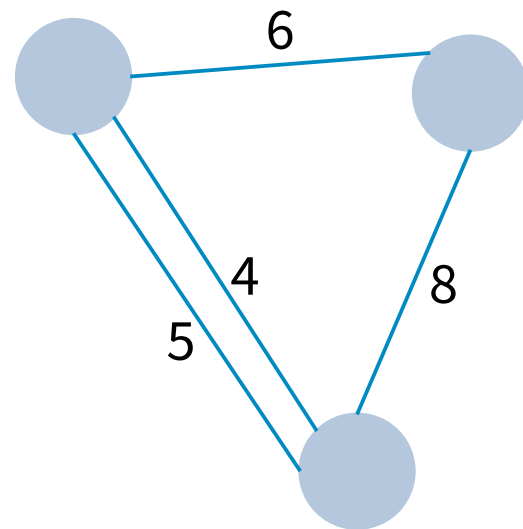
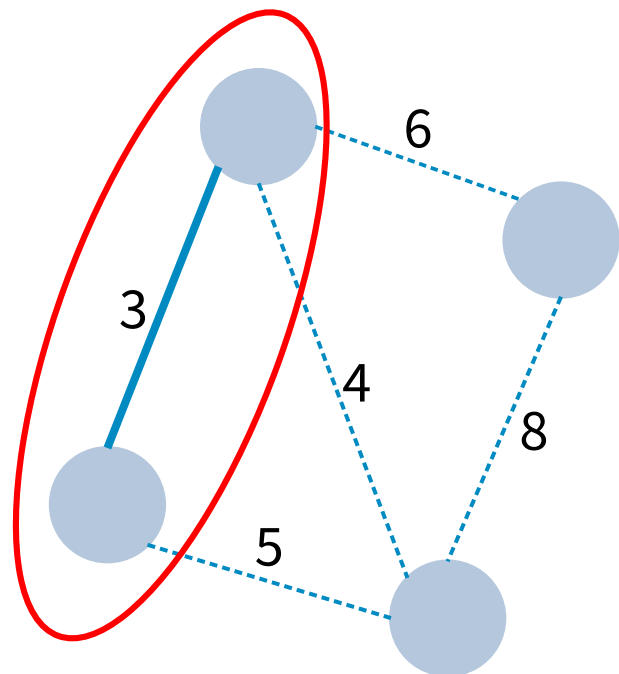


最小コストの辺を使っても損をしない

クラスカル法 証明

なぜこれでよいのか

最小コストの辺を使うことにしたので縮約して同じことを繰り返す
→ クラスカル法と同じことをしている



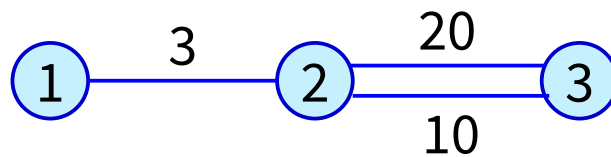
小課題 2

$$Q \leq 10$$

各クエリで各辺のコストを普通に計算 $\rightarrow$ クラスカル法
 $O(QM\log(M))$

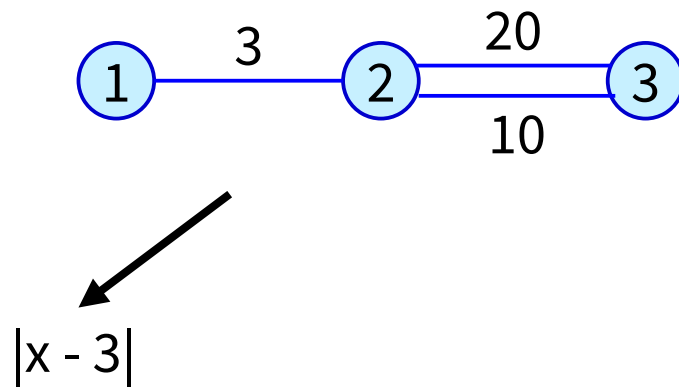
小課題 3

$$B_i = A_i + 1$$



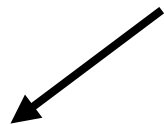
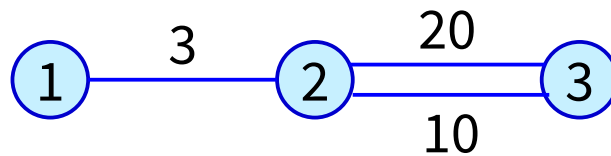
小課題 3

$$B_i = A_i + 1$$



小課題 3

$$B_i = A_i + 1$$

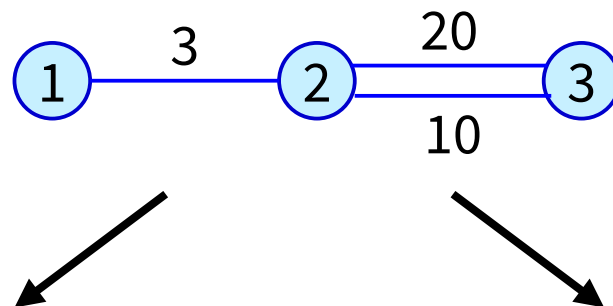


$$x < 3 \rightarrow 3 - x$$

$$x \geq 3 \rightarrow x - 3$$

小課題 3

$$B_i = A_i + 1$$



$$x < 3 \rightarrow 3 - x$$

$$x \geq 3 \rightarrow x - 3$$

$$x \leq 10 \rightarrow 10 - x$$

$$10 \leq x < 15 \rightarrow x - 10$$

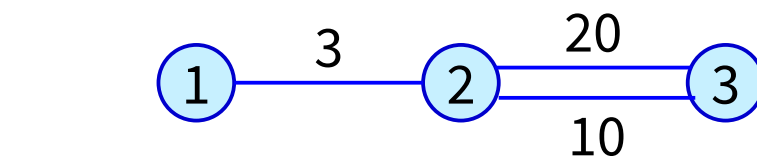
$$15 \leq x < 20 \rightarrow 20 - x$$

$$20 \leq x \rightarrow x - 20$$

コストは x の範囲ごとに x の 1 次関数

小課題 3

$$B_i = A_i + 1$$



$$\begin{aligned} x < 3 &\rightarrow 3 - x \\ x \geq 3 &\rightarrow x - 3 \end{aligned}$$

$+ (2x - 6)$

$$\begin{aligned} x &\leq 10 \rightarrow 10 - x \\ 10 &\leq x < 15 \rightarrow x - 10 \\ 15 &\leq x < 20 \rightarrow 20 - x \\ 20 &\leq x \rightarrow x - 20 \end{aligned}$$

1 次関数は変化する

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$



小課題 3

最初 : $13 - 2x$

$x = 1$



$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 5$ →

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 18$ →

$x = 20 : +(2x - 40)$

小課題 3

$$B_i = A_i + 1$$

全体のコストの 1 次式が変化するタイミングを列挙 & ソート
→ クエリを昇順に処理

$$O(M \log(M) + Q)$$

小課題 4,5

小課題 4 : $M \leq 1000$

小課題 5 : $Q \leq 20000$

毎クエリ全ての辺のコストを計算しては間に合わない
→ 視点を変える

小課題 4,5

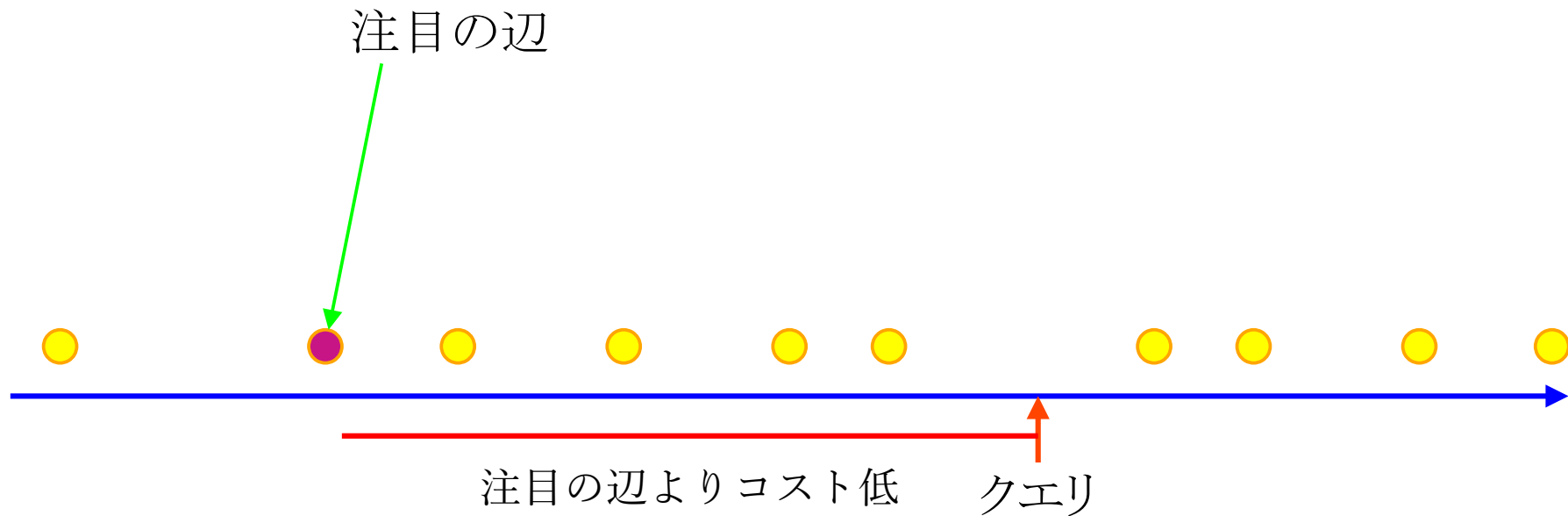
小課題 4 : $M \leq 1000$

小課題 5 : $Q \leq 20000$

毎クエリ全ての辺のコストを計算しては間に合わない
→ 視点を変える

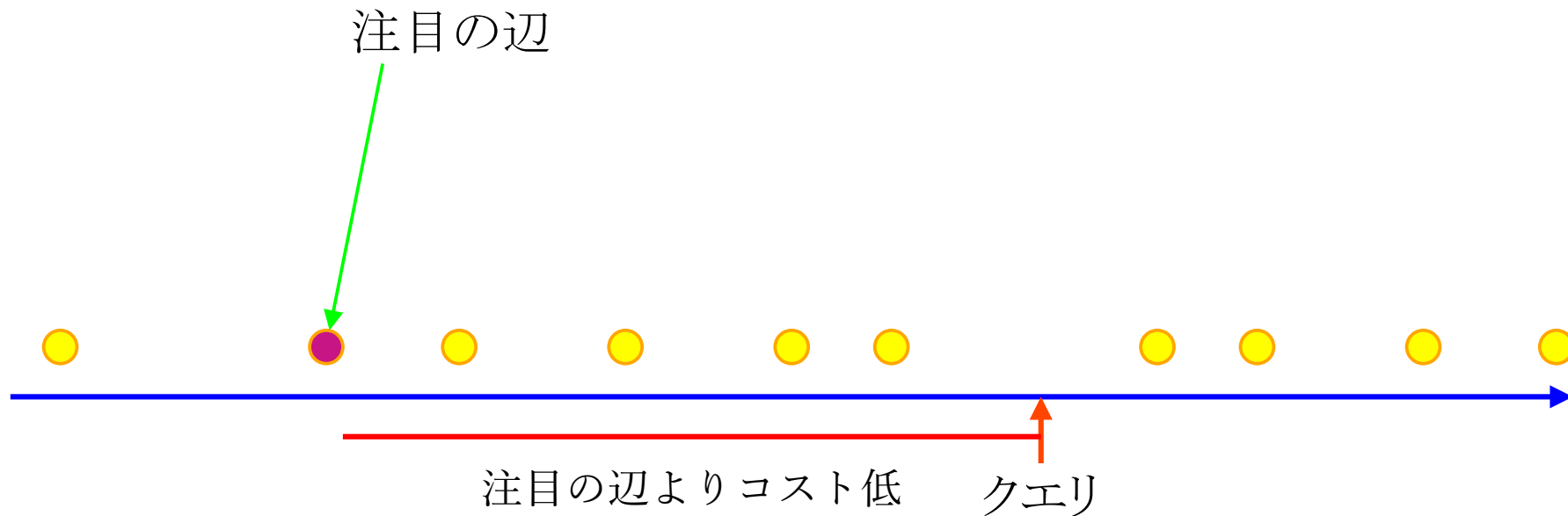
ある辺がクラスカル法で使われる可能性があるのは x がどんな値のときか？

考察



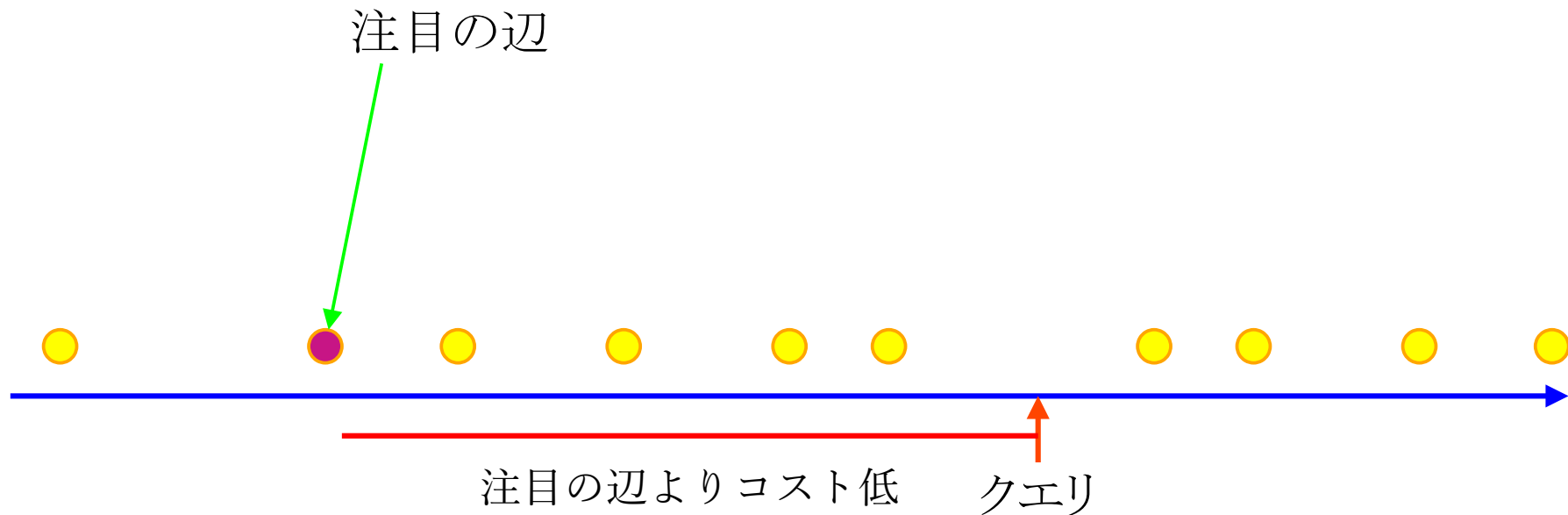
→ 注目の辺が頂点 a, b を繋いでいるとして、赤線だけで a, b が連結ならば注目の辺は使われない

考察



→ 注目の辺より右にある辺を順に追加していって a, b が連結になったら、そこより右にあるクエリでは注目の辺は使われない

考察



→ 注目の辺より右にある辺を順に追加していったら、a,b が連結になったら、そこより右にあるクエリでは注目の辺は使われない

左右で同じことをする

考察

各辺についてそれが使われる可能性のある x の範囲を求めるのを愚直にやって $O(M^2 \alpha(N))$

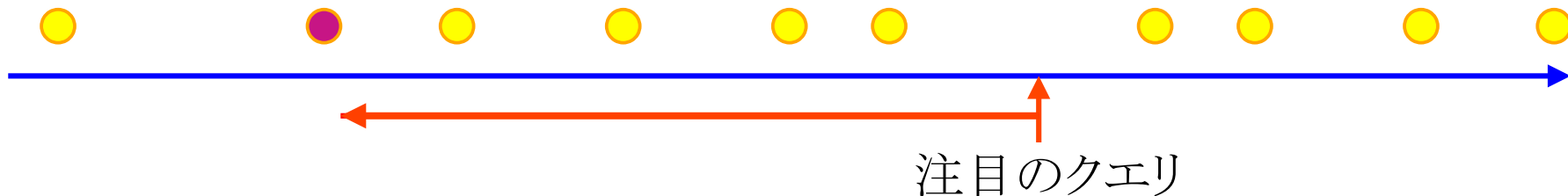
各クエリで絞った $2(N - 1)$ 本をソートしてクラスカル
→ $O(QN \log(N))$

小課題 4 ($M \leq 1000$) では $QN \log(N)$, 小課題 5 ($Q \leq 20000$) では $M^2 \alpha(N)$ が大きすぎる

小課題 5

$$Q \leq 20000$$

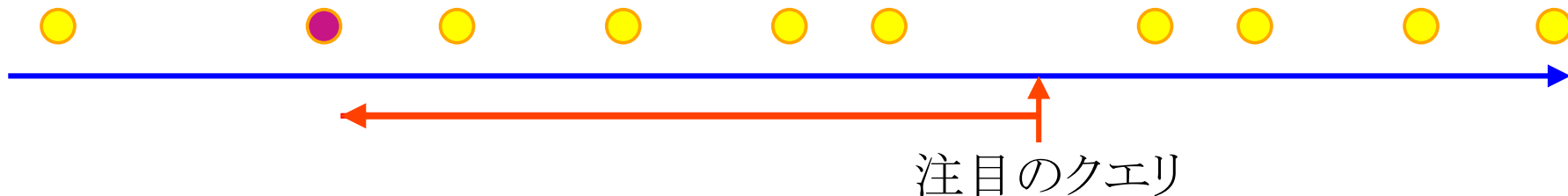
実は先ほどの解法はそのまま通る
クエリ側から使う可能性のある辺の数を考える



小課題 5

$Q \leq 20000$

実は先ほどの解法はそのまま通る
クエリ側から使う可能性のある辺の数を考える

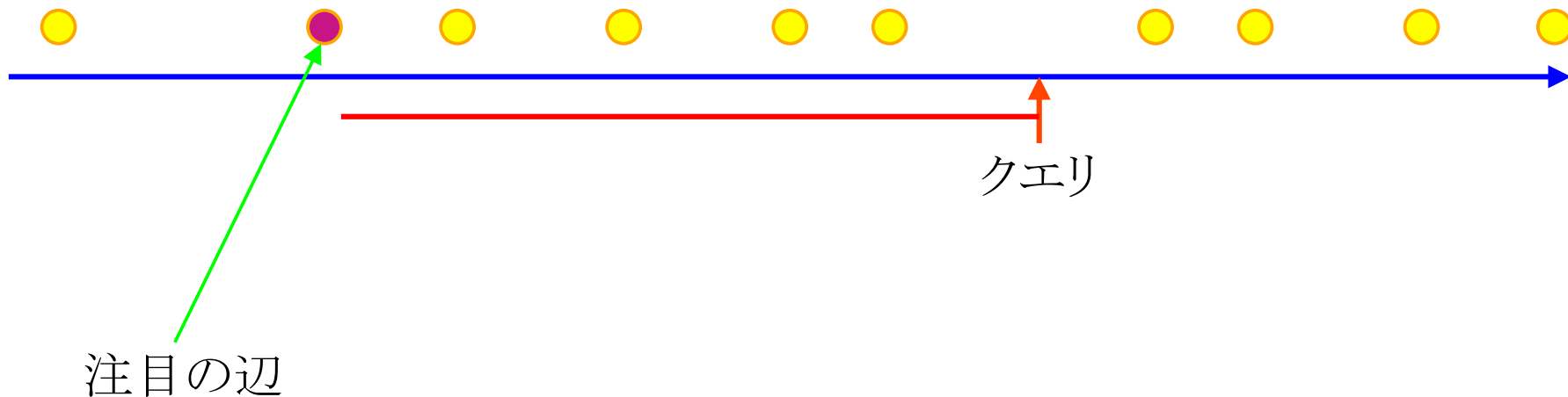


オレンジの範囲の辺で a, b が非連結 → オレンジの矢印の向きにクラスカル法をすると注目の辺は使われる
→ 各クエリ左右それぞれ $N - 1$ 本以下

小課題 5

同じ辺と辺の区間にあるクエリを同一視すると使われ得る辺とクエリの組は合計 $O(MN)$ 個

→ 実は先ほどの解法はそのまま通る ($O(MN \alpha(N) + QN \log(N))$)



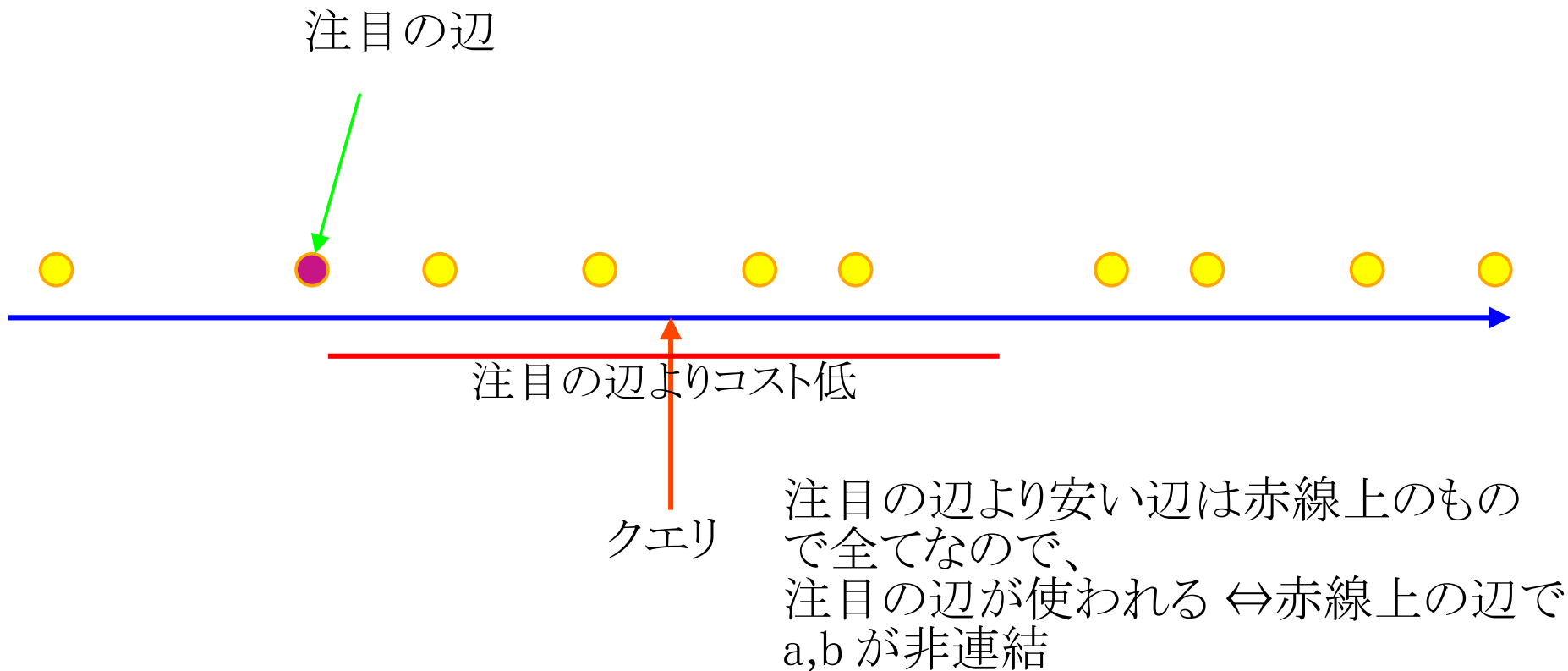
小課題 4

$$M \leq 1000$$

各クエリで $2(N - 1)$ 本の候補となる辺でクラスカル法を行う
部分 ($O(QN \log(N))$) が間に合わない
→ 「候補」ではなく使う辺を確定できないか

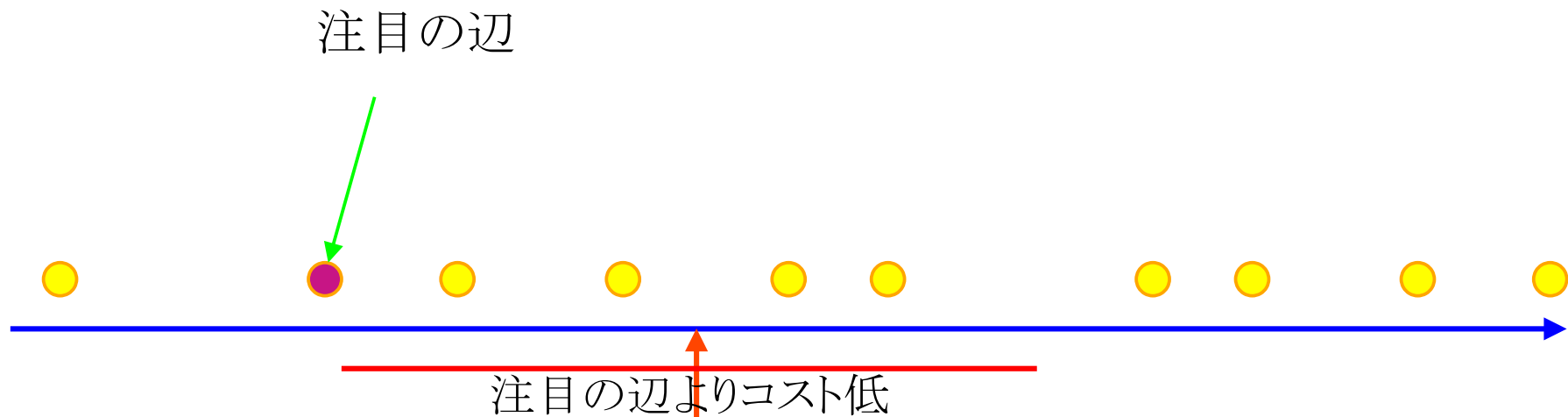
小課題 4

$M \leq 1000$



小課題 4

$M \leq 1000$



クエリ

a,b が連結にならないように限界まで赤線を伸ばしたときの半分のところが、注目の辺が使われるかどうかの境界

小課題 4

$$M \leq 1000$$

前処理 : $O(M^2 \alpha(N))$

クエリ処理 : $O(QN)$ or $O(M + Q \log(N))$

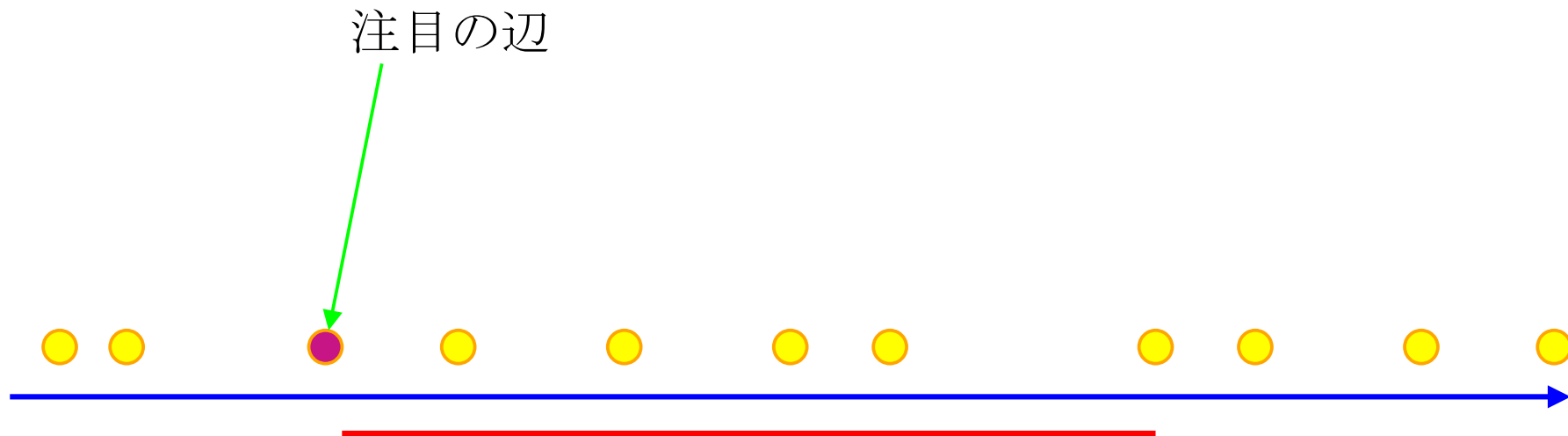
小課題 6 (満点)

追加制約なし

前処理 : $O(M^2 \alpha(N)) \rightarrow$ 実は $O(MN \alpha(N))$

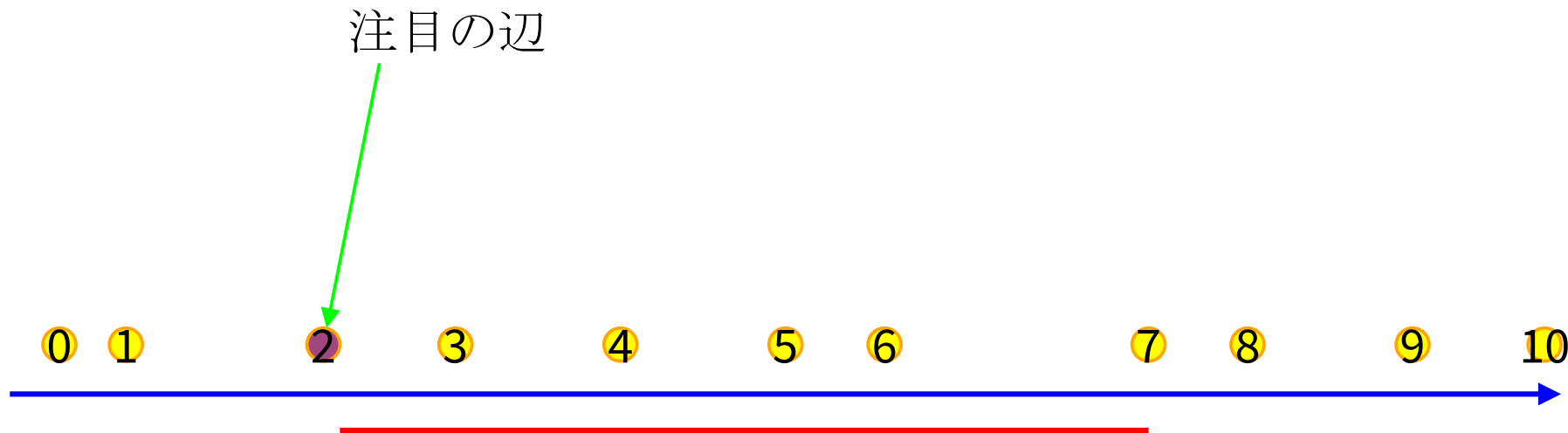
クエリ処理 : $O(QN)$ or $O(M + Q \log(N))$

小課題 6 (満点) 別解



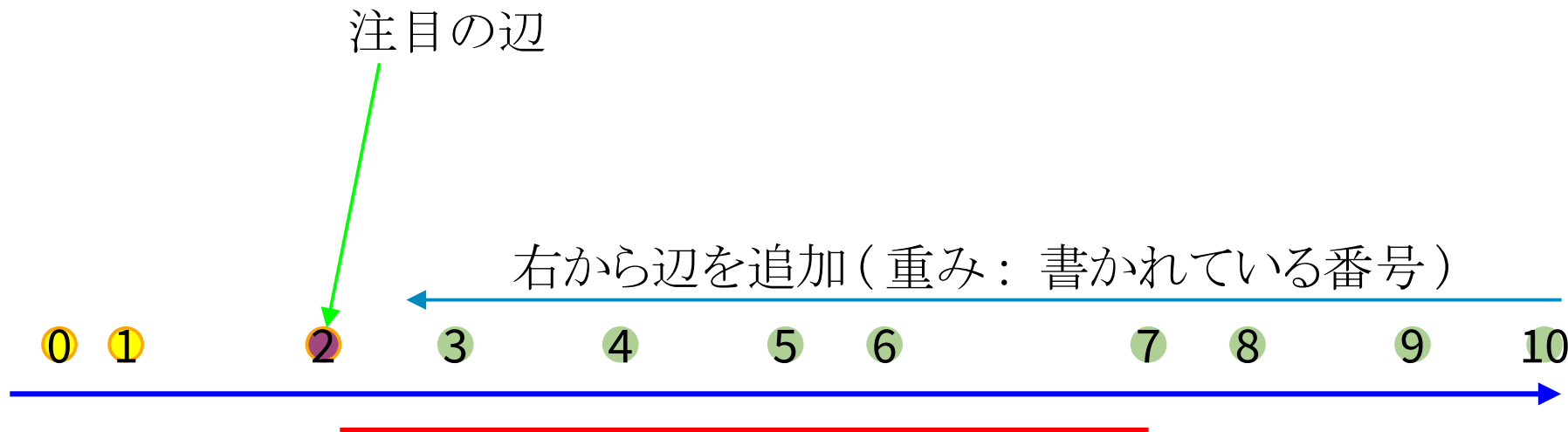
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



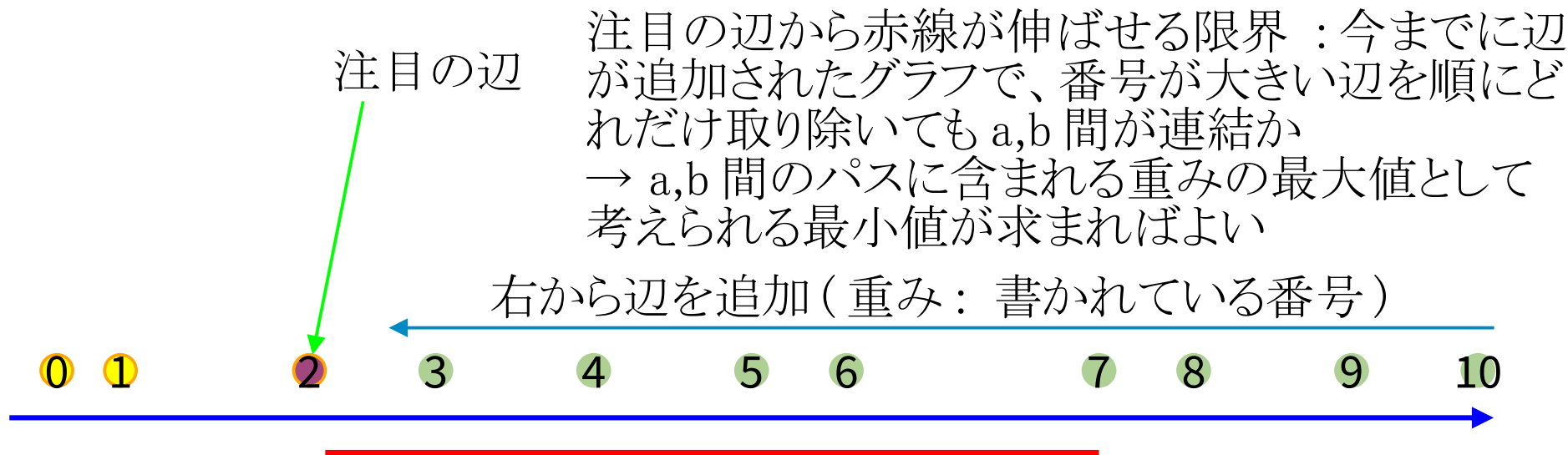
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



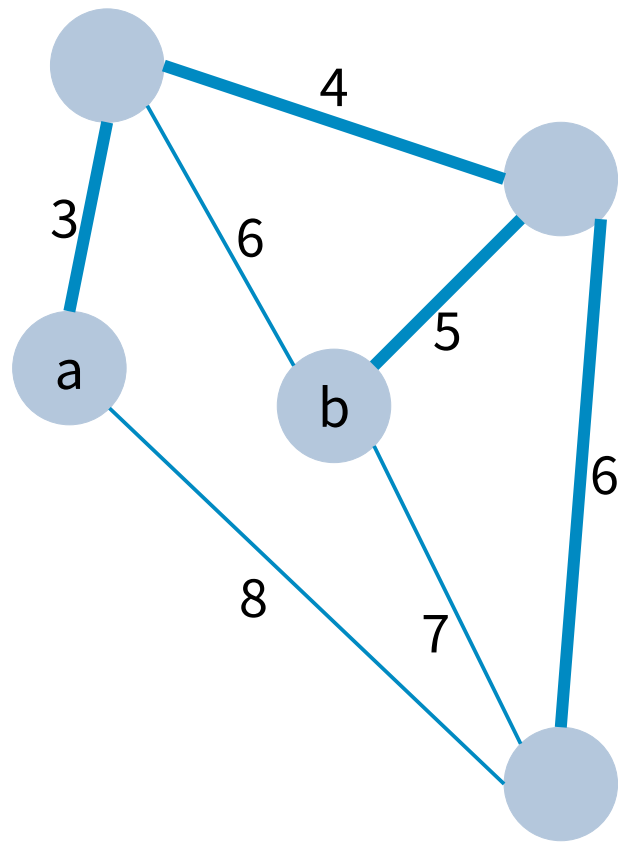
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



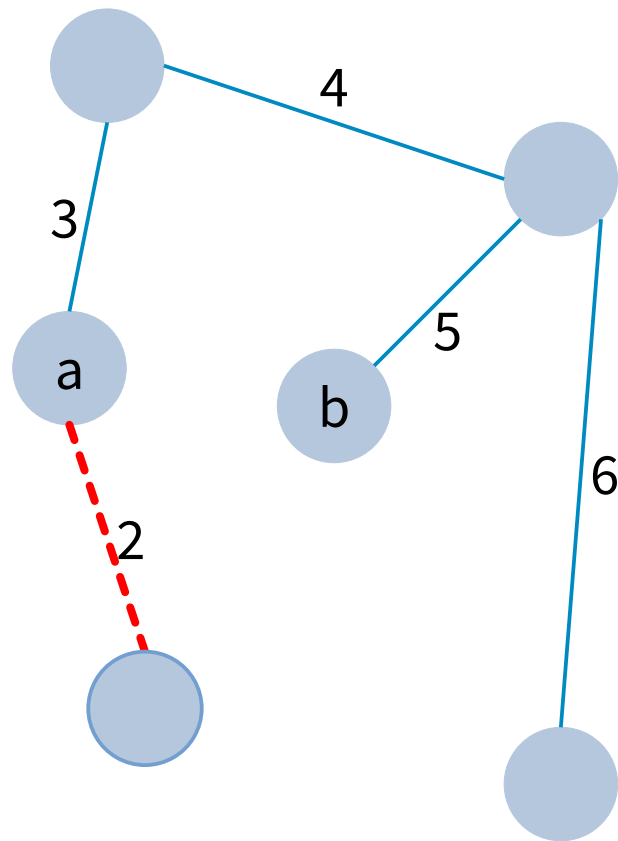
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



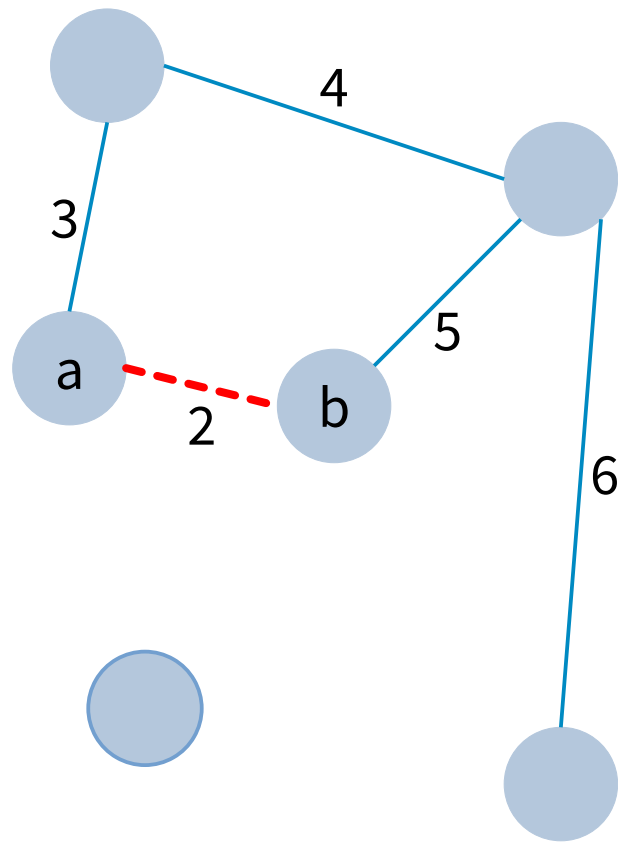
a,b 間のパスに含まれる重みの最大値として考えられる最小値：
グラフの最小全域木上での a,b の最短パス (1 つだけ) が最適であることが証明できる ([1])
→ 辺の追加ごとに最小全域木を保持できればよい

小課題 6 (満点) 別解



辺を追加して最小全域木
を保持する
繋ぐ頂点が非連結のとき
→ 繋ぐだけ

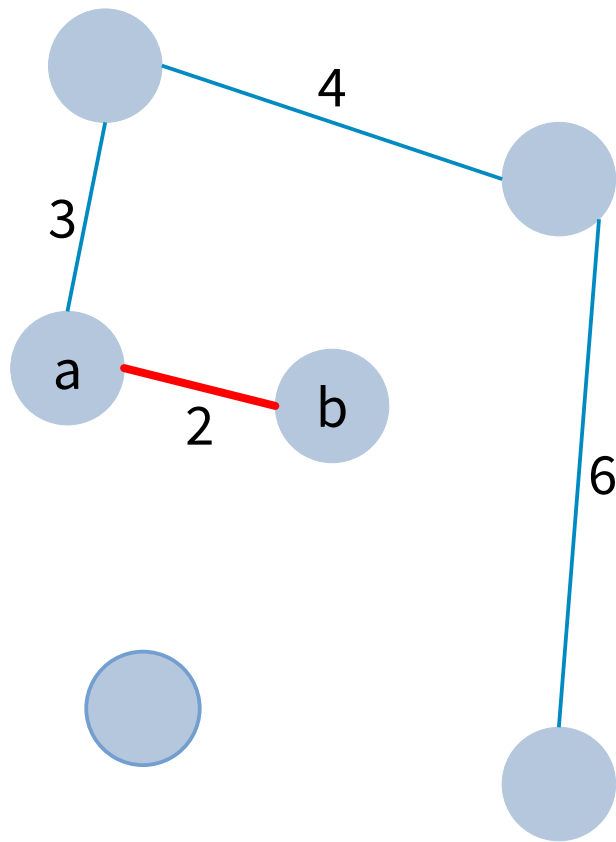
小課題 6 (満点) 別解



辺を追加して最小全域木を
保持する

繋ぐ頂点が連結のとき：
辺は右から追加しているので追
加する辺の重みは他のどの辺
の重みよりも小さい
→ ループの中で重み最大のも
のを消す

小課題 6 (満点) 別解



辺を追加して最小全域木を
保持する

繋ぐ頂点が連結のとき：
辺は右から追加しているので追
加する辺の重みは他のどの辺
の重みよりも小さい
→ ループの中で重み最大のも
のを消す

小課題 6 (満点) 別解

重み付き森に対して

- 指定した 2 頂点のパス上で重みが最大の辺の特定
- 辺追加 (閉路はできない)
- 辺削除

この問題では各 $O(N)$ で良いので簡単 $\rightarrow$ 合計 $O(MN + Q)$

小課題 6 (満点) 別解

重み付き森に対して

- 指定した 2 頂点のパス上で重みが最大の辺の特定
- 辺追加 (閉路はできない)
- 辺削除

~~この問題では各 $O(N)$ で良いので簡単 $\rightarrow$ 合計 $O(MN + Q)$~~

もっと減らせる

Link-Cut Tree 等で各 $O(\log(N)) \rightarrow$ 合計 $O(N + M\log(N) + Q)$

参考文献

[1] "Widest path problem", Wikipedia

https://en.wikipedia.org/wiki/Widest_path_problem